

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Dessy Nurulita NIM. 2310817220024

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN WEB II

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Dessy Nurulita
NIM : 2310817220024

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	5
DAFTAR TABEL.....	7
MODUL 1 : Android Basic with Kotlin	9
SOAL 1	9
A. Source Code	11
B. Output Program.....	17
C. Pembahasan.....	18
MODUL 2 : Android Layout	24
SOAL 1	24
A. Source Code	25
B. Output Program.....	32
C. Pembahasan.....	34
MODUL 3 : Build a Scrollable List.....	45
SOAL 1	45
A. Source Code	47
B. Output Program.....	65
C. Pembahasan.....	67
SOAL 2	80
MODUL 4 : ViewModel and Debugging	81
SOAL 1	81
A. Source Code	81
B. Output Program.....	101
C. Pembahasan.....	106
SOAL 2	125
MODUL 5 : Connect To the Internet.....	126

SOAL 1	126
A. Source Code	126
B. Output Program.....	154
C. Pembahasan.....	159
Tautan Git.....	180

DAFTAR GAMBAR

Gambar 1. Soal 1 Modul 1 (Tampilan Awal Aplikasi).....	9
Gambar 2. Soal 1 Modul 1 (Tampilan Dadu Setelah Di Roll).....	10
Gambar 3. Soal 1 Modul 1 (Tampilan Roll Dadu Double).....	11
Gambar 4. Screenshot Hasil Jawaban Soal 1 Modul 1	17
Gambar 5. Screenshot Hasil Jawaban Soal 1 Modul 1	17
Gambar 6. Screenshot Hasil Jawaban Soal 1 Modul 1	18
Gambar 7. Soal 1 Modul 2 (Tampilan Awal Aplikasi).....	24
Gambar 8. Soal 1 Modul 2 (Tampilan Aplikasi Setelah Dijalankan)	25
Gambar 9. Screenshot Hasil Jawaban Soal 1 Modul 2 (Tampilan Awal Aplikasi)	32
Gambar 10. Screenshot Hasil Jawaban Soal 1 Modul 2 (Tampilan Toast Error)	33
Gambar 11. Screenshot Hasil Jawaban Soal 1 (Hasil Setelah Dihitung).....	34
Gambar 12. Screenshot Hasil Jawaban Soal 1 (Tampilan Aplikasi Layar Dirotate) ..	34
Gambar 13. Soal 1 Modul 3 (Contoh UI List)	46
Gambar 14. Soal 1 Modul 3 (Contoh UI Detail).....	46
Gambar 15. Screenshot Hasil Jawaban Soal 1 Modul 3 (List Portrait)	65
Gambar 16. Screenshot Hasil Jawaban Soal 1 Modul 3 (Detail Portrait).....	66
Gambar 17. Screenshot Hasil Jawaban Soal 1 Modul 3 (List Landscape)	67
Gambar 18. Screenshot Hasil Jawaban Soal 1 Modul 3 (Detail Landscape).....	67
Gambar 19. Screenshot Hasil Jawaban Soal 1 Modul 4 (List Portrait)	101
Gambar 20. Screenshot Hasil Jawaban Soal 1 Modul 4 (Detail Portrait).....	102
Gambar 21. Screenshot Hasil Jawaban Soal 1 Modul 4 (List Landscape)	103
Gambar 22. Screenshot Hasil Jawaban Soal 1 Modul 4 (Detail Landscape).....	103
Gambar 23. Screenshot Hasil Jawaban Soal 1 Modul 4 (Step Into)	104
Gambar 24. Screenshot Hasil Jawaban Soal 1 Modul 4 (Step Out).....	104
Gambar 25. Screenshot Hasil Jawaban Soal 1 Modul 4 (Step Over).....	105
Gambar 26. Screenshot Hasil Jawaban Soal 1 Modul 4 (Logcat).....	105
Gambar 27. Screenshot Hasil Jawaban Soal 1 Modul 4 (Validasi Logcat)	106
Gambar 28. Screenshot Hasil Jawaban Soal 1 Modul 5 (Home Light Mode).....	154
Gambar 29. Screenshot Hasil Jawaban Soal 1 Modul 5 (Detail Light Mode).....	155

Gambar 30. Screenshot Hasil Jawaban Soal 1 Modul 5 (Home Dark Mode).....	156
Gambar 31. Screenshot Hasil Jawaban Soal 1 Modul 5 (Detail Dark Mode).....	157
Gambar 32. Screenshot Hasil Jawaban Soal 1 Modul 5 (Website).....	158

DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1 Modul 1 (MainActivity.kt)	11
Tabel 2. Source Code Jawaban Soal 1 Modul 1 (activity_main.xml).....	14
Tabel 3. Source Code Jawaban Soal 1 Modul 1 (result_background.xml).....	16
Tabel 4. Source Code Jawaban Soal 1 Modul 2 (MainActivity.kt)	25
Tabel 5. Source Code Jawaban Soal 1 Modul 2 (TipCalculatorApp.kt).....	26
Tabel 6. Source Code Jawaban Soal 1 Modul 2 (TipViewModel.kt)	31
Tabel 7. Source Code Jawaban Soal 1 Modul 3 (MainActivity.kt)	47
Tabel 8. Source Code Jawaban Soal 1 Modul 3 (Constellation.kt)	48
Tabel 9. Source Code Jawaban Soal 1 Modul 3 (ConstellationData.kt).....	49
Tabel 10. Source Code Jawaban Soal 1 Modul 3 (HomeFragment.kt).....	51
Tabel 11. Source Code Jawaban Soal 1 Modul 3 (DetailFragment.kt).....	53
Tabel 12. Source Code Jawaban Soal 1 Modul 3 (ListConstellationsAdapter.kt).....	55
Tabel 13. Source Code Jawaban Soal 1 Modul 3 (activity_main.xml).....	56
Tabel 14. Source Code Jawaban Soal 1 Modul 3 (fragment_home.xml)	57
Tabel 15. Source Code Jawaban Soal 1 Modul 3 (fragment_detail.xml)	58
Tabel 16. Source Code Jawaban Soal 1 Modul 3 (star_item.xml).....	59
Tabel 17. Source Code Jawaban Soal 1 Modul 3 (nav_graph.xml).....	62
Tabel 18. Source Code Jawaban Soal 1 Modul 3 (styles.xml).....	64
Tabel 19. Source Code Jawaban Soal 1 Modul 4 (MainActivity.kt)	81
Tabel 20. Source Code Jawaban Soal 1 Modul 4 (Constellation.kt)	83
Tabel 21. Source Code Jawaban Soal 1 Modul 4 (ConstellationData.kt).....	83
Tabel 22. Source Code Jawaban Soal 1 Modul 4 (HomeFragment.kt).....	86
Tabel 23. Source Code Jawaban Soal 1 Modul 4 (DetailFragment.kt).....	88
Tabel 24. Source Code Jawaban Soal 1 Modul 4 (ListConstellationsAdapter.kt).....	89
Tabel 25. Source Code Jawaban Soal 1 Modul 4 (activity_main.xml).....	91
Tabel 26. Source Code Jawaban Soal 1 Modul 4 (fragment_home.xml)	92
Tabel 27. Source Code Jawaban Soal 1 Modul 4 (fragment_detail.xml)	93
Tabel 28. Source Code Jawaban Soal 1 Modul 4 (star_item.xml).....	94
Tabel 29. Source Code Jawaban Soal 1 Modul 4 (nav_graph.xml).....	97

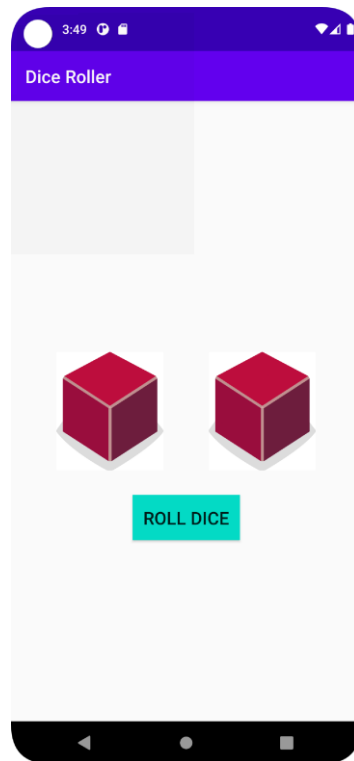
Tabel 30. Source Code Jawaban Soal 1 Modul 4 (styles.xml).....	98
Tabel 31. Source Code Jawaban Soal 1 Modul 4 (MainViewModel.kt)	99
Tabel 32. Source Code Jawaban Soal 1 Modul 4 (MainViewModelFactory.kt)	100
Tabel 34. Source Code Jawaban Soal 1 Modul 5 (ConstellationDao.kt).....	126
Tabel 35. Source Code Jawaban Soal 1 Modul 5 (ConstellationDatabase.kt).....	127
Tabel 36. Source Code Jawaban Soal 1 Modul 5 (ConstellationEntity.kt).....	128
Tabel 37. Source Code Jawaban Soal 1 Modul 5 (ApiResponse.kt)	128
Tabel 37. Source Code Jawaban Soal 1 Modul 5 (Constellation.kt)	129
Tabel 38. Source Code Jawaban Soal 1 Modul 5 (ApiService.kt).....	129
Tabel 39. Source Code Jawaban Soal 1 Modul 5 (RetrofitInstance.kt).....	130
Tabel 40. Source Code Jawaban Soal 1 Modul 5 (ConstellationRepository.kt).....	131
Tabel 41. Source Code Jawaban Soal 1 Modul 5 (ListConstellationAdapter.kt)	132
Tabel 42. Source Code Jawaban Soal 1 Modul 5 (DetailFragment.kt).....	134
Tabel 43. Source Code Jawaban Soal 1 Modul 5 (HomeFragment.kt).....	136
Tabel 44. Source Code Jawaban Soal 1 Modul 5 (MainActivity.kt)	138
Tabel 45. Source Code Jawaban Soal 1 Modul 5 (Resource.kt).....	141
Tabel 46. Source Code Jawaban Soal 1 Modul 5 (MainViewModel.kt)	141
Tabel 47. Source Code Jawaban Soal 1 Modul 5 (MainViewModelFactory.kt)	142
Tabel 48. Source Code Jawaban Soal 1 Modul 5 (AndroidAPIApp.kt).....	143
Tabel 49. Source Code Jawaban Soal 1 Modul 5 (activity_main.xml).....	144
Tabel 50. Source Code Jawaban Soal 1 Modul 5 (fragment_detail.xml)	145
Tabel 51. Source Code Jawaban Soal 1 Modul 5 (fragment_home.xml)	146
Tabel 52. Source Code Jawaban Soal 1 Modul 5 (star_item.xml).....	147
Tabel 53. Source Code Jawaban Soal 1 Modul 5 (menu_theme.xml).....	151
Tabel 54. Source Code Jawaban Soal 1 Modul 5 (navigation_graph.xml).....	151
Tabel 55. Source Code Jawaban Soal 1 Modul 5 (themes.xml)	152
Tabel 56. Source Code Jawaban Soal 1 Modul 5 (themes.xml (night))	153

MODUL 1 : Android Basic with Kotlin

SOAL 1

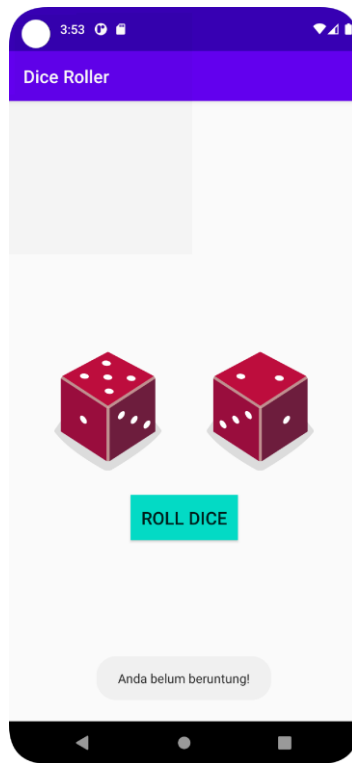
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



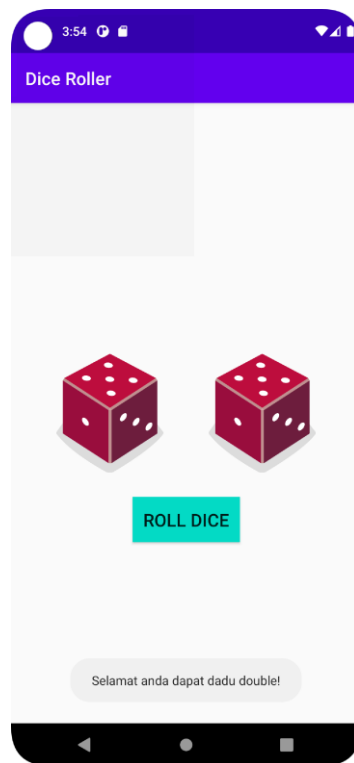
Gambar 1. Soal 1 Modul 1 (Tampilan Awal Aplikasi)

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Soal 1 Modul 1 (Tampilan Dadu Setelah Di Roll)

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
5. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2lIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3. Soal 1 Modul 1 (Tampilan Roll Dadu Double)

A. Source Code

1. MainActivity.kt

Tabel 1. Source Code Jawaban Soal 1 Modul 1 (MainActivity.kt)

1	package com.example.mydicerollerapp
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import
6	com.example.mydicerollerapp.databinding.ActivityMainBinding
7	
8	class MainActivity : AppCompatActivity() {
9	private lateinit var binding: ActivityMainBinding
10	private var lastDice1 = 0
11	private var lastDice2 = 0
12	

```

13         override fun onCreate(savedInstanceState: Bundle?) {
14             super.onCreate(savedInstanceState)
15
16             binding = ActivityMainBinding.inflate(layoutInflater)
17             setContentView(binding.root)
18
19
20             binding.resultText.setBackgroundResource(R.drawable.result_back
21             ground)
22
23             lastDice1 = savedInstanceState?.getInt("DICE1") ?: 0
24             lastDice2 = savedInstanceState?.getInt("DICE2") ?: 0
25
26             if (lastDice1 != 0 && lastDice2 != 0) {
27
28                 binding.diceImage1.setImageResource(getDiceDrawable(lastDice1))
29
30                 binding.diceImage2.setImageResource(getDiceDrawable(lastDice2))
31                 binding.resultText.text = if (lastDice1 ==
32                 lastDice2) {
33                     "Selamat anda dapat dadu double!"
34                 } else {
35                     "Anda belum beruntung!"
36                 }
37             }
38
39             binding.rollButton.setOnClickListener {
40                 val randomInt1 = (1..6).random()
41                 val randomInt2 = (1..6).random()
42
43                 lastDice1 = randomInt1
44                 lastDice2 = randomInt2
45

```

```

46
47 binding.diceImage1.setImageResource(getDiceDrawable(randomInt1)
48 )
49
50 binding.diceImage2.setImageResource(getDiceDrawable(randomInt2)
51 )
52
53         binding.resultText.text = if (randomInt1 ==
54 randomInt2) {
55             "Selamat anda dapat dadu double!"
56         } else {
57             "Anda belum beruntung!"
58         }
59     }
60 }
61
62 override fun onSaveInstanceState(outState: Bundle) {
63     super.onSaveInstanceState(outState)
64     outState.putInt("DICE1", lastDice1)
65     outState.putInt("DICE2", lastDice2)
66 }
67
68 private fun getDiceDrawable(number: Int): Int {
    return when (number) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        6 -> R.drawable.dice_6
        else -> R.drawable.dice_0
    }
}

```

	} }
--	--------

2. activity_main.xml

Tabel 2. Source Code Jawaban Soal 1 Modul 1 (activity_main.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<TextView
11	android:id="@+id/headerTitle"
12	android:layout_width="0dp"
13	android:layout_height="wrap_content"
14	android:text="Dice Roller"
15	android:textStyle="bold"
16	android:textColor="#FFFFFF"
17	android:textSize="20sp"
18	android:background="#6200EE"
19	android:padding="16dp"
20	android:gravity="left top"
21	app:layout_constraintTop_toTopOf="parent"
22	app:layout_constraintStart_toStartOf="parent"
23	app:layout_constraintEnd_toEndOf="parent" />
24	
25	<ImageView
26	android:id="@+id/diceImage1"
27	android:layout_width="100dp"
28	android:layout_height="100dp"

```

29         android:src="@drawable/dice_0"
30         app:layout_constraintTop_toBottomOf="@id/headerTitle"
31         app:layout_constraintEnd_toStartOf="@id/diceImage2"
32         app:layout_constraintStart_toStartOf="parent"
33         app:layout_constraintBottom_toTopOf="@id/rollButton"
34         app:layout_constraintHorizontal_chainStyle="packed" />
35
36     <ImageView
37         android:id="@+id/diceImage2"
38         android:layout_width="100dp"
39         android:layout_height="100dp"
40         android:src="@drawable/dice_0"
41         app:layout_constraintTop_toBottomOf="@id/headerTitle"
42         app:layout_constraintStart_toEndOf="@id/diceImage1"
43         app:layout_constraintEnd_toEndOf="parent"
44         app:layout_constraintBottom_toTopOf="@id/rollButton" />
45
46     <Button
47         android:id="@+id/rollButton"
48         android:layout_width="wrap_content"
49         android:layout_height="wrap_content"
50         android:text="ROLL DICE"
51         app:layout_constraintTop_toBottomOf="@id/diceImage1"
52         app:layout_constraintStart_toStartOf="parent"
53         app:layout_constraintEnd_toEndOf="parent"
54         app:layout_constraintBottom_toTopOf="@id/resultText" />
55
56     <TextView
57         android:id="@+id/resultText"
58         android:layout_width="wrap_content"
59         android:layout_height="wrap_content"
60         android:text=""
61         android:textColor="#000000"

```

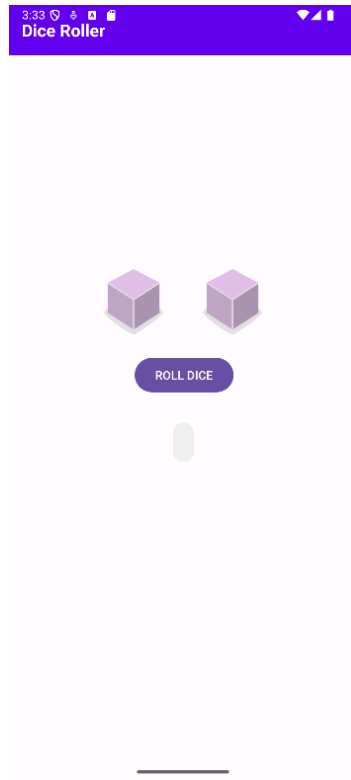
62	android:textSize="16sp"
63	android:padding="12dp"
64	android:background="@drawable/result_background"
65	app:layout_constraintTop_toBottomOf="@id/rollButton"
66	app:layout_constraintStart_toStartOf="parent"
67	app:layout_constraintEnd_toEndOf="parent"
68	app:layout_constraintBottom_toBottomOf="parent" />
69	</androidx.constraintlayout.widget.ConstraintLayout>

3. result_background.xml

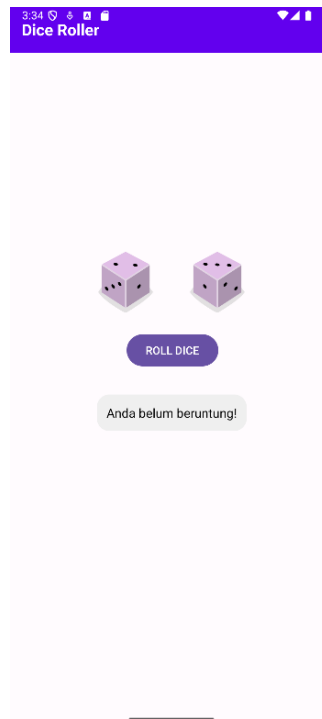
Tabel 3. Source Code Jawaban Soal 1 Modul 1 (result_background.xml)

1	<shape xmlns:android="http://schemas.android.com/apk/res/android"
2	android:shape="rectangle">
3	<solid android:color="#EFEFEF" />
4	<corners android:radius="16dp" />
5	</shape>

B. Output Program



Gambar 4. Screenshot Hasil Jawaban Soal 1 Modul 1



Gambar 5. Screenshot Hasil Jawaban Soal 1 Modul 1



Gambar 6. Screenshot Hasil Jawaban Soal 1 Modul 1

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. MainActivity.kt:

- Pada line 1, dideklarasikan nama package file Kotlin dan menunjukkan nama tempat file ini berada.
- Pada line 3, diimport `Bundle` untuk mengkomunikasikan dan menyimpan data antar aktivitas atau saat activity di-pause atau rotate.
- Pada line 4, diimport `AppCompatActivity` untuk mengatur kelas dasar untuk activity Android modern.
- Pada line 5, diimport `ActivityMainBinding` untuk mengatur akses komponen XML tanpa `findViewById`.
- Pada line 7, terdapat deklarasi `class MainActivity : AppCompatActivity() {` yang menunjukkan class utama yang mewakili tampilan utama aplikasi `DiceRollerApp`.

- Pada line 8, terdapat deklarasi `private lateinit var binding: ActivityMainBinding` yang merupakan variable binding untuk mengakses elemen-elemen di `activity_main.xml` menggunakan `ViewBinding`. Adapun `lateinit` artinya nanti akan di-inisialisasi (di dalam `onCreate`).
- Pada line 9 dan 10, terdapat deklarasi `private var lastDice1 = 0` dan `private var lastDice2 = 0` yang akan menyimpan angka dadu terakhir.
- Pada line 12, didefinisikan fungsi `onCreate`. Fungsi ini dipanggil saat activity pertama kali dibuat. Istilah `override` berarti fungsi ini akan menimpa fungsi bawaan dari `AppCompatActivity`. Parameter `savedInstanceState` bisa berisi data sebelumnya.
- Pada line 13, dipanggil versi `onCreate` dari `AppCompatActivity` agar Android tetap bisa menjalankan proses penting lainnya.
- Pada line 15, terdapat inisialisasi `ViewBinding (binding)`. Kata `ActivityMainBinding.inflate(layoutInflater)` artinya membuat objek binding dari layout XML agar bisa mengakses elemen-elemen di layout.
- Pada line 16, akan mengatur tampilan utama activity ke layout yang sudah di-inflate. `binding.root` adalah elemen paling atas dari `activity_main.xml`.
- Pada line 18, akan mengatur dan menampilkan background untuk `TextView` di pemberitahuan “Selamat anda dapat dadu double” dan “Maaf anda belum beruntung”.
- Pada line 20 dan 21, akan mengambil nilai dadu sebelumnya jika activity di-restore.
- Pada line 23, terdapat kondisional `if` untuk memeriksa dan memastikan bahwa kedua dadu meminili nilai (bukan 0).

- Pada line 24 dan 25, akan menampilkan gambar dadu pertama dan kedua sesuai angka terakhir.
- Pada line 26 hingga 31, terdapat kondisional yang akan membandingkan nilai kedua dadu dari data terakhir yang disimpan. Jika kedua dadu menampilkan angka yang sama, maka akan menampilkan teks "Selamat anda dapat dadu double!". Jika kedua dadu menunjukkan angka yang berbeda, maka akan menampilkan teks "Anda belum beruntung!".
- Pada line 33, terdapat listener tombol roll. Jika tombol ini ditekan oleh user, maka semua blok kode yang ada di dalamnya akan dijalankan.
- Pada line 34 dan 35, terdapat fungsi untuk generate angka acak antara 1 sampai 6 untuk kedua dadu.
- Pada line 37 dan 38, akan menyimpan angka acak dari dadu pertama ke variabel `lastDice1` dan menyimpan angka acak dari kedua ke variabel `lastDice2`.
- Pada line 40 dan 41, akan mengubah angka `randomInt1` jadi gambar dadu, lalu akan ditampilkan di `diceImage1`. Begitu juga dengan baris ke 41 yang akan mengubah angka `randomInt2` jadi gambar dadu, lalu akan ditampilkan di `diceImage2`.
- Pada line 43 hingga 48, terdapat kondisional yang akan membandingkan nilai kedua dadu dari data baru saat user menekan tombol. Jika kedua dadu menampilkan angka yang sama, maka akan menampilkan teks "Selamat anda dapat dadu double!". Jika kedua dadu menunjukkan angka yang berbeda, maka akan menampilkan teks "Anda belum beruntung!".
- Pada line 51, terdapat fungsi yang akan dipanggil saat activity akan dihapus untuk sementara. Data akan disimpan agar tidak hilang.
- Pada line 52, akan memanggil fungsi dari superclass (`AppCompatActivity`) sebagai langkah umum setiap kali ingin override suatu fungsi dari superclass.

- Pada line 53 dan 54, akan menyimpan nilai `lastDice1` ke bundle, dengan key “DICE1”. Begitu juga di baris 54 yang akan menyimpan nilai `lastDice2` ke bundle, dengan key “DICE2”.
- Pada line 57, terdapat fungsi yang menerima angka number, dan mengembalikan resource gambar dadu.
- Pada line 58, terdapat when expression, `return when (number)` untuk memeriksa nilai dari number.
- Pada line 59 hingga 65, terdapat blok kode yang akan dieksekusi saat when memeriksa nilainya dan mengembalikan gambar (drawable) dadu yang sesuai. Jika angkanya 1, maka akan mengembalikan `R.drawable.dice_1`, dan seterusnya. Jika angka bukan 1-6 (angka 0 atau 7), maka akan menjalankan blok kode else, yaitu `R.drawable.dice_0`.

2. activity_main.xml

- Pada line 1, terdapat baris pembuka XML.
- Pada line 2, merupakan deklarasi layout utama yang digunakan (ConstraintLayout).
- Pada line 3 hingga 5, dideklarasikan namespace khusus Android, atribut tambahan (`app:`), dan atribut untuk preview (`tools:`).
- Pada line 6 dan 7, merupakan bagian untuk penyesuaian ukuran layout utama. Dalam hal ini, layout diset agar lebar dan tingginya memenuhi seluruh layar (`match_parent`).
- Pada line 8, terdapat `tools:context=".MainActivity">` yang menunjukkan bahwa layout ini dipakai di MainActivity.
- Pada line 9 hingga 22, merupakan komponen dari TextView atau Judul. Komponen ini akan menampilkan “Dice Roller” sebagai judul di atas layar. Di-set dengan `match_parent` agar lebarnya full dan `wrap_content` agar tingginya secukupnya. Teksnya dibuat berwarna putih, bold, sebesar

20sp, dan memiliki background berwarna ungu. Diletakkan di paling atas layout, sejajar kiri dengan parent.

- Pada line 24 hingga 33 merupakan bagian untuk mengatur gambar dadu pertama. Di line 26 dan 27 mendeklarasikan gambar dadu pertama yang diatur dengan ukuran 100dp, posisinya di sebelah `diceImage2`. Awalnya akan diatur menggunakan gambar `dice_0`.
- Pada line 35 hingga 43 merupakan bagian untuk mengatur gambar dadu kedua. Ukurannya diatur menjadi 100dp, posisi di sebelah kiri. Keduanya muncul secara sejajar, di bawah judul dengan margin atas 240dp. Awalnya akan diatur menggunakan gambar `dice_0`.
- Pada line 45 hingga 53 merupakan bagian untuk tombol "ROLL DICE". Ukurannya disesuaikan dengan isi menggunakan `wrap_content`. Di tengah tombolnya bertuliskan "ROLL DICE". Letaknya diatur di tengah, di bawah gambar dadu.
- Pada line 55 hingga 67 merupakan bagian untuk menampilkan keterangan hasil lemparan. Awalnya dibuat kosong (`text=""`). Bagian ini menyisipkan background dari `result_background` (file shape XML) untuk styling box teks keterangan. Padding diatur sebesar 12dp, teks berwarna hitam dengan ukuran 16sp. Letaknya di bawah tombol roll (`rollButton`).
- Pada line 69, terdapat `</androidx.constraintlayout.widget.ConstraintLayout>` yang menutup tag layout utama `ConstraintLayout`.

3. `result_backgroud.xml`

- Pada line 1 terdapat tag utama yang menyatakan bahwa ini dokumen ini adalah shape drawable. `xmlns:android=...` adalah deklarasi namespace, agar Android mengetahui atribut-atribut di dalamnya.

- Pada line 2 terdapat `android:shape="rectangle">` yang berarti bentuk yang kita buat ini adalah persegi panjang (rectangle).
- Pada line 3 terdapat `<solid android:color="#EFEFEF" />` yang merupakan bagian isi warna dari shapenya. Solid artinya warnanya padat. Adapun `android:color="#EFEFEF"` berarti warna abu-abu terang.
- Pada line 4, `<corners android:radius="16dp" />` digunakan untuk mengatur sudut-sudut shapenya agar jadi membulat. Lalu, `android:radius="16dp"` artinya sudutnya membulat dengan radius 16dp (semakin besar, makin bulat).
- Pada line 5, terdapat `</shape>` yang menutup tag `<shape>` tadi.

Catatan tambahan:

Jika menggunakan ViewBinding, tambahkan

```
viewBinding {
    enable = true
}
```

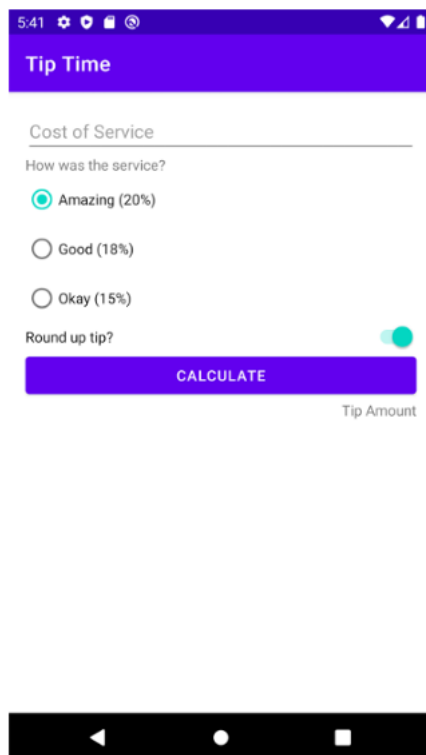
di file `build.gradle (Module: app)` dan tambahkan kode di atas ke dalam `android {}`. Setelah itu, sinkronisasikan ulang.

MODUL 2 : Android Layout

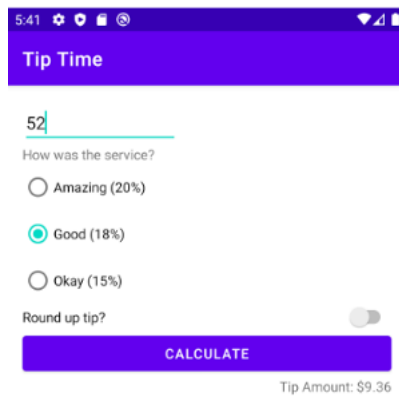
SOAL 1

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 7. Soal 1 Modul 2 (Tampilan Awal Aplikasi)



Gambar 8. Soal 1 Modul 2 (Tampilan Aplikasi Setelah Dijalankan)

A. Source Code

1. MainActivity.kt:

Tabel 4. Source Code Jawaban Soal 1 Modul 2 (MainActivity.kt)

1	package com.example.calculatetipapp
2	package com.example.calculatetipapp
3	
4	import android.os.Bundle
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.compose.material3.Surface
8	import androidx.compose.ui.Modifier
9	import androidx.compose.foundation.layout.fillMaxSize
10	import
11	com.example.calculatetipapp.ui.theme.CalculateTipAppTheme
12	
13	class MainActivity : ComponentActivity() {

14	override fun onCreate(savedInstanceState: Bundle?) {
15	super.onCreate(savedInstanceState)
16	setContent {
17	CalculateTipAppTheme {
18	Surface(
19	modifier = Modifier.fillMaxSize()
20	) {
21	TipCalculatorApp()
22	}
23	}
24	}
	}

2. TipCalculatorApp.kt

Tabel 5. Source Code Jawaban Soal 1 Modul 2 (TipCalculatorApp.kt)

1	package com.example.calculatetipapp
2	
3	import android.widget.Toast
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import androidx.compose.foundation.text.KeyboardOptions
7	import androidx.compose.foundation.verticalScroll
8	import androidx.compose.material3.*
9	import androidx.compose.material3.TopAppBar
10	import androidx.compose.runtime.Composable
11	import androidx.compose.ui.Alignment
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.graphics.Color
14	import androidx.compose.ui.platform.LocalContext
15	import androidx.compose.ui.text.font.FontWeight
16	import androidx.compose.ui.text.input.KeyboardType
17	import androidx.compose.ui.unit.dp

```

18 import androidx.lifecycle.viewmodel.compose.viewModel
19
20 @OptIn(ExperimentalMaterial3Api::class)
21 @Composable
22 fun TipCalculatorApp(tipViewModel: TipViewModel = viewModel())
23 {
24     val context = LocalContext.current
25     val scrollState = rememberScrollState()
26
27     Scaffold(
28         topBar = {
29             TopAppBar(
30                 title = {
31                     Text(
32                         text = "Tip Time",
33                         color = Color.White,
34                         fontWeight = FontWeight.Bold
35                     )
36                 },
37                 colors =
38                 TopAppBarDefaults.smallTopAppBarColors(containerColor =
39                 Color(0xFF6200EE))
40             )
41         }
42     ) { innerPadding ->
43         Column(
44             modifier = Modifier
45                 .padding(innerPadding)
46                 .padding(16.dp)
47                 .fillMaxSize()
48                 .verticalScroll(scrollState) // Aktifkan
49         scroll
50     ) {

```

51	TextField(
52	value = tipViewModel.serviceCostInput,
53	onValueChange = {
54	tipViewModel.serviceCostInput = it },
55	placeholder = {
56	if
57	(tipViewModel.serviceCostInput.isEmpty()) {
58	Text("Cost of Service", color =
59	Color.Gray)
60	}
61	},
62	keyboardOptions =
63	KeyboardOptions.Default.copy(keyboardType =
64	KeyboardType.Number),
65	modifier = Modifier.fillMaxWidth(),
66	colors = TextFieldDefaults.textFieldColors(
67	containerColor = Color.Transparent,
68	)
69	)
70	
71	Spacer(modifier = Modifier.height(16.dp))
72	Text("How was the service?", color = Color.Gray)
73	
74	Spacer(modifier = Modifier.height(8.dp))
75	listOf(
76	"Amazing (20%)" to 20,
77	"Good (18%)" to 18,
78	"Okay (15%)" to 15
79	).forEach { (label, value) ->
80	Row(verticalAlignment =
81	Alignment.CenterVertically) {
82	RadioButton(
83	selected =

```

84 tipViewModel.selectedTipPercent == value,
85         onClick = {
86 tipViewModel.selectedTipPercent = value }
87         )
88         Text(text = label)
89     }
90 }
91
92     Spacer(modifier = Modifier.height(8.dp))
93     Row(
94         modifier = Modifier
95             .fillMaxWidth()
96             .padding(top = 16.dp),
97         verticalAlignment =
98 Alignment.CenterVertically,
99         horizontalArrangement =
100 Arrangement.SpaceBetween
101     ) {
102         Text("Round up tip?")
103         Switch(
104             checked = tipViewModel.roundUp,
105             onChange = { tipViewModel.roundUp =
106 it }
107         )
108     }
109
110     Spacer(modifier = Modifier.height(16.dp))
111     Button(
112         onClick = {
113             val cost =
114 tipViewModel.serviceCostInput.toDoubleOrNull()
115             if (cost == null || cost <= 0.0) {
116                 Toast.makeText(context, "Jangan

```

```

117 masukan nilai 0 atau minus!", Toast.LENGTH_SHORT).show()
118             tipViewModel.showTip = false
119         } else {
120             tipViewModel.onCalculateClicked()
121         }
122     },
123     colors =
124 ButtonDefaults.buttonColors(containerColor =
125 Color(0xFF6200EE)),
126         modifier = Modifier.fillMaxWidth()
127     ) {
128         Text("CALCULATE")
129     }
130
131     if (!tipViewModel.showTip) {
132         Spacer(modifier = Modifier.height(8.dp))
133         Text(
134             text = "Tip Amount",
135             color = Color.Gray,
136             style =
137 MaterialTheme.typography.bodySmall,
138             modifier = Modifier.align(Alignment.End)
139         )
140     }
141
142     if (tipViewModel.showTip) {
143         Spacer(modifier = Modifier.height(24.dp))
144         Text(
145             text = "Tip Amount:
146 \${tipViewModel.calculatedTip}",
147             color = Color.Gray,
148             style =
149 MaterialTheme.typography.bodySmall,

```

	<pre> modifier = Modifier.align(Alignment.End)) } } } } </pre>
--	--

3. TipViewModel.kt

Tabel 6. Source Code Jawaban Soal 1 Modul 2 (TipViewModel.kt)

1	package com.example.calculatetipapp
2	
3	import androidx.compose.runtime.getValue
4	import androidx.compose.runtime.mutableStateOf
5	import androidx.compose.runtime.setValue
6	import androidx.lifecycle.ViewModel
7	import kotlin.math.ceil
8	
9	class TipViewModel : ViewModel() {
10	var serviceCostInput by mutableStateOf("")
11	var selectedTipPercent by mutableStateOf(18)
12	var roundUp by mutableStateOf(false)
13	var showTip by mutableStateOf(false)
14	
15	val calculatedTip: String
16	get() {
17	val cost = serviceCostInput.toDoubleOrNull() ?:
18	return "0.00"
19	var tip = cost * selectedTipPercent / 100
20	if (roundUp) tip = ceil(tip)
21	return String.format("%.2f", tip)
22	}
23	
24	fun onCalculateClicked() {

25	<code>showTip = true</code>
26	<code>}</code>
	<code>}</code>

B. Output Program

Gambar 9. Screenshot Hasil Jawaban Soal 1 Modul 2 (Tampilan Awal Aplikasi)

2:42

Tip Time

0

How was the service?

☐ Amazing (20%)

☒ Good (18%)

☐ Okay (15%)

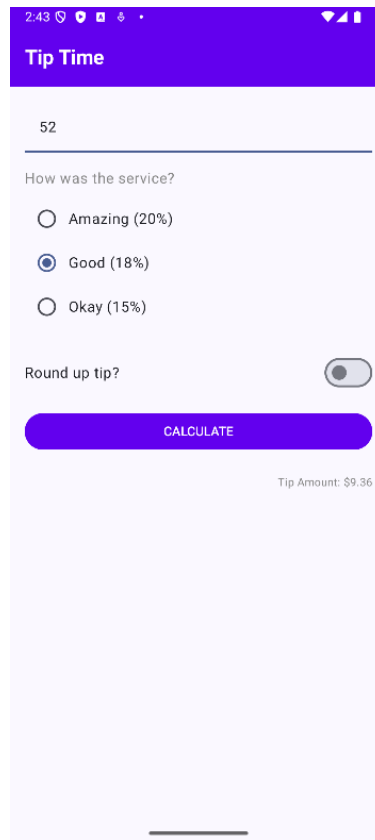
Round up tip? ☐

CALCULATE

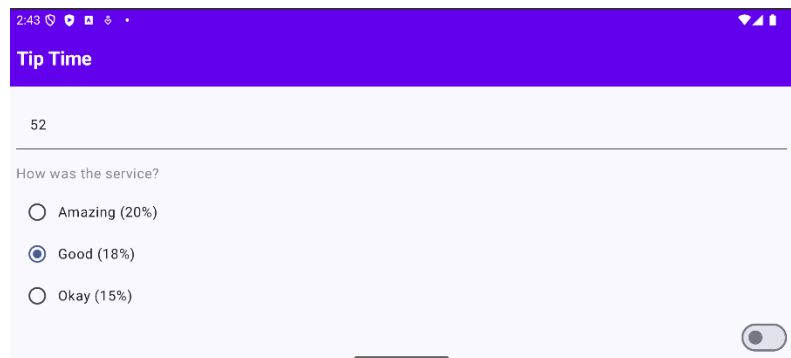
Tip Amount

 Jangan masukkan nilai 0 atau minus!

Gambar 10. Screenshot Hasil Jawaban Soal 1 Modul 2 (Tampilan Toast Error)



Gambar 11. Screenshot Hasil Jawaban Soal 1 (Hasil Setelah Dihitung)



Gambar 12. Screenshot Hasil Jawaban Soal 1 (Tampilan Aplikasi Layar Dirotate)

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. MainActivity.kt:

- Pada line 1, dideklarasikan nama package file Kotlin dan menunjukkan nama tempat file ini berada.

- Pada line 3, diimport `Bundle` untuk mengkomunikasikan dan menyimpan data antar aktivitas atau saat activity di-pause atau rotate.
- Pada line 4, diimport `androidx.activity.ComponentActivity` untuk kelas dasar dari activity yang support Jetpack Compose.
- Pada line 5, diimport `androidx.activity.compose.setContent` untuk menampilkan UI berbasis Jetpack Compose di dalam activity.
- Pada line 6, diimport `androidx.compose.material3.Surface` untuk untuk membungkus konten UI dengan properti seperti warna latar atau shape.
- Pada line 7, diimport `androidx.compose.ui.Modifier` untuk memodifikasi tampilan UI Compose seperti padding, ukuran, posisi, dll.
- Pada line 8, diimport `androidx.compose.foundation.layout.fillMaxSize` untuk membuat komponen memenuhi seluruh ukuran layar.
- Pada line 9, diimport `com.example.calculatetipapp.ui.theme.CalculateTipAppTheme` untuk mengimpor tema compose.
- Pada line 11, terdapat `class MainActivity : ComponentActivity()` yang mendefinisikan `MainActivity` sebagai turunan dari `ComponentActivity`, artinya ini adalah activity utama aplikasi.
- Pada line 12, terdapat override fungsi `onCreate`, yaitu titik masuk ketika activity pertama kali dibuat. `savedInstanceState` menyimpan state sebelumnya jika ada.
- Pada line 13, terdapat pemanggilan implementasi `onCreate` dari superclass agar aktivitas bisa berfungsi normal.
- Pada line 14, terdapat `setContent {` untuk memulai UI Compose. Segala yang ada di dalam kurung kurawal ini adalah UI berbasis Compose.

- Pada line 15, terdapat `CalculateTipAppTheme {` yang akan menerapkan tema khusus yang diimpor tadi ke seluruh UI di dalamnya.
- Pada line 16, terdapat `Surface (` yang akan membuat container `Surface` sebagai dasar UI.
- Pada line 17, terdapat `modifier = Modifier.fillMaxSize()` yang akan menjadikan modifier ini membuat `Surface` memenuhi seluruh ukuran layar.
- Pada line 19, akan memanggil fungsi `TipCalculatorApp()` yang merupakan composable utama yang berisi tampilan kalkulator tip.

2. TipCalculatorApp.kt

- Pada line 1, dideklarasikan nama package file Kotlin dan menunjukkan nama tempat file ini berada.
- Pada line 3, diimport `Toast` untuk menampilkan pesan pemberitahuan jika ada error atau ketika user salah memasukkan input.
- Pada line 4, diimport `foundation.layout` untuk mengimpor berbagai layout tools: `Column`, `Row`, `Spacer`, `padding`, `fillMaxSize`, dll.
- Pada line 5, diimport `rememberScrollState` untuk membuat scroll state yang diingat `Compose`.
- Pada line 6, diimport `KeyboardOptions` untuk mengatur opsi keyboard.
- Pada line 7, diimport `verticalScroll` agar konten di-layout bisa di-scroll ke atas-bawah.
- Pada line 8, diimport komponen `material3`, seperti `TextField`, `Button`, `Text`, `Switch`, dll.
- Pada line 9, diimport komponen `TopAppBar`.
- Pada line 10, diimport `androidx.compose.runtime.Composable` untuk mendefinisikan fungsi UI yang bisa dipakai di `Compose`.
- Pada line 11, diimport `androidx.compose.ui.Alignment` untuk mengatur posisi alignment elemen.

- Pada line 12, diimport `androidx.compose.ui.Modifier` untuk styling seperti padding, fill, dll.
- Pada line 13, diimport `Color` untuk mengatur warna.
- Pada line 14, diimport `androidx.compose.ui.platform.LocalContext` untuk mengakses konteks Android (dibutuhkan buat Toast).
- Pada line 15, diimport `text.font.FontWeight` untuk mengatur ketebalan font.
- Pada line 16, diimport `KeyboardType` untuk menentukan tipe keyboard.
- Pada line 17, diimport `unit.dp` untuk menentukan ukuran padding, spacing dalam satuan dp.
- Pada line 18, diimport `androidx.lifecycle.viewmodel.compose.viewModel` untuk memanggil ViewModel dari Compose.
- Pada line 20, terdapat `@OptIn(ExperimentalMaterial3Api::class)` untuk mengizinkan penggunaan fitur eksperimental dari Material3.
- Pada line 21, terdapat `@Composable` yang menandakan bahwa fungsi ini sebagai UI di Jetpack Compose.
- Pada line 22, terdapat `fun TipCalculatorApp(tipViewModel: TipViewModel = viewModel()) {` yang merupakan fungsi utama UI dan akan mengambil `TipViewModel` agar data state bisa dipantau.
- Pada line 23, terdapat `val context = LocalContext.current` yang akan mengambil Context aplikasi, dibutuhkan untuk Toast.
- Pada line 24, terdapat `val scrollState = rememberScrollState()` yang akan menyimpan status scroll agar bisa dipakai pada `verticalScroll`.
- Pada line 26, terdapat `Scaffold()` yang merupakan komponen layout dasar yang menyediakan `topBar`, `bottomBar`, dll.

- Pada line 27, terdapat `topBar = {` yang menentukan isi dari topBar.
- Pada line 28, terdapat `TopAppBar(` yang artinya akan menggunakan `TopAppBar` dari `Material3`.
- Pada line 29, terdapat `title = {` yang menampilkan judul di TopBar.
- Pada line 30 hingga 34, terdapat `Text(text = "Tip Time", color = Color.White, fontWeight = FontWeight.Bold)` yang akan menampilkan teks "Tip Time" dengan warna putih dan bold.
- Pada line 36, terdapat `colors = TopAppBarDefaults.smallTopAppBarColors(containerColor = Color(0xFF6200EE))` yang akan mengatur warna background TopAppBar menjadi warna ungu.
- Pada line 39, terdapat `{ innerPadding ->` yang merupakan parameter `innerPadding` dari Scaffold.
- Pada line 40, terdapat `Column(` yang akan mengatur isi aplikasi dalam satu kolom vertikal.
- Pada line 41 hingga 46, terdapat `modifier = Modifier.padding(innerPadding).padding(16.dp).fillMaxSize().verticalScroll(scrollState)` yang merupakan modifier untuk mengatur padding, memenuhi layar, dan bisa di-scroll.
- Pada line 47, terdapat `TextField(` yang akan membuat inputan untuk cost of service.
- Pada line 48 hingga 49, terdapat `value = tipViewModel.serviceCostInput, onChange = { tipViewModel.serviceCostInput = it },` yang akan mengambil dan menyimpan input user ke dalam ViewModel.
- Pada line 50 hingga 54, terdapat `placeholder = { if (tipViewModel.serviceCostInput.isEmpty()) { Text("Cost of Service", color = Color.Gray) } },`

yang akan menampilkan placeholder jika input masih kosong. Di dalamnya ada text "Cost of Service".

- Pada line 55, terdapat `keyboardOptions = KeyboardOptions.Default.copy(keyboardType = KeyboardType.Number)` yang akan mengatur keyboard jadi angka.
- Pada line 56, terdapat `modifier = Modifier.fillMaxWidth()` yang akan mengatur layar menjadi lebar penuh.
- Pada line 57 hingga 58, terdapat `colors = TextFieldDefaults.textFieldColors( containerColor = Color.Transparent` yang akan mengatur background menjadi transparent.
- Pada line 62, terdapat `Spacer(modifier = Modifier.height(16.dp))` yang akan menampilkan teks dengan jarak 16dp.
- Pada line 63, terdapat `Text("How was the service?", color = Color.Gray)` yang akan menampilkan teks "How was the service?" dengan teks berwarna abu-abu.
- Pada line 65 terdapat `Spacer(modifier = Modifier.height(8.dp))` yang akan menampilkan teks dengan jarak 8dp.
- Pada line 66 hingga 70, terdapat `listOf("Amazing (20%)" to 20, "Good (18%)" to 18, "Okay (15%)" to 15 ).forEach { (label, value) ->` yang akan menampilkan list pilihan tip dengan bentuk radio button.
- Pada line 71, terdapat `Row(verticalAlignment = Alignment.CenterVertically) {` yang akan menampilkan baris horizontal dan memastikan isi (RadioButton dan Text) sejajar secara vertikal di tengah.

- Pada line 72 hingga 73, terdapat `RadioButton(selected = tipViewModel.selectedTipPercent == value, yang akan menampilkan radio button.`
- Pada line 74, terdapat `onClick = { tipViewModel.selectedTipPercent = value }` yang akan menampilkan nilai `selectedTipPercent` jika tombol diklik.
- Pada line 76, terdapat `Text(text = label)` yang akan menampilkan label pilihan, misalnya "Amazing (20%)", di samping radio button.
- Pada line 80, terdapat `Spacer(modifier = Modifier.height(8.dp))` yang akan memberi jarak vertikal 8dp sebelum row berikutnya.
- Pada line 81 hingga 84, terdapat `Row(modifier = Modifier.fillMaxWidth().padding(top = 16.dp), yang akan menampilkan baris horizontal dengan lebar penuh. dan padding atas 16dp`
- Pada line 85 terdapat `verticalAlignment = Alignment.CenterVertically`, yang mengatur agar isi sejajar secara vertikal.
- Pada line 86, terdapat `horizontalArrangement = Arrangement.SpaceBetween` yang mengatur jarak antara dua elemen diatur biar satu di kiri (text), satu di kanan (switch).
- Pada line 88, terdapat `Text("Round up tip?")` yang akan menampilkan teks "Round up tip?" di sisi kiri row.
- Pada line 89 hingga 93, terdapat `Switch(checked = tipViewModel.roundUp, onChangeChange = { tipViewModel.roundUp = it })` yang akan menampilkan toggle switch di sisi kanan, memeriksa switch menyala atau tidak tergantung nilai `roundUp` dari `ViewModel`, dan mengubah nilai `roundup` jika switch digeser.

- Pada line 95, terdapat `Spacer(modifier = Modifier.height(16.dp))` yang akan memberi jarak vertikal 16dp sebelum tombol muncul.
- Pada line 96 hingga 97, terdapat `Button(onClick = {` yang merupakan komponen tombol. `onClick` berisi logika ketika tombol ditekan.
- Pada line 98, terdapat `val cost = tipViewModel.serviceCostInput.toDoubleOrNull()` yang akan mengubah input biaya dari string ke Double. Jika gagal (misalnya kosong/karakter), maka hasilnya null.
- Pada line 99, terdapat `if (cost == null || cost <= 0.0) {` yang akan memeriksa input. Jika input tidak valid (kosong atau ≤ 0), maka akan menjalankan program di dalamnya.
- Pada line 100 hingga 101 terdapat `Toast.makeText(context, "Jangan masukkan nilai 0 atau minus!", Toast.LENGTH_SHORT).show()` `tipViewModel.showTip = false` yang akan menampilkan peringatan menggunakan Toast, dan menyembunyikan hasil kalkulasi.
- Pada line 102 hingga 105 terdapat `} else { tipViewModel.onCalculateClicked() }`, yang akan dijalankan jika valid, lalu akan memanggil fungsi `onCalculateClicked()` dari ViewModel untuk menghitung tip.
- Pada line 106, terdapat `colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF6200EE))`, yang akan membuat warna tombol menjadi ungu.
- Pada line 107, terdapat `modifier = Modifier.fillMaxWidth()` yang akan mengatur agar tombol memenuhi lebar container.

- Pada line 109, terdapat `Text("CALCULATE")` agar ada teks "CALCULATE" di dalam tombol.
- Pada line 112, terdapat `if (!tipViewModel.showTip)` { yang akan memeriksa jika hasil belum ditampilkan.
- Pada line 113 hingga 120, terdapat `Spacer(modifier = Modifier.height(8.dp)) Text(text = "Tip Amount", color = Color.Gray, style = MaterialTheme.typography.bodySmall, modifier = Modifier.align(Alignment.End))` yang akan menampilkan teks "Tip Amount" sebagai label, warnanya abu-abu dan rata kanan.
- Pada line 122 hingga 123, terdapat `if (tipViewModel.showTip)` { `Spacer(modifier = Modifier.height(24.dp))` yang akan dijalankan jika hasil tersedia, terdapat spasi sebesar 24dp untuk memisahkan dari tombol.
- Pada line 124 hingga 125, terdapat `Text(text = "Tip Amount: \${tipViewModel.calculatedTip}", color = Color.Gray, style = MaterialTheme.typography.bodySmall, modifier = Modifier.align(Alignment.End))` yang akan menampilkan hasil kalkulasi tip dengan format \$... dan menjadikan teksnya berwarna abu-abu, kecil, dan posisinya di kanan.

3. TipViewModul.kt

- Pada line 1, dideklarasikan nama package file Kotlin dan menunjukkan nama tempat file ini berada.
- Pada line 3, diimport `getValue` agar bisa mengambil nilai dari `mutableStateOf`.

- Pada line 4, diimport `androidx.compose.runtime.mutableStateOf` yang menjadikan suatu nilai jadi state yang bisa dipantau.
- Pada line 5, diimport `setValue` agar bisa mengubah nilai `mutableStateOf`.
- Pada line 6, diimport `ViewModel`, class dari Android Jetpack yang menyimpan data UI dan survive saat rotasi layar atau perubahan konfigurasi.
- Pada line 7, diimport fungsi `ceil()` dari Kotlin, untuk pembulatan ke atas.
- Pada line 9, akan membuat kelas `TipViewModel` yang mewarisi dari `ViewModel`. yang akan menyimpan dan mengatur data yang dibutuhkan UI (hasil tip ataupun input user)
- Pada line 10, terdapat `var serviceCostInput by mutableStateOf("")` yang merupakan state untuk menyimpan input biaya layanan dari user. Awalnya string kosong ("").
- Pada line 11, terdapat `var selectedTipPercent by mutableStateOf(18)` yang akan menyimpan persentase tip yang dipilih user dengan default-nya 18%.
- Pada line 12, terdapat `var roundUp by mutableStateOf(false)` yang akan menyimpan apakah tip akan dibulatkan ke atas, default-nya false.
- Pada line 13, terdapat `var showTip by mutableStateOf(false)` yang akan menyimpan status apakah hasil tip akan ditampilkan atau belum, default-nya false.
- Pada line 15, terdapat `val calculatedTip: String` yang akan menyimpan hasil kalkulasi tip dalam bentuk string.
- Pada line 16, terdapat `get() {` yang merupakan awal dari custom getter. Setiap kali `calculatedTip` dipanggil, logika di dalamnya ini akan dijalankan.
- Pada line 17, terdapat `val cost = serviceCostInput.toDoubleOrNull() ?: return "0.00"`

yang akan mengubah input string jadi Double. Jika gagal (misal user input huruf), akan mengembalikan "0.00".

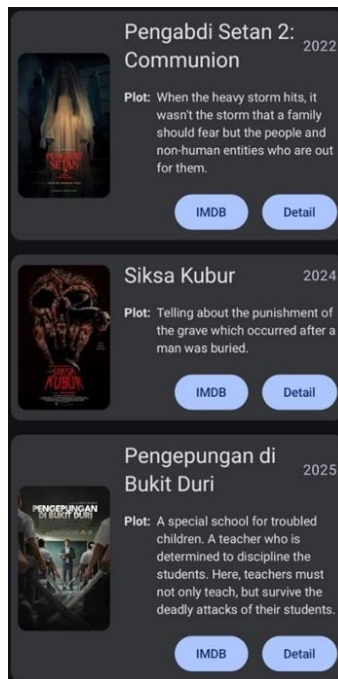
- Pada line 18, terdapat `var tip = cost * selectedTipPercent / 100` yang merupakan rumus dalam menghitung tip.
- Pada line 19, terdapat `if (roundUp) tip = ceil(tip)` yang akan diterapkan jika user memilih tip dibulatkan ke atas.
- Pada line 20, terdapat `return String.format("%.2f", tip)` yang akan mengembalikan hasil tip dalam format dua angka decimal.
- Pada line 23, terdapat `fun onCalculateClicked() {` yang akan dipanggil Ketika tombol "Calculate" diklik.
- Pada line 24, akan set `showTip` jadi `true`, sehingga UI akan tahu harus menampilkan hasil tip.

MODUL 3 : Build a Scrollable List

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut: 1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose) 2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas 3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah 4. Terdapat 2 button dalam list, dengan fungsi berikut: a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item 5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius 6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang 7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment) 8. Aplikasi berbasis XML harus menggunakan ViewBinding.

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 13. Soal 1 Modul 3 (Contoh UI List)

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 14. Soal 1 Modul 3 (Contoh UI Detail)

A. Source Code

1. MainActivity.kt

Tabel 7. Source Code Jawaban Soal 1 Modul 3 (MainActivity.kt)

1	package com.example.starconstellations
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.navigation.fragment.NavHostFragment
6	import
7	androidx.navigation.ui.setupActionBarWithNavController
8	import
9	com.example.starconstellations.databinding.ActivityMainB
10	inding
11	
12	class MainActivity : AppCompatActivity() {
13	
14	private lateinit var binding: ActivityMainBinding
15	
16	override fun onCreate(savedInstanceState: Bundle?) {
17	super.onCreate(savedInstanceState)
18	
19	binding =
20	ActivityMainBinding.inflate(layoutInflater)
21	setContentView(binding.root)
22	
23	setSupportActionBar(binding.toolbar)
24	
25	val navHostFragment = supportFragmentManager
26	.findFragmentById(R.id.nav_host_fragment) as
27	NavHostFragment
28	val navController =
29	navHostFragment.navController
30	

31	<code>setupActionBarWithNavController(navController)</code>
32	<code>}</code>
33	
34	<code>override fun onSupportNavigateUp(): Boolean {</code>
	<code> val navHostFragment = supportFragmentManager</code>
	<code> .findFragmentById(R.id.nav_host_fragment) as</code>
	<code>NavHostFragment</code>
	<code> return</code>
	<code>navHostFragment.navController.navigateUp() </code>
	<code>super.onSupportNavigateUp()</code>
	<code>}</code>
	<code>}</code>

2. Constellation.kt

Tabel 8. Source Code Jawaban Soal 1 Modul 3 (Constellation.kt)

1	<code>package com.example.starconstellations</code>
2	
3	<code>import android.os.Parcelable</code>
4	<code>import kotlinx.parcelize.Parcelize</code>
5	
6	<code>@Parcelize</code>
7	<code>data class Constellation(</code>
8	<code> val name: String,</code>
9	<code> val description: String,</code>
10	<code> val year: String,</code>
11	<code> val imageResId: Int,</code>
12	<code> val webUrl: String</code>
13	<code>) : Parcelable</code>

3. ConstellationData.kt

Tabel 9. Source Code Jawaban Soal 1 Modul 3 (ConstellationData.kt)

1	package com.example.starconstellations
2	
3	object ConstellationData {
4	val listConstellations = listOf(
5	Constellation(
6	name = "Hydra",
7	description = "Hydra, alias sang ular air
8	(The Water Serpent), adalah rasi bintang terpanjang dan
9	terbesar di langit malam. Luasnya sebesar 1.303 derajat
10	persegi (~3,16% dari seluruh langit malam). Bentuknya
11	menyerupai ular air dan membentang dari selatan Cancer
12	hingga Libra. Meski ukurannya besar, hanya sedikit
13	bintang terang yang terlihat. Hydra dikenal dalam
14	mitologi Yunani sebagai ular berkepala banyak yang
15	dikalahkan oleh Herkules.",
16	imageResId = R.drawable.hydra,
17	year = "420 juta tahun",
18	webUrl =
19	"https://en.m.wikipedia.org/wiki/Hydra_(constellation)"
20	),
21	Constellation(
22	name = "Virgo",
23	description = "Virgo, alias sang perawan
24	suci (The Maiden), adalah rasi bintang terbesar kedua
25	dan salah satu dari zodiak. Luasnya sebesar 1.294
26	derajat persegi (~3,14% dari langit malam). Bintang
27	terangnya, Spica, sangat mudah dikenali. Virgo
28	melambangkan dewi panen dan kesuburan dalam berbagai
29	mitologi, termasuk Yunani dan Romawi. Rasi ini juga
30	penting dalam astronomi karena menjadi lokasi
31	superklaster galaksi yang dinamakan Virgo Cluster.",

32	imageResId = R.drawable.virgo,
33	year = "12-400 juta tahun",
34	webUrl =
35	"https://en.m.wikipedia.org/wiki/Virgo_(constellation)"
36	),
37	Constellation(
38	name = "Ursa Major",
39	description = "Ursa Major, alias sang
40	beruang besar (The Great Bear), adalah salah satu rasi
41	bintang paling dikenal karena mengandung asterisma Big
	Dipper (Gayung Besar). Luasnya sebesar 1.280 derajat
	persegi (~3,11% dari langit malam). Rasi ini penting
	dalam navigasi karena dua bintangnya menunjuk ke Polaris
	(Bintang Utara).",
	imageResId = R.drawable.ursa_major,
	year = "300 juta - 1 miliar tahun",
	webUrl =
	"https://en.m.wikipedia.org/wiki/Ursa_Major"
	),
	Constellation(
	name = "Cetus",
	description = "Cetus, alias sang monster
	laut (The Sea Monster), dikenal sebagai 'Ikan Paus' atau
	'Monster Laut' dalam mitologi Yunani. Luasnya sebesar
	1.231 derajat persegi (~2,99% dari langit malam). Rasi
	ini cukup besar dan terletak di dekat rasi Pisces dan
	Eridanus. Meskipun luas, Cetus tidak memiliki banyak
	bintang terang, namun menjadi rumah bagi salah satu
	bintang variabel terkenal, Mira.",
	imageResId = R.drawable.cetus,
	year = "2 miliar tahun",
	webUrl =
	"https://en.m.wikipedia.org/wiki/Cetus"

	<pre>), Constellation(name = "Hercules", description = "Hercules, alias sang pahlawan kuat, dinamai dari pahlawan mitologi Yunani, yaitu 'Heracles'. Luasnya sebesar 1.225 derajat persegi (~2,97% dari langit malam). Bentuknya tidak mudah dikenali, tapi rasi ini memiliki formasi yang dikenal sebagai \"Keystone\" (batu penjuru). Salah satu daya tariknya adalah Gugus Bola M13, salah satu gugus bintang paling terang yang bisa dilihat dengan teleskop kecil.", imageResId = R.drawable.hercules, year = " 11 miliar tahun", webUrl = "https://en.m.wikipedia.org/wiki/Hercules_(constellation)"),) } </pre>
--	---

4. HomeFragment.kt

Tabel 10. Source Code Jawaban Soal 1 Modul 3 (HomeFragment.kt)

1	package com.example.starconstellations
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.navigation.fragment.findNavController
11	

```

12 import androidx.recyclerview.widget.LinearLayoutManager
13 import com.example.starconstellations.databinding.
14 FragmentHomeBinding
15
16 class HomeFragment : Fragment() {
17
18     private var _binding: FragmentHomeBinding? = null
19     private val binding get() = _binding!!
20
21     private lateinit var adapter:
22 ListConstellationsAdapter
23
24     override fun onCreateView(
25         inflater: LayoutInflater, container: ViewGroup?,
26         savedInstanceState: Bundle?
27     ): View {
28         _binding = FragmentHomeBinding.inflate(inflater,
29 container, false)
30         return binding.root
31     }
32
33     override fun onViewCreated(view: View,
34 savedInstanceState: Bundle?) {
35         super.onViewCreated(view, savedInstanceState)
36
37         val data = ConstellationData.listConstellations
38
39         adapter = ListConstellationsAdapter(data,
40 onWebClick = { url ->
41             val intent = Intent(Intent.ACTION_VIEW,
42 Uri.parse(url))
43             startActivity(intent)
44         }, onDetailClick = { constellation ->

```

45	val action =
46	HomeFragmentDirections.actionHomeFragmentToDetailFragmen
47	t(constellation)
48	findNavController().navigate(action)
49	})
50	
	binding.rvStars.layoutManager =
	LinearLayoutManager(requireContext())
	binding.rvStars.adapter = adapter
	}
	override fun onDestroyView() {
	super.onDestroyView()
	_binding = null
	}
	}

5. DetailFragment.kt

Tabel 11. Source Code Jawaban Soal 1 Modul 3 (DetailFragment.kt)

1	package com.example.starconstellations
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.navArgs
9	import com.bumptech.glide.Glide
10	import
11	com.example.starconstellations.databinding.FragmentDetail
12	Binding
13	
14	class DetailFragment : Fragment() {

```

15
16     private var _binding: FragmentDetailBinding? = null
17     private val binding get() = _binding!!
18     private val args: DetailFragmentArgs by navArgs()
19
20     override fun onCreateView(
21         inflater: LayoutInflater, container: ViewGroup?,
22         savedInstanceState: Bundle?
23     ): View {
24         _binding =
25     FragmentDetailBinding.inflate(inflater, container, false)
26         return binding.root
27     }
28
29     override fun onViewCreated(view: View,
30 savedInstanceState: Bundle?) {
31         super.onViewCreated(view, savedInstanceState)
32
33         val data = args.constellation
34
35         binding.apply {
36             detailName.text = data.name
37             detailDescription.text = data.description
38             detailYear.text = data.year
39
40     Glide.with(this@DetailFragment).load(data.imageResId).into(
41 detailImage)
42         }
43
44     }
45
46     override fun onDestroyView() {
47         super.onDestroyView()

```

	<pre> _binding = null } }</pre>
--	---

6. ListConstellationsAdapter.kt

Tabel 12. Source Code Jawaban Soal 1 Modul 3 (ListConstellationsAdapter.kt)

1	package com.example.starconstellations
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.bumptech.glide.Glide
7	import
8	com.example.starconstellations.databinding.StarItemBinding
9	
10	
11	class ListConstellationsAdapter(
12	private val list: List<Constellation>,
13	private val onWebClick: (String) -> Unit,
14	private val onDetailClick: (Constellation) -> Unit
15	) :
16	RecyclerView.Adapter<ListConstellationsAdapter.ViewHolder>()
17	{
18	
19	inner class ViewHolder(val binding: StarItemBinding) :
20	RecyclerView.ViewHolder(binding.root)
21	
22	override fun onCreateViewHolder(parent: ViewGroup,
23	viewType: Int): ViewHolder {
24	val binding =
25	StarItemBinding.inflate(LayoutInflater.from(parent.context),
26	parent, false)
27	return ViewHolder(binding)

28	}
29	
30	override fun getItemCount(): Int = list.size
31	
32	override fun onBindViewHolder(holder: ViewHolder,
33	position: Int) {
34	val item = list[position]
35	with(holder.binding) {
36	tvName.text = item.name
37	tvDescription.text = item.description
38	tvYear.text = item.year
39	
40	Glide.with(img.context).load(item.imageResId).into(img)
41	
42	btnWeb.setOnClickListener {
	onWebClick(item.webUrl)
	}
	btnDetail.setOnClickListener {
	onDetailClick(item)
	}
	}
	}
	}

7. activity_main.xml

Tabel 13. Source Code Jawaban Soal 1 Modul 3 (activity_main.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/android"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"

7	android:layout_height="match_parent">
8	
9	<androidx.appcompat.widget.Toolbar
10	android:id="@+id/toolbar"
11	android:layout_width="match_parent"
12	android:layout_height="?attr/actionBarSize"
13	android:background="?attr/colorPrimary"
14	android:elevation="4dp"
15	app:titleTextColor="@android:color/white" />
16	
17	<androidx.fragment.app.FragmentContainerView
18	android:id="@+id/nav_host_fragment"
19	
20	android:name="androidx.navigation.fragment.NavHostFragment"
21	android:layout_width="match_parent"
22	android:layout_height="match_parent"
23	android:layout_marginTop="?attr/actionBarSize"
24	app:defaultNavHost="true"
25	app:navGraph="@navigation/nav_graph" />
	</androidx.coordinatorlayout.widget.CoordinatorLayout>

8. fragment_home.xml

Tabel 14. Source Code Jawaban Soal 1 Modul 3 (fragment_home.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:id="@+id/homeFragment"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="8dp">
8	
9	<androidx.recyclerview.widget.RecyclerView

10	android:id="@+id/rvStars"
11	android:layout_width="match_parent"
12	android:layout_height="match_parent"
13	android:clipToPadding="false" />
14	
	</FrameLayout>

9. fragment_detail.xml

Tabel 15. Source Code Jawaban Soal 1 Modul 3 (fragment_detail.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/detailFragment"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:padding="16dp">
9	
10	<LinearLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:orientation="vertical"
14	android:gravity="center_horizontal">
15	
16	<ImageView
17	android:id="@+id/detailImage"
18	android:layout_width="350dp"
19	android:layout_height="400dp"
20	android:scaleType="centerCrop"
21	android:layout_marginBottom="16dp"
22	
23	app:shapeAppearanceOverlay="@style/RoundedImageView" />
24	

25	<TextView
26	android:id="@+id/detailName"
27	android:layout_width="wrap_content"
28	android:layout_height="wrap_content"
29	android:text="Hydra"
30	android:textSize="22sp"
31	android:textStyle="bold"
32	android:layout_marginBottom="8dp" />
33	
34	<TextView
35	android:id="@+id/detailYear"
36	android:layout_width="wrap_content"
37	android:layout_height="wrap_content"
38	android:text=""
39	android:textSize="16sp"
40	android:layout_marginBottom="16dp" />
41	
42	<TextView
43	android:id="@+id/detailDescription"
44	android:layout_width="match_parent"
45	android:layout_height="wrap_content"
46	android:text="This constellation is known
47	for..."
	android:textSize="16sp" />
	</LinearLayout>
	</ScrollView>

10. star_item.xml

Tabel 16. Source Code Jawaban Soal 1 Modul 3 (star_item.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"

5	android:layout_height="wrap_content"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	android:orientation="vertical"
8	android:padding="8dp">
9	
10	<androidx.cardview.widget.CardView
11	xmlns:card_view="http://schemas.android.com/apk/res-auto"
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:layout_margin="8dp"
15	card_view:cardCornerRadius="16dp"
16	card_view:cardElevation="4dp">
17	
18	<LinearLayout
19	android:layout_width="match_parent"
20	android:layout_height="wrap_content"
21	android:orientation="horizontal"
22	android:padding="8dp">
23	
24	
25	<com.google.android.material.imageview.ShapeableImageView
26	android:id="@+id/img"
27	android:layout_width="100dp"
28	android:layout_height="150dp"
29	android:scaleType="centerCrop"
30	android:layout_marginEnd="8dp"
31	
32	app:shapeAppearanceOverlay="@style/RoundedImageView" />
33	
34	<LinearLayout
35	android:layout_width="0dp"
36	android:layout_height="match_parent"
37	android:layout_weight="1"

38	android:orientation="vertical">
39	
40	<LinearLayout
41	android:layout_width="match_parent"
42	android:layout_height="wrap_content"
43	android:orientation="horizontal"
44	android:gravity="center_vertical">
45	
46	<TextView
47	android:id="@+id/tvName"
48	android:layout_width="0dp"
49	android:layout_height="wrap_content"
50	android:layout_weight="1"
51	android:text="Orion"
52	android:textStyle="bold"
53	android:textSize="16sp" />
54	
55	<TextView
56	android:id="@+id/tvYear"
57	android:layout_width="wrap_content"
58	android:layout_height="wrap_content"
59	android:text=""
60	android:textSize="14sp" />
61	</LinearLayout>
62	
63	<TextView
64	android:id="@+id/tvDescription"
65	android:layout_width="match_parent"
66	android:layout_height="wrap_content"
67	android:text="One of the brightest
68	constellations..."
69	android:textSize="14sp"
70	android:layout_marginTop="4dp"

71	android:maxLines="3"
72	android:ellipsize="end" />
73	
74	<LinearLayout
75	android:layout_width="match_parent"
76	android:layout_height="wrap_content"
77	android:orientation="horizontal"
78	android:layout_marginTop="8dp">
79	
80	<Button
81	android:id="@+id/btnWeb"
82	android:layout_width="0dp"
83	android:layout_height="wrap_content"
84	android:layout_weight="1"
85	android:text="Website" />
86	
87	<Button
88	android:id="@+id/btnDetail"
89	android:layout_width="0dp"
90	android:layout_height="wrap_content"
91	android:layout_weight="1"
92	android:text="Detail"
93	android:layout_marginStart="8dp" />
	</LinearLayout>
	</LinearLayout>
	</LinearLayout>
	</androidx.cardview.widget.CardView>
	</LinearLayout>

11. nav_graph.xml

Tabel 17. Source Code Jawaban Soal 1 Modul 3 (nav_graph.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation

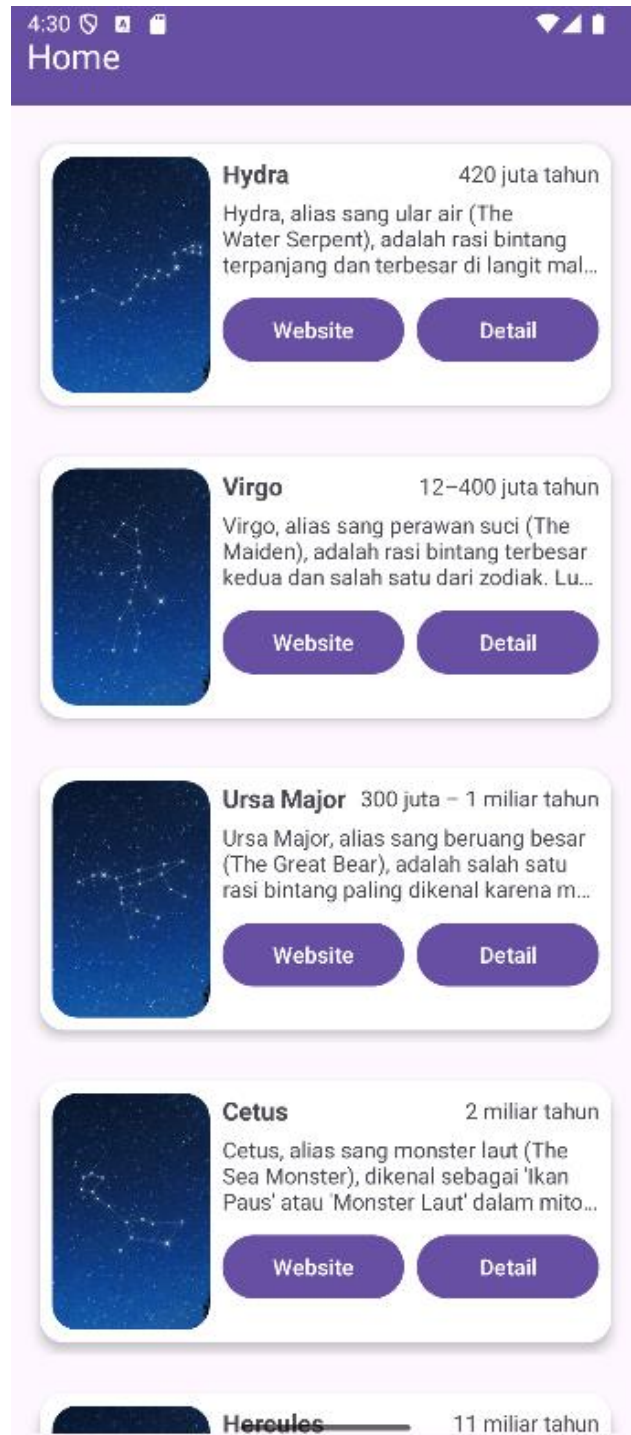
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_graph"
6	app:startDestination="@id/homeFragment">
7	
8	<fragment
9	android:id="@+id/homeFragment"
10	
11	android:name="com.example.starconstellations.HomeFragment"
12	android:label="Home">
13	<action
14	
15	android:id="@+id/action_homeFragment_to_detailFragment"
16	app:destination="@id/detailFragment" />
17	</fragment>
18	
19	<fragment
20	android:id="@+id/detailFragment"
21	
22	android:name="com.example.starconstellations.DetailFragment
23	"
24	android:label="Detail">
	<argument
	android:name="constellation"
	app:argType="com.example.starconstellations.Constellation"
	/>
	</fragment>
	</navigation>

12. styles.xml

Tabel 18. Source Code Jawaban Soal 1 Modul 3 (styles.xml)

1	<?xml version="1.0" encoding="utf-8"?>
1	<resources>
2	<style name="RoundedImageView" parent="">
3	<item name="cornerFamily">rounded</item>
4	<item name="cornerSize">16dp</item>
5	</style>
6	</resources>
7	

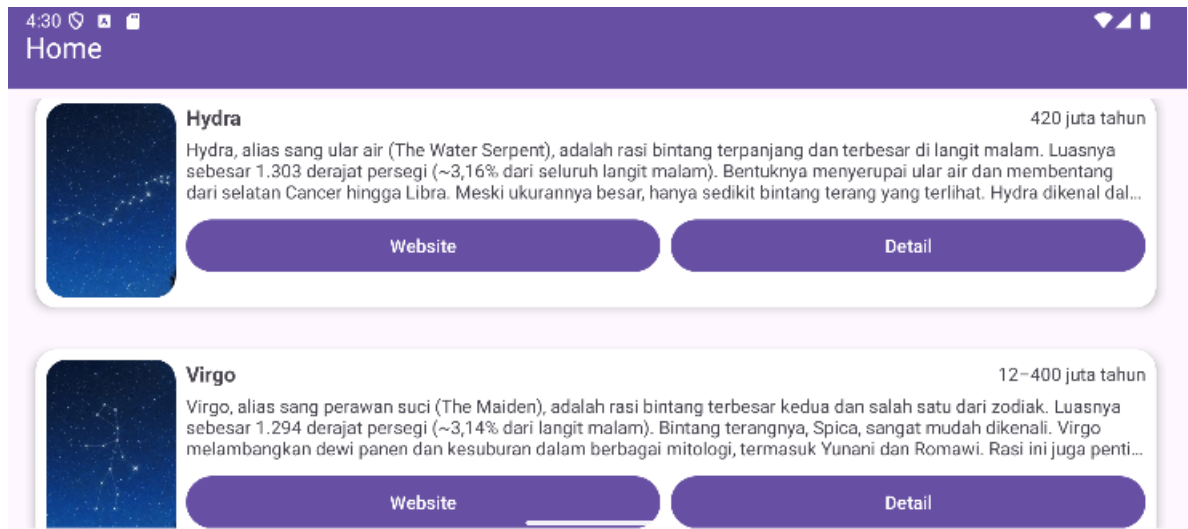
B. Output Program



Gambar 15. Screenshot Hasil Jawaban Soal 1 Modul 3 (List Portrait)



Gambar 16. Screenshot Hasil Jawaban Soal 1 Modul 3 (Detail Portrait)



Gambar 17. Screenshot Hasil Jawaban Soal 1 Modul 3 (List Landscape)



Gambar 18. Screenshot Hasil Jawaban Soal 1 Modul 3 (Detail Landscape)

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. MainActivity.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.starconstellations`.
- Pada line 3, mengimpor `Bundle` untuk menyimpan status activity saat di-pause atau di-rotate.

- Pada line 4, mengimpor `AppCompatActivity` untuk menggunakan fitur kompatibilitas activity pada aplikasi.
- Pada line 5, mengimpor `NavHostFragment` untuk mendukung navigasi menggunakan fragment.
- Pada line 6, mengimpor `setupActionBarWithNavController` untuk mengatur aksi bar dengan navigasi controller.
- Pada line 7, mengimpor `ActivityMainBinding` untuk binding layout dari XML ke dalam activity.
- Pada line 9, mendeklarasikan kelas `MainActivity` yang mewarisi `AppCompatActivity`.
- Pada line 11, mendeklarasikan properti binding untuk akses komponen UI melalui `ViewBinding`.
- Pada line 13-17, membuat `onCreate`, meng-inflate layout dan mengatur konten view menggunakan binding.
- Pada line 19, menyetel toolbar sebagai action bar untuk activity.
- Pada line 21-23, mendapatkan `NavHostFragment` dan `NavController` untuk mendukung navigasi antar fragment.
- Pada line 25, menyiapkan action bar untuk bekerja dengan `NavController` yang ada.
- Pada line 28-32, terdapat Override `onSupportNavigateUp` untuk meng-handle navigasi kembali saat menekan tombol up di action bar.

2. Constellation.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.starconstellations`.
- Pada line 3, mengimpor `Parcelable` untuk memungkinkan objek dikirim antar komponen Android.
- Pada line 4, mengimpor `Parcelize` untuk menyederhanakan implementasi `Parcelable` pada kelas.

- Pada line 6, terdapat anotasi `@Parcelize` digunakan untuk otomatis mengimplementasikan `Parcelable`.
- Pada line 7-12, mendefinisikan kelas data `Constellation` dengan properti: `name`, `description`, `year`, `imageResId`, dan `webUrl`.
- Pada line 13, terdapat `Parcelable` yang artinya kelas `Constellation` mengimplementasikan `Parcelable` untuk mendukung pengiriman objek antar komponen.

3. `ConstellationData.kt`:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.starconstellations`.
- Pada line 3, mendeklarasikan objek `ConstellationData`, yang menyimpan data statis tentang konstelasi bintang.
- Pada line 4, mendeklarasikan properti `listConstellations` sebagai daftar (list) objek `Constellation`.
- Pada line 5-39, menambahkan beberapa objek `Constellation` ke dalam `listConstellations`, masing-masing dengan `name` (nama konstelasi), `description` (deskripsi singkat mengenai konstelasi), `imageResId` (ID gambar konstelasi), `year` (perkiraan usia konstelasi), dan `webUrl` (URL untuk informasi lebih lanjut mengenai konstelasi).

4. `HomeFragment.kt`:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.starconstellations`.
- Pada line 3, diimpor `Intent` untuk memungkinkan pengiriman aksi atau membuka aplikasi lain, seperti membuka browser.
- Pada line 4, diimpor `Uri` untuk menangani URL yang digunakan dalam `Intent` (misalnya untuk membuka link di browser).

- Pada line 5, diimpor `Bundle` untuk membawa data antara komponen, seperti saat berpindah fragment.
- Pada line 6, diimpor `LayoutInflater` untuk mengubah XML layout menjadi objek view di Android.
- Pada line 7, diimpor `View` untuk merepresentasikan elemen antarmuka pengguna dalam Android.
- Pada line 8, diimpor `ViewGroup` yang merupakan kelas dasar untuk layout yang dapat memuat view lainnya.
- Pada line 9, diimpor `Fragment` untuk membuat fragment, yaitu bagian dari antarmuka pengguna dalam aplikasi.
- Pada line 10, diimpor `findNavController` untuk navigasi antar fragment menggunakan `Navigation Component`.
- Pada line 11, diimpor `LinearLayoutManager` untuk mengatur layout item dalam `RecyclerView` secara vertikal.
- Pada line 12, diimpor `FragmentHomeBinding` untuk menggunakan `ViewBinding` yang menghubungkan layout XML `fragment_home.xml` dengan kode program di dalam fragment.
- Pada line 14, mendeklarasikan kelas `HomeFragment`, yang merupakan sebuah fragment untuk menampilkan daftar konstelasi.
- Pada line 16, mendeklarasikan properti `_binding` untuk mengakses komponen UI menggunakan `ViewBinding`.
- Pada line 17, mendeklarasikan properti `binding` untuk memudahkan akses ke `FragmentHomeBinding`.
- Pada line 19, mendeklarasikan adapter untuk `RecyclerView` yang menampilkan data konstelasi.
- Pada line 21-27, membuat `onCreateView`, meng-inflate layout dan mengembalikan root view untuk fragment.

- Pada line 29-40, membuat `onViewCreated`, mengatur adapter untuk `RecyclerView` dan menangani klik pada item, seperti `onWebClick` (membuka URL di browser saat item diklik), `onDetailClick` (menavigasi ke fragment detail dengan membawa data konstelasi yang dipilih).
- Pada line 42, menetapkan `LayoutManager` untuk `RecyclerView` dan mengatur tampilan item dalam `RecyclerView` secara vertikal (seperti daftar atau list).
- Pada line 43, menetapkan adapter untuk `RecyclerView`. Adapter ini menghubungkan data yang ada di dalam list (dalam hal ini `ConstellationData.listConstellations`) dengan tampilan item di `RecyclerView`.
- Pada line 46-49, membuat `onDestroyView` untuk membersihkan binding untuk menghindari memory leak.

5. DetailFragment.kt

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.starconstellations`.
- Pada line 3, diimpor `android.os.Bundle` untuk menangani data yang dikirim antar komponen dalam aplikasi, seperti data yang dikirim saat navigasi antar fragment.
- Pada line 4, diimpor `android.view.LayoutInflater` untuk meng-inflate layout XML ke dalam objek View di dalam fragment.
- Pada line 5, diimpor `android.view.View` untuk merepresentasikan elemen tampilan di dalam fragment.
- Pada line 6, diimpor `android.view.ViewGroup` untuk merepresentasikan container dari elemen tampilan dalam fragment.

- Pada line 7, diimpor `androidx.fragment.app.Fragment` untuk memungkinkan class ini mewarisi fungsionalitas dari fragment, seperti menangani lifecycle dan tampilan.
- Pada line 8, diimpor `androidx.navigation.fragment.navArgs` untuk mengakses argumen yang dikirim saat navigasi antar fragment menggunakan komponen Navigation.
- Pada line 9, diimpor `com.bumptech.glide.Glide` untuk memudahkan memuat dan menampilkan gambar dari URL atau sumber lainnya ke dalam `ImageView`.
- Pada line 10, diimpor `com.example.starconstellations.databinding.FragmentDetailBinding` untuk mengakses elemen-elemen di layout `fragment_detail.xml` menggunakan `ViewBinding`.
- Pada line 12, mendeklarasikan kelas `DetailFragment`, yang menampilkan detail konstelasi saat item dipilih dari `HomeFragment`.
- Pada line 14, mendeklarasikan properti `_binding` untuk mengakses komponen UI menggunakan `ViewBinding`.
- Pada line 15, mendeklarasikan properti binding untuk memudahkan akses ke `FragmentDetailBinding`.
- Pada line 10, menggunakan `navArgs` untuk mengambil data constellation yang dipassing dari `HomeFragment`.
- Pada line 18-24, membuat `onCreateView`, meng-inflate layout dan mengembalikan root view untuk fragment.
- Pada line 26-38, membuat `onViewCreated`, mengatur tampilan detail dengan data yang diterima, seperti menampilkan nama, deskripsi, dan tahun konstelasi, Llau, menggunakan `Glide` untuk memuat gambar konstelasi ke dalam `detailImage`.
- Pada line 40-43, membuat `onDestroyView`, membersihkan binding untuk menghindari memory leak.

6. ListConstellationsAdapter.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.starconstellations`.
- Pada line 3, diimpor `LayoutInflater` untuk mengkonversi XML layout menjadi objek tampilan (view) di kode program.
- Pada line 4, diimpor `ViewGroup` untuk menjadi wadah tampilan yang dapat menyimpan tampilan lainnya, seperti `RecyclerView`.
- Pada line 5, diimpor `RecyclerView` untuk menangani pengelolaan dan penyajian daftar data di UI.
- Pada line 6, diimpor `Glide` untuk memudahkan pemuatan gambar (image loading).
- Pada line 7, diimpor `StarItemBinding` untuk memudahkan binding antara tampilan XML (`star_item.xml`) dan kode program melalui `ViewBinding`.
- Pada line 10, mendeklarasikan kelas `ListConstellationsAdapter`, yang mengatur tampilan dan interaksi `RecyclerView` untuk daftar konstelasi.
- Pada line 11-13, mendeklarasikan dua property, yaitu `onWebClick` untuk menangani klik pada tombol web dan `onDetailClick` untuk menangani klik pada tombol detail.
- Pada line 14, terdapat kelas `ViewHolder` digunakan untuk memegang referensi ke binding item tampilan.
- Pada line 18-21, membuat `onCreateViewHolder`, meng-inflate layout item menggunakan `ViewBinding` untuk menghubungkan tampilan.
- Pada line 23, terdapat `getItemCount` mengembalikan jumlah item dalam daftar konstelasi.
- Pada line 25-40, membuat `onBindViewHolder`, mengikat data ke tampilan, seperti menampilkan nama, deskripsi, tahun, dan gambar

konstelasi dan menetapkan aksi untuk tombol `btnWeb` (membuka URL) dan `btnDetail` (navigasi ke detail).

7. `activity_main.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–25, terdapat elemen root `CoordinatorLayout` dipakai untuk mendukung layout berbasis koordinasi antar elemen, seperti toolbar dan fragment. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 5-6, terdapat `layout_width` & `layout_height` yang di-set ke `match_parent`, artinya mengisi seluruh layar.
- Pada line 8-14, membuat `Toolbar` yang ditambahkan sebagai action bar custom:
 - `layout_height` mengikuti tinggi standar action bar (`?attr/actionBarSize`).
 - `background` mengikuti warna utama tema (`colorPrimary`).
 - `elevation 4dp` memberi efek bayangan.
 - `titleTextColor` di-set ke putih.
- Pada line 16-23, terdapat `FragmentContainerView` untuk menampilkan fragment berdasarkan `Navigation Component`:
 - `layout_marginTop` disesuaikan dengan tinggi action bar supaya fragment tidak tertutup toolbar.
 - `android:name` menunjuk ke `NavHostFragment`, elemen utama navigasi.
 - `app:defaultNavHost="true"` menjadikan fragment ini sebagai host utama.
 - `app:navGraph` menunjuk ke file navigasi `nav_graph.xml`.
- Pada line 25, penutup `CoordinatorLayout`.

8. `fragment_home.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, terdapat root `FrameLayout` digunakan sebagai wadah layout fragment, dengan ID `homeFragment`, ukuran penuh, dan padding 8dp.
- Pada line 8-12, terdapat `RecyclerView` dengan ID `rvStars` digunakan untuk menampilkan daftar rasi bintang dalam list scrollable, memenuhi seluruh ruang yang tersedia dan `clipToPadding` diset false agar konten tetap bisa tampil meski melebihi padding.

9. `fragment_detail.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, terdapat deklarasi `ScrollView` sebagai root layout yang memungkinkan isi halaman dapat discroll secara vertikal. Ini penting karena kontennya bisa lebih panjang dari tinggi layar.
- Pada line 3, terdapat atribut `xmlns` untuk mendefinisikan namespace Android dan app, yang wajib ada agar atribut khusus (seperti `app:shapeAppearanceOverlay`) bisa digunakan.
- Pada line 4, terdapat `android:id="@+id/detailFragment"` sebagai ID unik untuk `ScrollView`, meskipun tidak wajib jika tidak digunakan di kode.
- Pada line 6-7, `layout_width` dan `layout_height` diatur `match_parent` agar `ScrollView` mengisi seluruh layar.
- Pada line 7, `android:padding="16dp"` memberi jarak di sekeliling isi layout supaya tidak terlalu mepet ke tepi layar.
- Pada line 9-13, terdapat `LinearLayout` dengan orientasi vertikal dan `gravity center_horizontal`, artinya elemen di dalamnya akan ditumpuk ke bawah dan disejajarkan di tengah horizontal.
- Pada line 15-21, terdapat `ImageView (@id/detailImage)` untuk menampilkan gambar rasi bintang. Lebar nya 350dp dan tingginya 400dp,

`scaleType="centerCrop"` agar gambar tetap proporsional meski di-crop, dan `layout_marginBottom="16dp"` memberi jarak ke bawah. Atribut `app:shapeAppearanceOverlay` digunakan untuk membuat gambar tampil dengan sudut membulat (rounded).

- Pada line 23–30, terdapat `TextView (@id/detailName)` untuk menampilkan nama rasi bintang, teks ditampilkan bold, ukuran besar (22sp), dan diberi jarak bawah 8dp agar rapi.
- Pada line 32–38, terdapat `TextView (@id/detailYear)` untuk menampilkan tahun atau data waktu terkait rasi bintang. Ukuran teks 16sp dan jarak bawah 16dp.
- Pada line 40–45, terdapat `TextView (@id/detailDescription)` yang menampilkan deskripsi panjang tentang rasi bintang. `layout_width` diatur `match_parent` supaya teks memenuhi lebar layar, dan ukuran teks tetap 16sp agar mudah dibaca.

10. `star_item.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, deklarasi root `LinearLayout` vertikal yang membungkus satu item rasi bintang. Layout-nya `match_parent` lebar, tinggi menyesuaikan isi. `Padding 8dp` biar ga terlalu mepet tepi.
- Pada line 5, tambahan namespace `app` agar bisa memakai properti khusus seperti `shapeAppearanceOverlay`.
- Pada line 9–14, `CardView` untuk memberi tampilan seperti kartu. Diberi `margin 8dp`, sudut melengkung (`cardCornerRadius=16dp`), dan bayangan (`cardElevation=4dp`) biar ada efek ngambang.
- Pada line 16–20, `LinearLayout` horizontal di dalam `CardView` untuk menampilkan dua sisi, yaitu kiri (gambar) & kanan (info). `Padding 8dp` untuk ruang dalam.

- Pada line 22-28, `ShapeableImageView (@id/img)` untuk menampilkan gambar rasi. Ukuran 100x150dp, crop tengah, dan dibuat rounded lewat `shapeAppearanceOverlay`.
- Pada line 30-34, `LinearLayout` vertikal dengan `layout_weight="1"` supaya mengisi sisa ruang di sebelah kanan gambar.
- Pada line 36-40, `LinearLayout` horizontal untuk baris pertama info: nama rasi dan tahun. `gravity="center_vertical"` agar teks sejajar tengah secara vertikal.
- Pada line 42-49, `TextView (@id/tvName)` untuk nama rasi bintang, pakai `layout_weight="1"` agar fleksibel. Ukuran teks 16sp, bold, responsif ke sisa ruang.
- Pada line 51-56, `TextView (@id/tvYear)` untuk menampilkan tahun, posisinya di sebelah kanan nama.
- Pada line 59-67, `TextView (@id/tvDescription)` untuk deskripsi singkat. Margin atas 4dp, teks 14sp, maksimal 3 baris, dan `ellipsize="end"` artinya kalau terlalu panjang, akan dipotong dan dikasih tanda titik tiga (...).
- Pada line 69-89, `LinearLayout` horizontal untuk dua tombol: Website & Detail. Margin atas 8dp biar terpisah dari deskripsi.
- Pada line 75-80, `Button (@id/btnWeb)` untuk buka URL website rasi. Pakai `layout_weight="1"` untuk membagi ruang setengah layar.
- Pada line 82-88, `Button (@id/btnDetail)` untuk masuk ke halaman detail rasi. Dikasih `layout_marginStart="8dp"` agar tidak mepet ke tombol sebelumnya.

11. nav_graph.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.

- Pada line 2, terdapat elemen root `<navigation>` yang menjaji tempat mengatur alur navigasi antar fragment. Terdapat namespace android untuk atribut standar Android.
- Pada line 3, namespace app untuk properti khusus dari Android Jetpack Navigation.
- Pada line 4, `android:id="@+id/nav_graph"` adalah ID untuk grafik navigasi ini.
- Pada line 5, `app:startDestination="@id/homeFragment"` menunjukkan bahwa homeFragment adalah layar awal saat aplikasi dibuka.
- Pada line 7-14, deklarasi fragment untuk homeFragment (`@+id/homeFragment`).
- Pada line 9, menunjukkan bahwa kelas fragment-nya adalah `com.example.starconstellations.HomeFragment`
- Pada line 10, menunjukkan bahwa label yang muncul di toolbar adalah "Home"
- Pada line 11-13, terdapat `<action>` yang mengatur aksi `action_homeFragment_to_detailFragment` untuk navigasi dari Home ke Detail. `app:destination="@id/detailFragment"` menunjukkan tujuan navigasinya
- Pada line 14-18, deklarasi fragment untuk detailFragment (`@+id/detailFragment`).
- Pada line 18, menunjukkan bahwa kelas fragment-nya: `com.example.starconstellations.DetailFragment`
- Pada line 19, menunjukkan bahwa nama label adalah "Detail"
- Pada line 20-22, terdapat argument bernama "constellation". Hal ini digunakan untuk mengirim data objek bertipe `com.example.starconstellations.Constellation` dari Home ke Detail.

12. styles.xml:

- Pada line 1, deklarasi encoding XML UTF-8.
- Pada line 2, terdapat `<resources>` yang merupakan root element untuk menyimpan semua resource seperti style, color, dll.
- Pada line 3, deklarasi style baru bernama `RoundedImageView`, tidak mewarisi style lain (`parent=""`).
- Pada line 4, terdapat `cornerFamily` yang di-set ke `rounded` artinya semua sudut akan dibulatkan.
- Pada line 5, `cornerSize` di-set ke `16dp` artinya radius lengkung sudut sebesar `16dp`.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

1. Kompatibilitas dan Dukungan: RecyclerView telah ada lebih lama dan memiliki dukungan yang luas di berbagai versi Android. Selain itu, Banyak aplikasi yang sudah dibangun sebelumnya menggunakan RecyclerView, sehingga sulit untuk melakukan migrasi ke LazyColumn.
2. Fleksibilitas dan Kontrol: RecyclerView menawarkan fleksibilitas yang tinggi dan dapat dikustom dengan berbagai cara, seperti mengatur layout manager (Linear, Grid, Staggered), animasi item, membuat item dekorasi khusus sesuai kebutuhan aplikasi, bahkan integrasi dengan sistem kompleks (misalnya paging, nested scrolling).
3. Performa: RecyclerView dirancang untuk menangani daftar data yang sangat besar dengan efisien. Dengan mekanisme view recycling, RecyclerView dapat mengurangi penggunaan memori dan meningkatkan performa aplikasi.
4. Ekosistem yang Kuat dan Matang: Terdapat banyak library dan alat bantu yang telah dikembangkan untuk bekerja dengan RecyclerView, seperti DiffUtil untuk pembaruan data yang efisien, dan berbagai adapter library yang mempermudah pengelolaan data.
5. Dokumentasi dan Komunitas: RecyclerView sudah lama digunakan, sehingga sudah banyak dokumentasi dan komunitasnya sangat luas.

Jadi, meskipun LazyColumn menawarkan kode yang lebih singkat dan mudah dipahami, terutama dalam konteks Jetpack Compose, RecyclerView tetap menjadi pilihan yang kuat dan relevan untuk banyak proyek, terutama jika membutuhkan kompatibilitas dan fleksibilitas tinggi.

MODUL 4 : ViewModel and Debugging

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory dalam pembuatan ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

1. MainActivity.kt

Tabel 19. Source Code Jawaban Soal 1 Modul 4 (MainActivity.kt)

1	package com.example.modul4
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.navigation.fragment.NavHostFragment
6	import
7	androidx.navigation.ui.setupActionBarWithNavController
8	import com.example.

9	modul4.databinding.ActivityMainBinding
10	
11	class MainActivity : AppCompatActivity() {
12	
13	private lateinit var binding: ActivityMainBinding
14	
15	override fun onCreate(savedInstanceState: Bundle?) {
16	super.onCreate(savedInstanceState)
17	
18	binding =
19	ActivityMainBinding.inflate(layoutInflater)
20	setContentView(binding.root)
21	
22	setSupportActionBar(binding.toolbar)
23	
24	val navHostFragment = supportFragmentManager
25	.findFragmentById(R.id.nav_host_fragment) as
26	NavHostFragment
27	val navController =
28	navHostFragment.navController
29	
30	setUpActionBarWithNavController(navController)
31	}
32	
33	override fun onSupportNavigateUp(): Boolean {
34	val navHostFragment = supportFragmentManager
	.findFragmentById(R.id.nav_host_fragment) as
	NavHostFragment
	return
	navHostFragment.navController.navigateUp()
	super.onSupportNavigateUp()
	}
	}

2. Constellation.kt

Tabel 20. Source Code Jawaban Soal 1 Modul 4 (Constellation.kt)

1	package com.example.modul4
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Constellation(8 val name: String, 9 val description: String, 10 val year: String, 11 val imageResId: Int, 12 val webUrl: String 13) : Parcelable

3. ConstellationData.kt

Tabel 21. Source Code Jawaban Soal 1 Modul 4 (ConstellationData.kt)

1	package com.example. modul4
2	
3	object ConstellationData {
4	val listConstellations = listOf(5 Constellation(6 name = "Hydra", 7 description = "Hydra, alias sang ular air (The 8 Water Serpent), adalah rasi bintang terpanjang dan terbesar 9 di langit malam. Luasnya sebesar 1.303 derajat persegi 10 (~3,16% dari seluruh langit malam). Bentuknya menyerupai 11 ular air dan membentang dari selatan Cancer hingga Libra. 12 Meski ukurannya besar, hanya sedikit bintang terang yang 13 terlihat. Hydra dikenal dalam mitologi Yunani sebagai ular

14	berkepala banyak yang dikalahkan oleh Herkules.",
15	imageResId = R.drawable.hydra,
16	year = "420 juta tahun",
17	webUrl =
18	"https://en.m.wikipedia.org/wiki/Hydra_(constellation)"
19	),
20	Constellation(
21	name = "Virgo",
22	description = "Virgo, alias sang perawan suci
23	(The Maiden), adalah rasi bintang terbesar kedua dan salah
24	satu dari zodiak. Luasnya sebesar 1.294 derajat persegi
25	(~3,14% dari langit malam). Bintang terangnya, Spica, sangat
26	mudah dikenali. Virgo melambangkan dewi panen dan kesuburan
27	dalam berbagai mitologi, termasuk Yunani dan Romawi. Rasi
28	ini juga penting dalam astronomi karena menjadi lokasi
29	superklaster galaksi yang dinamakan Virgo Cluster.",
30	imageResId = R.drawable.virgo,
31	year = "12-400 juta tahun",
32	webUrl =
33	"https://en.m.wikipedia.org/wiki/Virgo_(constellation)"
34	),
35	Constellation(
36	name = "Ursa Major",
37	description = "Ursa Major, alias sang beruang
38	besar (The Great Bear), adalah salah satu rasi bintang
39	paling dikenal karena mengandung asterisma Big Dipper
40	(Gayung Besar). Luasnya sebesar 1.280 derajat persegi
41	(~3,11% dari langit malam). Rasi ini penting dalam navigasi
	karena dua bintangnya menunjuk ke Polaris (Bintang Utara).",
	imageResId = R.drawable.ursa_major,
	year = "300 juta - 1 miliar tahun",
	webUrl =
	"https://en.m.wikipedia.org/wiki/Ursa_Major"

	<pre>), Constellation(name = "Cetus", description = "Cetus, alias sang monster laut (The Sea Monster), dikenal sebagai 'Ikan Paus' atau 'Monster Laut' dalam mitologi Yunani. Luasnya sebesar 1.231 derajat persegi (~2,99% dari langit malam). Rasi ini cukup besar dan terletak di dekat rasi Pisces dan Eridanus. Meskipun luas, Cetus tidak memiliki banyak bintang terang, namun menjadi rumah bagi salah satu bintang variabel terkenal, Mira.", imageResId = R.drawable.cetus, year = "2 miliar tahun", webUrl = "https://en.m.wikipedia.org/wiki/Cetus"), Constellation(name = "Hercules", description = "Hercules, alias sang pahlawan kuat, dinamai dari pahlawan mitologi Yunani, yaitu 'Heracles'. Luasnya sebesar 1.225 derajat persegi (~2,97% dari langit malam). Bentuknya tidak mudah dikenali, tapi rasi ini memiliki formasi yang dikenal sebagai \"Keystone\" (batu penjuru). Salah satu daya tariknya adalah Gugus Bola M13, salah satu gugus bintang paling terang yang bisa dilihat dengan teleskop kecil.", imageResId = R.drawable.hercules, year = " 11 miliar tahun", webUrl = "https://en.m.wikipedia.org/wiki/Hercules_(constellation)"),) } </pre>
--	--

4. HomeFragment.kt

Tabel 22. Source Code Jawaban Soal 1 Modul 4 (HomeFragment.kt)

1	package com.example.modul4
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.LifecycleScope
12	import androidx.navigation.fragment.findNavController
13	import androidx.recyclerview.widget.LinearLayoutManager
14	import com.example.modul4.databinding.FragmentHomeBinding
15	import kotlinx.coroutines.flow.collectLatest
16	import kotlinx.coroutines.launch
17	
18	class HomeFragment : Fragment() {
19	
20	private var _binding: FragmentHomeBinding? = null
21	private val binding get() = _binding!!
22	
23	private val viewModel: MainViewModel by viewModels {
24	MainViewModelFactory() }
25	
26	private lateinit var adapter: ListConstellationsAdapter
27	
28	override fun onCreateView(
29	inflater: LayoutInflater, container: ViewGroup?,
30	savedInstanceState: Bundle?
31	): View {

```

32         _binding = FragmentHomeBinding.inflate(inflater,
33 container, false)
34         return binding.root
35     }
36
37     override fun onViewCreated(view: View,
38 savedInstanceState: Bundle?) {
39         super.onViewCreated(view, savedInstanceState)
40
41         adapter = ListConstellationsAdapter(
42             onWebClick = { constellation ->
43                 viewModel.onWebClicked(constellation)
44                 val intent = Intent(Intent.ACTION_VIEW,
45 Uri.parse(constellation.webUrl))
46                 startActivity(intent)
47             },
48             onDetailClick = { constellation ->
49                 viewModel.onDetailClicked(constellation)
50                 val action =
51 HomeFragmentDirections.actionHomeFragmentToDetailFragment(co
42 nstellatation)
53                 findNavController().navigate(action)
54             }
55         )
56
57         binding.rvStars.layoutManager =
58 LinearLayoutManager(requireContext())
59         binding.rvStars.adapter = adapter
60
61         lifecycleScope.launch {
62             viewModel.constellations.collectLatest {
63                 adapter.submitList(it)
64             }

```

65	<pre> } } override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
----	---

5. DetailFragment.kt

Tabel 23. Source Code Jawaban Soal 1 Modul 4 (DetailFragment.kt)

1	<code>package com.example.modul4</code>
2	
3	<code>import android.os.Bundle</code>
4	<code>import android.view.LayoutInflater</code>
5	<code>import android.view.View</code>
6	<code>import android.view.ViewGroup</code>
7	<code>import androidx.fragment.app.Fragment</code>
8	<code>import androidx.navigation.fragment.navArgs</code>
9	<code>import com.bumptech.glide.Glide</code>
10	<code>import com.example.modul4.databinding.FragmentDetailBinding</code>
11	
12	<code>class DetailFragment : Fragment() {</code>
13	
14	<code> private var _binding: FragmentDetailBinding? = null</code>
15	<code> private val binding get() = _binding!!</code>
16	<code> private val args: DetailFragmentArgs by navArgs()</code>
17	
18	<code> override fun onCreateView(</code>
19	<code> inflater: LayoutInflater, container: ViewGroup?,</code>
20	<code> savedInstanceState: Bundle?</code>
21	<code>): View {</code>
22	<code> _binding = FragmentDetailBinding.inflate(inflater,</code>

23	container, false)
24	return binding.root
25	}
26	
27	override fun onCreateView(view: View,
28	savedInstanceState: Bundle?) {
29	super.onCreateView(view, savedInstanceState)
30	
31	val data = args.constellation
32	
33	binding.apply {
34	detailName.text = data.name
35	detailDescription.text = data.description
36	detailYear.text = data.year
37	
38	Glide.with(this@DetailFragment).load(data.imageResId).into(d
39	etailImage)
40	}
41	
42	}
43	
44	override fun onDestroyView() {
	super.onDestroyView()
	_binding = null
	}
	}

6. ListConstellationsAdapter.kt

Tabel 24. Source Code Jawaban Soal 1 Modul 4 (ListConstellationsAdapter.kt)

1	package com.example.modul4
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup

```

5 import androidx.recyclerview.widget.DiffUtil
6 import androidx.recyclerview.widget.ListAdapter
7 import androidx.recyclerview.widget.RecyclerView
8 import com.bumptech.glide.Glide
9 import com.example.modul4.databinding.StarItemBinding
10
11 class ListConstellationsAdapter(
12     private val onClick: (Constellation) -> Unit,
13     private val onDetailClick: (Constellation) -> Unit
14 ) : ListAdapter<Constellation,
15 ListConstellationsAdapter.ViewHolder>(DIFF_CALLBACK) {
16
17     inner class ViewHolder(val binding: StarItemBinding) :
18 RecyclerView.ViewHolder(binding.root)
19
20     override fun onCreateViewHolder(parent: ViewGroup,
21 viewType: Int): ViewHolder {
22         val binding =
23 StarItemBinding.inflate(LayoutInflater.from(parent.context),
24 parent, false)
25         return ViewHolder(binding)
26     }
27
28     override fun onBindViewHolder(holder: ViewHolder,
29 position: Int) {
30         val item = getItem(position)
31         with(holder.binding) {
32             tvName.text = item.name
33             tvDescription.text = item.description
34             tvYear.text = item.year
35
36 Glide.with(img.context).load(item.imageResId).into(img)
37

```

38	btnWeb.setOnClickListener { onWebClick(item) }
39	btnDetail.setOnClickListener {
40	onDetailClick(item) }
41	}
42	}
43	
44	companion object {
45	val DIFF_CALLBACK = object :
46	DiffUtil.ItemCallback<Constellation>() {
47	override fun areItemsTheSame(oldItem:
	Constellation, newItem: Constellation): Boolean {
	return oldItem.name == newItem.name
	}
	override fun areContentsTheSame(oldItem:
	Constellation, newItem: Constellation): Boolean {
	return oldItem == newItem
	}
	}
	}
	}

7. activity_main.xml

Tabel 25. Source Code Jawaban Soal 1 Modul 4 (activity_main.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/android"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent">
8	
9	<androidx.appcompat.widget.Toolbar

10	android:id="@+id/toolbar"
11	android:layout_width="match_parent"
12	android:layout_height="?attr/actionBarSize"
13	android:background="?attr/colorPrimary"
14	android:elevation="4dp"
15	app:titleTextColor="@android:color/white" />
16	
17	<androidx.fragment.app.FragmentContainerView
18	android:id="@+id/nav_host_fragment"
19	
20	android:name="androidx.navigation.fragment.NavHostFragment"
21	android:layout_width="match_parent"
22	android:layout_height="match_parent"
23	android:layout_marginTop="?attr/actionBarSize"
24	app:defaultNavHost="true"
25	app:navGraph="@navigation/nav_graph" />
	</androidx.coordinatorlayout.widget.CoordinatorLayout>

8. fragment_home.xml

Tabel 26. Source Code Jawaban Soal 1 Modul 4 (fragment_home.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:id="@+id/homeFragment"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="8dp">
8	
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/rvStars"
11	android:layout_width="match_parent"
12	android:layout_height="match_parent"

13	android:clipToPadding="false" />
14	</FrameLayout>

9. fragment_detail.xml

Tabel 27. Source Code Jawaban Soal 1 Modul 4 (fragment_detail.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/detailFragment"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:padding="16dp">
9	
10	<LinearLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:orientation="vertical"
14	android:gravity="center_horizontal">
15	
16	<ImageView
17	android:id="@+id/detailImage"
18	android:layout_width="350dp"
19	android:layout_height="400dp"
20	android:scaleType="centerCrop"
21	android:layout_marginBottom="16dp"
22	
23	app:shapeAppearanceOverlay="@style/RoundedImageView" />
24	
25	<TextView
26	android:id="@+id/detailName"
27	android:layout_width="wrap_content"

28	android:layout_height="wrap_content"
29	android:text="Hydra"
30	android:textSize="22sp"
31	android:textStyle="bold"
32	android:layout_marginBottom="8dp" />
33	
34	<TextView
35	android:id="@+id/detailYear"
36	android:layout_width="wrap_content"
37	android:layout_height="wrap_content"
38	android:text=""
39	android:textSize="16sp"
40	android:layout_marginBottom="16dp" />
41	
42	<TextView
43	android:id="@+id/detailDescription"
44	android:layout_width="match_parent"
45	android:layout_height="wrap_content"
46	android:text="This constellation is known
47	for..."
	android:textSize="16sp" />
	</LinearLayout>
	</ScrollView>

10. star_item.xml

Tabel 28. Source Code Jawaban Soal 1 Modul 4 (star_item.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	android:orientation="vertical"

8	android:padding="8dp">
9	
10	<androidx.cardview.widget.CardView
11	xmlns:card_view="http://schemas.android.com/apk/res-auto"
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:layout_margin="8dp"
15	card_view:cardCornerRadius="16dp"
16	card_view:cardElevation="4dp">
17	
18	<LinearLayout
19	android:layout_width="match_parent"
20	android:layout_height="wrap_content"
21	android:orientation="horizontal"
22	android:padding="8dp">
23	
24	
25	<com.google.android.material.imageview.ShapeableImageView
26	android:id="@+id/img"
27	android:layout_width="100dp"
28	android:layout_height="150dp"
29	android:scaleType="centerCrop"
30	android:layout_marginEnd="8dp"
31	
32	app:shapeAppearanceOverlay="@style/RoundedImageView" />
33	
34	<LinearLayout
35	android:layout_width="0dp"
36	android:layout_height="match_parent"
37	android:layout_weight="1"
38	android:orientation="vertical">
39	
40	<LinearLayout

41	android:layout_width="match_parent"
42	android:layout_height="wrap_content"
43	android:orientation="horizontal"
44	android:gravity="center_vertical">
45	
46	<TextView
47	android:id="@+id/tvName"
48	android:layout_width="0dp"
49	android:layout_height="wrap_content"
50	android:layout_weight="1"
51	android:text="Orion"
52	android:textStyle="bold"
53	android:textSize="16sp" />
54	
55	<TextView
56	android:id="@+id/tvYear"
57	android:layout_width="wrap_content"
58	android:layout_height="wrap_content"
59	android:text=""
60	android:textSize="14sp" />
61	</LinearLayout>
62	
63	<TextView
64	android:id="@+id/tvDescription"
65	android:layout_width="match_parent"
66	android:layout_height="wrap_content"
67	android:text="One of the brightest
68	constellations..."
69	android:textSize="14sp"
70	android:layout_marginTop="4dp"
71	android:maxLines="3"
72	android:ellipsize="end" />
73	

74	<LinearLayout
75	android:layout_width="match_parent"
76	android:layout_height="wrap_content"
77	android:orientation="horizontal"
78	android:layout_marginTop="8dp">
79	
80	<Button
81	android:id="@+id/btnWeb"
82	android:layout_width="0dp"
83	android:layout_height="wrap_content"
84	android:layout_weight="1"
85	android:text="Website" />
86	
87	<Button
88	android:id="@+id/btnDetail"
89	android:layout_width="0dp"
90	android:layout_height="wrap_content"
91	android:layout_weight="1"
92	android:text="Detail"
93	android:layout_marginStart="8dp" />
	</LinearLayout>
	</LinearLayout>
	</LinearLayout>
	</androidx.cardview.widget.CardView>
	</LinearLayout>

11. nav_graph.xml

Tabel 29. Source Code Jawaban Soal 1 Modul 4 (nav_graph.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_graph"

6	app:startDestination="@id/homeFragment">
7	
8	<fragment
9	android:id="@+id/homeFragment"
10	android:name="com.example.modul4.HomeFragment"
11	android:label="Home">
12	<action
13	
14	android:id="@+id/action_homeFragment_to_detailFragment"
15	app:destination="@id/detailFragment" />
16	</fragment>
17	
18	<fragment
19	android:id="@+id/detailFragment"
20	android:name="com.example.modul4.DetailFragment"
21	android:label="Detail">
22	<argument
23	android:name="constellation"
24	app:argType="com.example.modul4.Constellation"
	/>
	</fragment>
	</navigation>

12. styles.xml

Tabel 30. Source Code Jawaban Soal 1 Modul 4 (styles.xml)

1	<?xml version="1.0" encoding="utf-8"?>
1	<resources>
2	<style name="RoundedImageView" parent="">
3	<item name="cornerFamily">rounded</item>
4	<item name="cornerSize">16dp</item>
5	</style>
6	</resources>
7	

13. MainViewModel.kt

Tabel 31. Source Code Jawaban Soal 1 Modul 4 (MainViewModel.kt)

1	package com.example.modul4
2	
3	import android.util.Log
4	import androidx.lifecycle.ViewModel
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	
8	class MainViewModel : ViewModel() {
9	
10	private val _constellations =
11	MutableStateFlow<List<Constellation>>(emptyList())
12	val constellations: StateFlow<List<Constellation>> =
13	_constellations
14	
15	private val _selectedItem =
16	MutableStateFlow<Constellation?>(null)
17	val selectedItem: StateFlow<Constellation?> =
18	_selectedItem
19	
20	init {
21	val list = ConstellationData.listConstellations
22	_constellations.value = list
23	Log.d("ViewModel", "Data item masuk ke dalam list:
24	\$list")
25	}
26	
27	fun onDetailClicked(constellation: Constellation) {
28	_selectedItem.value = constellation
29	Log.d("ViewModel", "Tombol Detail ditekan. Item:
30	\$constellation")
	}

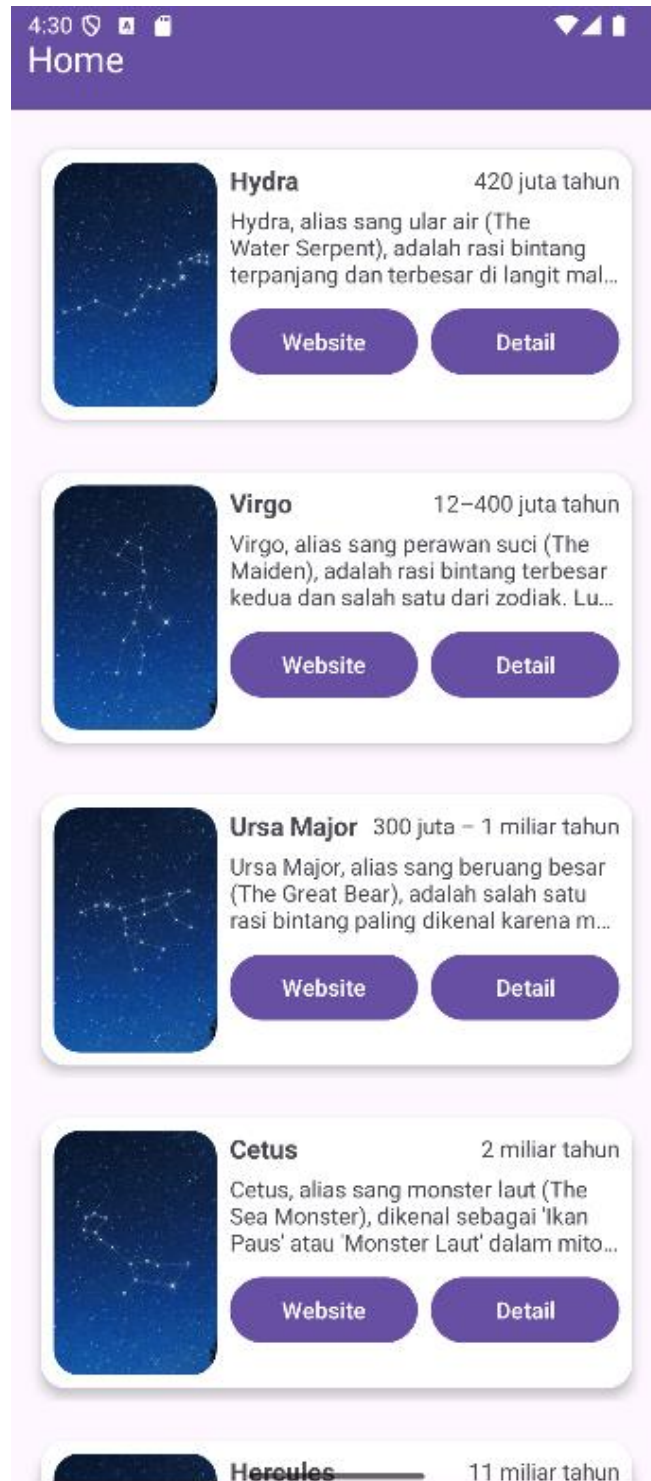
	<pre> fun onWebClicked(constellation: Constellation) { Log.d("ViewModel", "Tombol Explicit Intent ditekan. URL: \${constellation.webUrl}") } } </pre>
--	---

14. MainViewModelFactory.kt

Tabel 32. Source Code Jawaban Soal 1 Modul 4 (MainViewModelFactory.kt)

1	package com.example.modul4
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class MainViewModelFactory : ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass:
8	Class<T>): T {
9	if
10	(modelClass.isAssignableFrom(MainViewModel::class.java)) {
11	return MainViewModel() as T
12	}
13	throw IllegalArgumentException("Unknown ViewModel
	class")
	}
	}

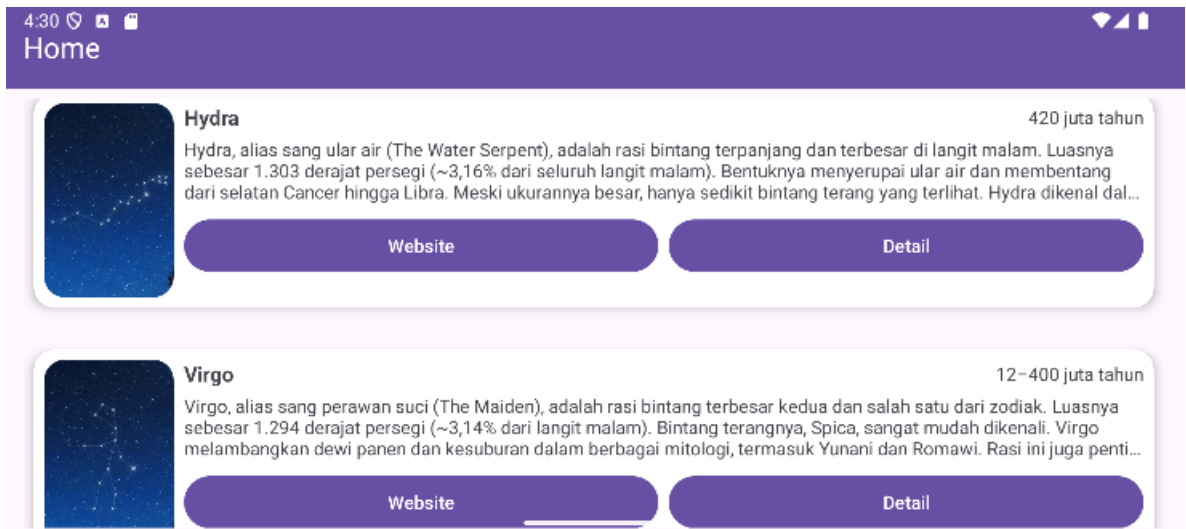
B. Output Program



Gambar 19. Screenshot Hasil Jawaban Soal 1 Modul 4 (List Portrait)



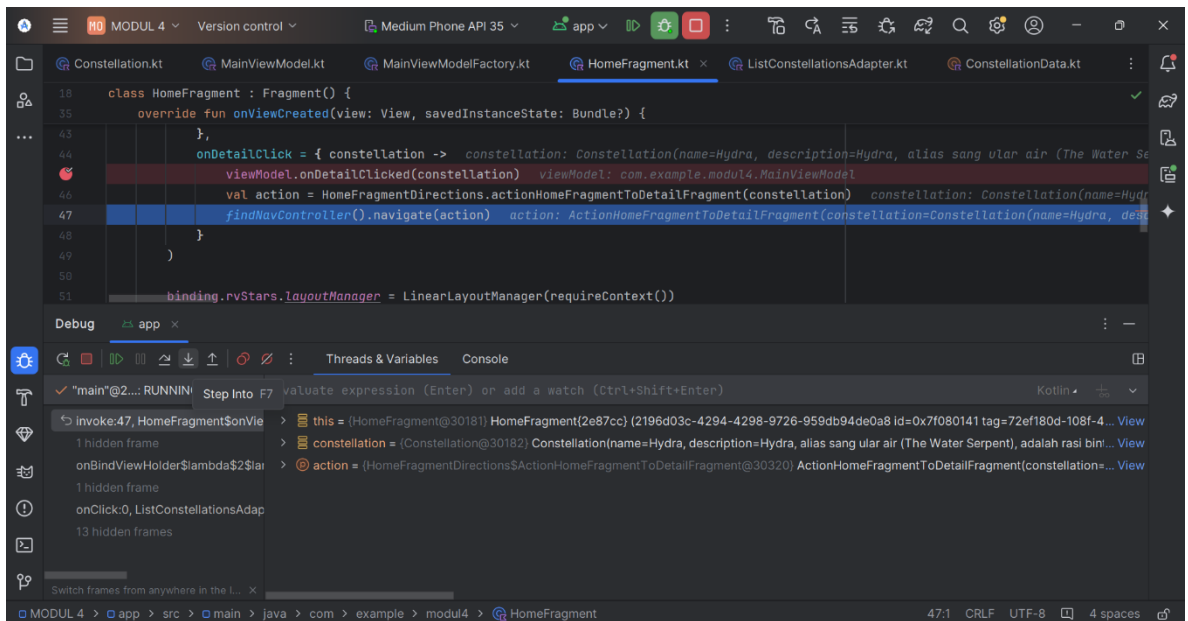
Gambar 20. Screenshot Hasil Jawaban Soal 1 Modul 4 (Detail Portrait)



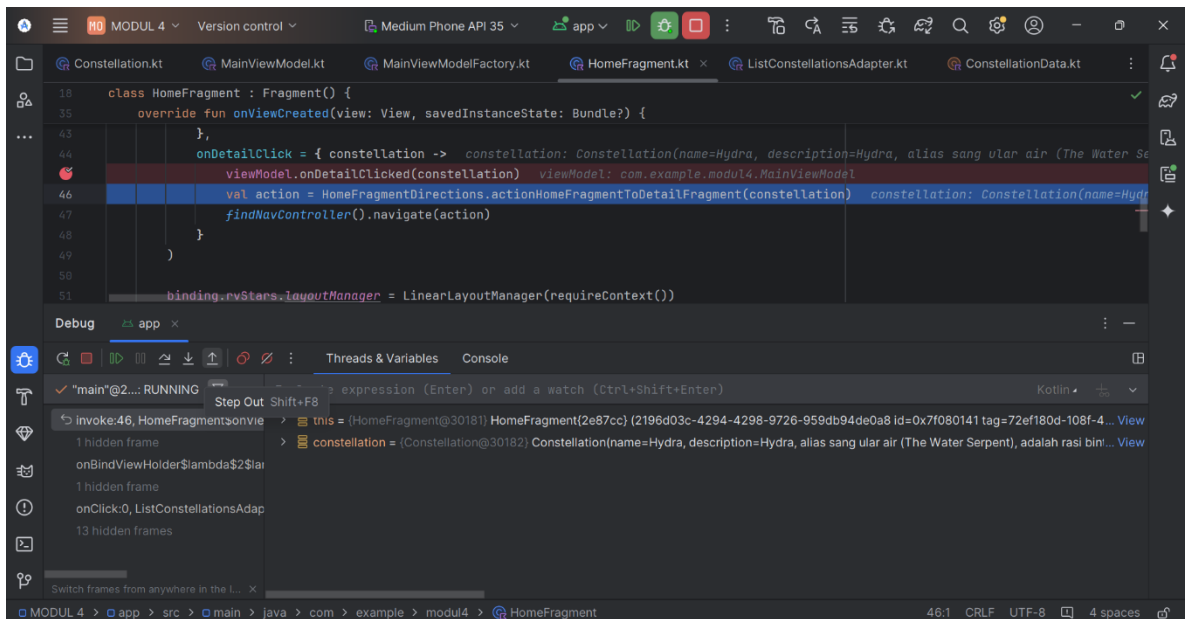
Gambar 21. Screenshot Hasil Jawaban Soal 1 Modul 4 (List Landscape)



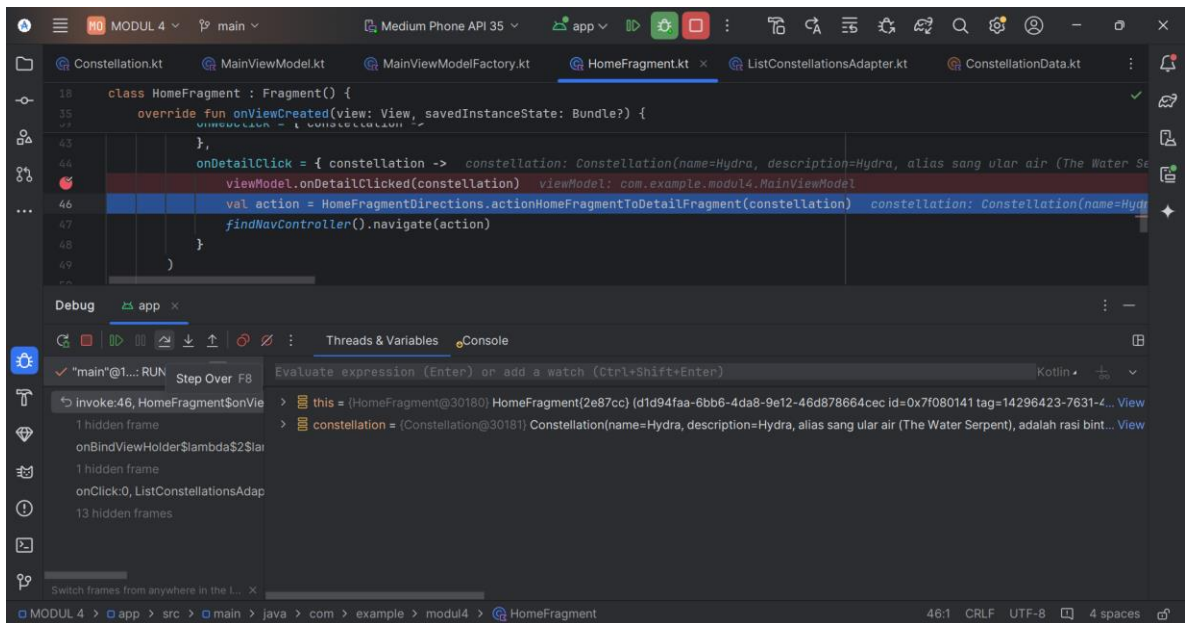
Gambar 22. Screenshot Hasil Jawaban Soal 1 Modul 4 (Detail Landscape)



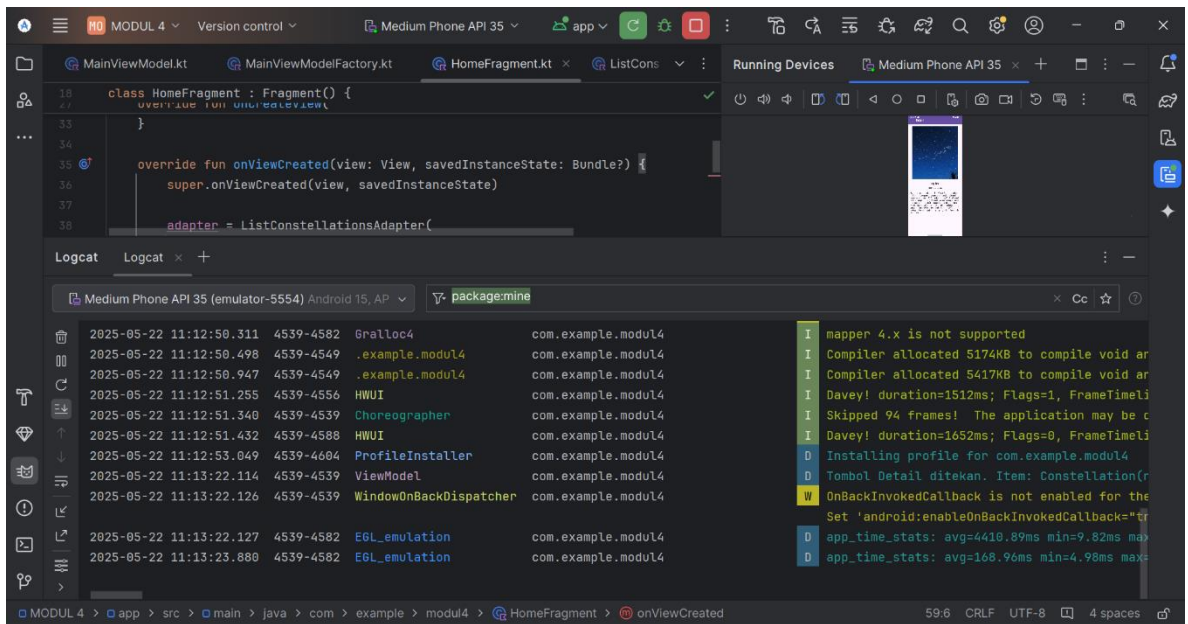
Gambar 23. Screenshot Hasil Jawaban Soal 1 Modul 4 (Step Into)



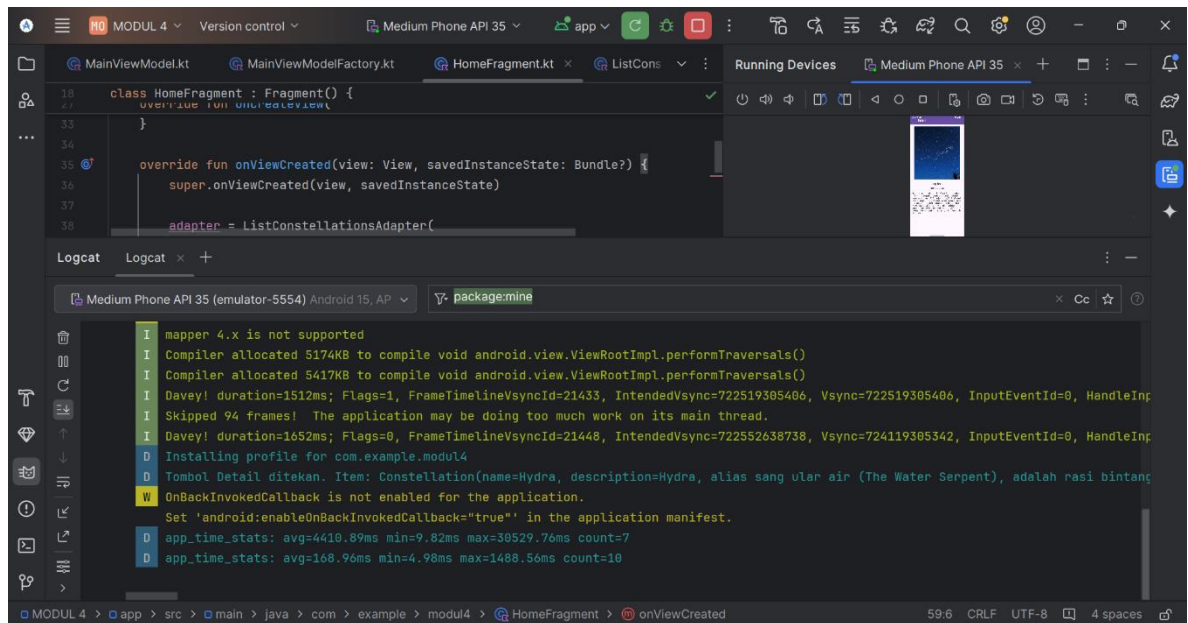
Gambar 24. Screenshot Hasil Jawaban Soal 1 Modul 4 (Step Out)



Gambar 25. Screenshot Hasil Jawaban Soal 1 Modul 4 (Step Over)



Gambar 26. Screenshot Hasil Jawaban Soal 1 Modul 4 (Logcat)



Gambar 27. Screenshot Hasil Jawaban Soal 1 Modul 4 (Validasi Logcat)

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. MainActivity.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Bundle` untuk menyimpan status activity saat di-pause atau di-rotate.
- Pada line 4, mengimpor `AppCompatActivity` untuk menggunakan fitur kompatibilitas activity pada aplikasi.
- Pada line 5, mengimpor `NavHostFragment` untuk mendukung navigasi menggunakan fragment.
- Pada line 6, mengimpor `setupActionBarWithNavController` untuk mengatur aksi bar dengan navigasi controller.
- Pada line 7, mengimpor `ActivityMainBinding` untuk binding layout dari XML ke dalam activity.

- Pada line 9, mendeklarasikan kelas `MainActivity` yang mewarisi `AppCompatActivity`.
- Pada line 11, mendeklarasikan properti binding untuk akses komponen UI melalui `ViewBinding`.
- Pada line 13-17, membuat `onCreate`, meng-inflate layout dan mengatur konten view menggunakan binding.
- Pada line 19, menyetel toolbar sebagai action bar untuk activity.
- Pada line 21-23, mendapatkan `NavHostFragment` dan `NavController` untuk mendukung navigasi antar fragment.
- Pada line 25, menyiapkan action bar untuk bekerja dengan `NavController` yang ada.
- Pada line 28-32, terdapat Override `onSupportNavigateUp` untuk meng-handle navigasi kembali saat menekan tombol up di action bar.

2. Constellation.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Parcelable` untuk memungkinkan objek dikirim antar komponen Android.
- Pada line 4, mengimpor `Parcelize` untuk menyederhanakan implementasi `Parcelable` pada kelas.
- Pada line 6, terdapat anotasi `@Parcelize` digunakan untuk otomatis mengimplementasikan `Parcelable`.
- Pada line 7-12, mendefinisikan kelas data `Constellation` dengan properti: `name`, `description`, `year`, `imageResId`, dan `webViewUrl`.
- Pada line 13, terdapat `Parcelable` yang artinya kelas `Constellation` mengimplementasikan `Parcelable` untuk mendukung pengiriman objek antar komponen.

3. ConstellationData.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mendeklarasikan objek `ConstellationData`, yang menyimpan data statis tentang konstelasi bintang.
- Pada line 4, mendeklarasikan properti `listConstellations` sebagai daftar (list) objek `Constellation`.
- Pada line 5-39, menambahkan beberapa objek `Constellation` ke dalam `listConstellations`, masing-masing dengan `name` (nama konstelasi), `description` (deskripsi singkat mengenai konstelasi), `imageResId` (ID gambar konstelasi), `year` (perkiraan usia konstelasi), dan `webUrl` (URL untuk informasi lebih lanjut mengenai konstelasi).

4. HomeFragment.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Intent` dari Android, yang digunakan untuk melakukan operasi seperti membuka browser atau berpindah Activity.
- Pada line 4, mengimpor `Uri` dari Android, yang digunakan untuk memarsing dan merepresentasikan URL.
- Pada line 5, mengimpor `Bundle` yang digunakan untuk menyimpan dan memulihkan data saat fragment dibuat.
- Pada line 6, mengimpor `LayoutInflater` yang digunakan untuk menampilkan layout XML ke dalam objek View.
- Pada line 7, mengimpor `View`, komponen dasar antarmuka pengguna Android.
- Pada line 8, mengimpor `ViewGroup`, elemen yang bisa berisi banyak View.

- Pada line 9, mengimpor `Fragment`, kelas dasar untuk membuat bagian UI yang modular di Android.
- Pada line 10, mengimpor fungsi `viewModels`, digunakan untuk memperoleh instance `ViewModel` dengan pendekatan delegasi.
- Pada line 11, mengimpor `lifecycleScope` untuk menjalankan coroutine yang mengikuti lifecycle `Fragment`.
- Pada line 12, mengimpor `findNavController` untuk melakukan navigasi antar-fragment.
- Pada line 13, mengimpor `LinearLayoutManager` yang digunakan untuk menampilkan item di `RecyclerView` secara vertikal.
- Pada line 14, mengimpor binding otomatis `FragmentHomeBinding`, yang dibuat dari layout XML `fragment_home.xml`.
- Pada line 15, mengimpor fungsi `collectLatest` dari Kotlin Flow untuk mengamati dan mengambil data terbaru.
- Pada line 16, mengimpor `launch` dari `Coroutine` untuk menjalankan coroutine di dalam `lifecycleScope`.
- Pada line 18, mendeklarasikan kelas `HomeFragment` yang merupakan turunan dari `Fragment`.
- Pada line 20, mendeklarasikan variabel `_binding` sebagai nullable `FragmentHomeBinding`, yang digunakan untuk mengakses elemen UI di XML.
- Pada line 21, mendefinisikan properti `binding` sebagai non-nullable dengan menggunakan `!!`, sehingga kita bisa langsung mengakses binding dengan aman selama view masih aktif.
- Pada line 23, membuat instance `viewModel` dari `MainViewModel` menggunakan delegasi `by` `viewModels` dan `MainViewModelFactory`.

- Pada line 25, mendeklarasikan variabel adapter dari tipe `ListConstellationsAdapter` untuk menampilkan daftar konstelasi.
- Pada line 27-33, mendefinisikan fungsi `onCreateView`, yang dipanggil saat fragment membuat UI-nya. `inflater` digunakan untuk meng-inflate layout XML ke dalam objek `View`.
- Pada line 31, `FragmentHomeBinding.inflate()` digunakan untuk menghubungkan XML layout dengan objek Kotlin.
- Pada line 32, mengembalikan `binding.root` sebagai tampilan utama dari fragment.
- Pada line 35-59, mendefinisikan fungsi `onViewCreated`, yang dipanggil setelah fragment view selesai dibuat.
- Pada line 38-49, menginisialisasi adapter dari `ListConstellationsAdapter` dengan dua lambda:
 - `onWebClick`: Menangani klik tombol web. Memanggil fungsi `onWebClicked()` di `ViewModel`, lalu membuat `Intent` eksplisit untuk membuka browser ke URL konstelasi.
 - `onDetailClick`: Menangani klik tombol detail. Memanggil fungsi `onDetailClicked()` di `ViewModel`, lalu membuat navigasi ke `DetailFragment` menggunakan `Safe Args`.
- Pada line 51-52, mengatur `layoutManager` dari `RecyclerView` agar menampilkan daftar item secara vertikal.
- Pada line 54-58, menggunakan `coroutine (lifecycleScope.launch)` untuk mengamati data `constellations` dari `ViewModel`. Jika ada perubahan data, maka akan diserahkan ke `adapter.submitList()` untuk ditampilkan ke UI.
- Pada line 61-64, mendefinisikan `onDestroyView`, yang dipanggil saat fragment dihancurkan. Di sini `binding` di-set ke `null` untuk mencegah memory leak.

5. DetailFragment.kt

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, diimpor `android.os.Bundle` untuk menangani data yang dikirim antar komponen dalam aplikasi, seperti data yang dikirim saat navigasi antar fragment.
- Pada line 4, diimpor `android.view.LayoutInflater` untuk meng-inflate layout XML ke dalam objek View di dalam fragment.
- Pada line 5, diimpor `android.view.View` untuk merepresentasikan elemen tampilan di dalam fragment.
- Pada line 6, diimpor `android.view.ViewGroup` untuk merepresentasikan container dari elemen tampilan dalam fragment.
- Pada line 7, diimpor `androidx.fragment.app.Fragment` untuk memungkinkan class ini mewarisi fungsionalitas dari fragment, seperti menangani lifecycle dan tampilan.
- Pada line 8, diimpor `androidx.navigation.fragment.navArgs` untuk mengakses argumen yang dikirim saat navigasi antar fragment menggunakan komponen Navigation.
- Pada line 9, diimpor `com.bumptech.glide.Glide` untuk memudahkan memuat dan menampilkan gambar dari URL atau sumber lainnya ke dalam ImageView.
- Pada line 10, diimpor `com.example.modul4.databinding.FragmentDetailBinding` untuk mengakses elemen-elemen di layout `fragment_detail.xml` menggunakan ViewBinding.
- Pada line 12, mendeklarasikan kelas `DetailFragment`, yang menampilkan detail konstelasi saat item dipilih dari `HomeFragment`.

- Pada line 14, mendeklarasikan properti `_binding` untuk mengakses komponen UI menggunakan `ViewBinding`.
- Pada line 15, mendeklarasikan properti `binding` untuk memudahkan akses ke `FragmentDetailBinding`.
- Pada line 10, menggunakan `navArgs` untuk mengambil data `constellation` yang dipassing dari `HomeFragment`.
- Pada line 18-24, membuat `onCreateView`, meng-inflate layout dan mengembalikan root view untuk fragment.
- Pada line 26-38, membuat `onViewCreated`, mengatur tampilan detail dengan data yang diterima, seperti menampilkan nama, deskripsi, dan tahun konstelasi, Llau, menggunakan `Glide` untuk memuat gambar konstelasi ke dalam `detailImage`.
- Pada line 40-43, membuat `onDestroyView`, membersihkan binding untuk menghindari memory leak.

6. ListConstellationsAdapter.kt:

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `LayoutInflater`, digunakan untuk meng-inflate layout XML menjadi objek `View`.
- Pada line 4, mengimpor `ViewGroup`, elemen yang bisa berisi banyak `View` dan biasanya menjadi parent layout.
- Pada line 5, mengimpor `DiffUtil`, utility untuk membandingkan data lama dan baru dalam `ListAdapter`.
- Pada line 6, mengimpor `ListAdapter`, adapter khusus yang otomatis mengelola perbedaan data menggunakan `DiffUtil`.
- Pada line 7, mengimpor `RecyclerView`, komponen untuk menampilkan daftar data secara efisien.

- Pada line 8, mengimpor Glide, library eksternal untuk memuat dan menampilkan gambar dari URL atau resource.
- Pada line 9, mengimpor `StarItemBinding`, kelas binding otomatis dari layout XML `star_item.xml`.
- Pada line 11–14, mendefinisikan kelas `ListConstellationsAdapter` yang merupakan turunan dari `ListAdapter`.
 - Adapter ini menerima dua parameter lambda: `onWebClick` dan `onDetailClick` untuk menangani event klik tombol Web dan Detail.
 - Constellation adalah tipe data yang ditampilkan dalam daftar.
 - `DIFF_CALLBACK` digunakan untuk mendeteksi perubahan data dengan efisien.
- Pada line 16, mendefinisikan inner class `ViewHolder` yang menerima `StarItemBinding`, digunakan untuk merepresentasikan setiap item di `RecyclerView`.
- Pada line 18–21, mengoverride fungsi `onCreateViewHolder`, yang bertanggung jawab membuat tampilan (View) untuk setiap item list.
 - `LayoutInflater.from(parent.context)` digunakan untuk meng-inflate layout XML.
 - `StarItemBinding.inflate()` menghasilkan objek binding dari layout `star_item.xml`.
- Pada line 23–34, mengoverride fungsi `onBindViewHolder`, untuk menghubungkan data ke tampilan UI berdasarkan posisi item.
 - `getItem(position)` mengambil data konstelasi berdasarkan posisi.
 - Dengan `with(holder.binding)`, kita mengakses elemen UI seperti `tvName`, `tvDescription`, dll.

- `Glide.with(img.context).load(item.imageResId).into(img)` digunakan untuk memuat gambar ke dalam `ImageView`.
- `btnWeb.setOnClickListener` dan `btnDetail.setOnClickListener` menetapkan aksi klik untuk masing-masing tombol.
- Pada line 36-46, mendeklarasikan objek `DIFF_CALLBACK` sebagai implementasi `DiffUtil.ItemCallback<Constellation>`.
 - Fungsi `areItemsTheSame` membandingkan apakah dua item memiliki ID atau key yang sama (di sini digunakan `name`).
 - Fungsi `areContentsTheSame` membandingkan isi dari dua item, apakah benar-benar identik (menggunakan `==`).

7. **MainViewModel.kt:**

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Log` dari Android, digunakan untuk mencetak pesan ke Logcat, biasanya untuk keperluan debugging.
- Pada line 4, mengimpor `ViewModel` dari Jetpack Lifecycle, yang digunakan untuk menyimpan dan mengelola data UI agar tetap bertahan ketika terjadi perubahan konfigurasi (misalnya rotasi layar).
- Pada line 5, mengimpor `MutableStateFlow` dari Kotlin Coroutines, yang merupakan state holder yang bisa diubah dan diamati untuk reactive programming.
- Pada line 6, mengimpor `StateFlow`, yaitu versi read-only dari `MutableStateFlow` yang bisa diamati oleh UI.
- Pada line 8, mendeklarasikan kelas `MainViewModel` yang merupakan subclass dari `ViewModel`.

- Pada line 10, mendeklarasikan properti privat `_constellations` bertipe `MutableStateFlow<List<Constellation>>` dan diinisialisasi dengan list kosong. Ini digunakan untuk menyimpan daftar konstelasi secara reaktif.
- Pada line 11, mendeklarasikan properti publik `constellations` sebagai `StateFlow<List<Constellation>>`, yaitu versi hanya-baca dari `_constellations` agar data tidak bisa diubah dari luar kelas.
- Pada line 13, mendeklarasikan properti privat `_selectedItem` bertipe `MutableStateFlow<Constellation?>` dan diinisialisasi dengan `null`. Ini digunakan untuk menyimpan konstelasi yang sedang dipilih oleh user.
- Pada line 14, mendeklarasikan properti publik `selectedItem` sebagai `StateFlow<Constellation?>`, yaitu versi read-only dari `_selectedItem`.
- Pada line 16, mendeklarasikan blok `init`, yaitu blok inisialisasi yang akan dijalankan saat `MainViewModel` pertama kali dibuat.
- Pada line 17, mengambil data konstelasi dari `ConstellationData.listConstellations` dan menyimpannya dalam variabel `list`.
- Pada line 18, mengatur nilai `_constellations` dengan `list` agar data konstelasi bisa diamati dari UI.
- Pada line 19, mencetak log dengan tag "ViewModel" dan pesan bahwa data item sudah dimasukkan ke dalam `list`.
- Pada line 22, mendefinisikan fungsi `onDetailClicked` dengan parameter `constellation` yang akan dipanggil ketika tombol detail diklik oleh pengguna.
- Pada line 23, mengubah nilai `_selectedItem` menjadi `constellation` yang dipilih, sehingga UI bisa merespons perubahan tersebut.

- Pada line 24, mencetak log bahwa tombol detail telah ditekan beserta item yang dipilih.
- Pada line 27, mendefinisikan fungsi `onWebClicked` dengan parameter `constellation` yang akan dipanggil ketika tombol menuju halaman web diklik.
- Pada line 28, mencetak log bahwa tombol web ditekan dan menampilkan URL dari konstelasi tersebut.

8. MainViewModelFactory.kt:

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `ViewModel` dari Jetpack Lifecycle, yang merupakan superclass untuk semua `ViewModel`.
- Pada line 4, mengimpor `ViewModelProvider` dari Jetpack Lifecycle, yang digunakan untuk membuat dan mengelola objek `ViewModel` dengan cara yang aman terhadap siklus hidup (lifecycle-aware).
- Pada line 6, mendeklarasikan kelas `MainViewModelFactory` yang mengimplementasikan `ViewModelProvider.Factory`. Kelas ini bertugas membuat instance `MainViewModel`.

9. activity_main.xml:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–25, terdapat elemen root `CoordinatorLayout` dipakai untuk mendukung layout berbasis koordinasi antar elemen, seperti toolbar dan fragment. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 5-6, terdapat `layout_width` & `layout_height` yang di-set ke `match_parent`, artinya mengisi seluruh layar.
- Pada line 8-14, membuat `Toolbar` yang ditambahkan sebagai action bar custom:

- `layout_height` mengikuti tinggi standar action bar (`?attr/actionBarSize`).
- `background` mengikuti warna utama tema (`colorPrimary`).
- `elevation 4dp` memberi efek bayangan.
- `titleTextColor` di-set ke putih.
- Pada line 16-23, terdapat `FragmentContainerView` untuk menampilkan fragment berdasarkan Navigation Component:
 - `layout_marginTop` disesuaikan dengan tinggi action bar supaya fragment tidak tertutup toolbar.
 - `android:name` menunjuk ke `NavHostFragment`, elemen utama navigasi.
 - `app:defaultNavHost="true"` menjadikan fragment ini sebagai host utama.
 - `app:navGraph` menunjuk ke file navigasi `nav_graph.xml`.
- Pada line 25, penutup `CoordinatorLayout`.

10. `fragment_home.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, terdapat root `FrameLayout` digunakan sebagai wadah layout fragment, dengan ID `homeFragment`, ukuran penuh, dan padding 8dp.
- Pada line 8-12, terdapat `RecyclerView` dengan ID `rvStars` digunakan untuk menampilkan daftar rasi bintang dalam list scrollable, memenuhi seluruh ruang yang tersedia dan `clipToPadding` diset false agar konten tetap bisa tampil meski melebihi padding.

11. `fragment_detail.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.

- Pada line 2, terdapat deklarasi `ScrollView` sebagai root layout yang memungkinkan isi halaman dapat discroll secara vertikal. Ini penting karena kontennya bisa lebih panjang dari tinggi layar.
- Pada line 3, terdapat atribut `xmlns` untuk mendefinisikan namespace `Android` dan `app`, yang wajib ada agar atribut khusus (seperti `app:shapeAppearanceOverlay`) bisa digunakan.
- Pada line 4, terdapat `android:id="@+id/detailFragment"` sebagai ID unik untuk `ScrollView`, meskipun tidak wajib jika tidak digunakan di kode.
- Pada line 6–7, `layout_width` dan `layout_height` diatur `match_parent` agar `ScrollView` mengisi seluruh layar.
- Pada line 7, `android:padding="16dp"` memberi jarak di sekeliling isi layout supaya tidak terlalu mepet ke tepi layar.
- Pada line 9–13, terdapat `LinearLayout` dengan orientasi vertikal dan `gravity center_horizontal`, artinya elemen di dalamnya akan ditumpuk ke bawah dan disejajarkan di tengah horizontal.
- Pada line 15–21, terdapat `ImageView (@id/detailImage)` untuk menampilkan gambar rasi bintang. Lebar nya 350dp dan tingginya 400dp, `scaleType="centerCrop"` agar gambar tetap proporsional meski di-crop, dan `layout_marginBottom="16dp"` memberi jarak ke bawah. Atribut `app:shapeAppearanceOverlay` digunakan untuk membuat gambar tampil dengan sudut membulat (rounded).
- Pada line 23–30, terdapat `TextView (@id/detailName)` untuk menampilkan nama rasi bintang, teks ditampilkan bold, ukuran besar (22sp), dan diberi jarak bawah 8dp agar rapi.
- Pada line 32–38, terdapat `TextView (@id/detailYear)` untuk menampilkan tahun atau data waktu terkait rasi bintang. Ukuran teks 16sp dan jarak bawah 16dp.

- Pada line 40-45, terdapat `TextView (@id/detailDescription)` yang menampilkan deskripsi panjang tentang rasi bintang. `layout_width` diatur `match_parent` supaya teks memenuhi lebar layar, dan ukuran teks tetap 16sp agar mudah dibaca.

12. `star_item.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, deklarasi root `LinearLayout` vertikal yang membungkus satu item rasi bintang. Layout-nya `match_parent` lebar, tinggi menyesuaikan isi. `Padding 8dp` biar ga terlalu mepet tepi.
- Pada line 5, tambahan namespace `app` agar bisa memakai properti khusus seperti `shapeAppearanceOverlay`.
- Pada line 9-14, `CardView` untuk memberi tampilan seperti kartu. Diberi `margin 8dp`, sudut melengkung (`cardCornerRadius=16dp`), dan bayangan (`cardElevation=4dp`) biar ada efek ngambang.
- Pada line 16-20, `LinearLayout` horizontal di dalam `CardView` untuk menampilkan dua sisi, yaitu kiri (gambar) & kanan (info). `Padding 8dp` untuk ruang dalam.
- Pada line 22-28, `ShapeableImageView (@id/img)` untuk menampilkan gambar rasi. Ukuran 100x150dp, crop tengah, dan dibuat rounded lewat `shapeAppearanceOverlay`.
- Pada line 30-34, `LinearLayout` vertikal dengan `layout_weight="1"` supaya mengisi sisa ruang di sebelah kanan gambar.
- Pada line 36-40, `LinearLayout` horizontal untuk baris pertama info: nama rasi dan tahun. `gravity="center_vertical"` agar teks sejajar tengah secara vertikal.

- Pada line 42-49, `TextView (@id/tvName)` untuk nama rasi bintang, pakai `layout_weight="1"` agar fleksibel. Ukuran teks 16sp, bold, responsif ke sisa ruang.
- Pada line 51-56, `TextView (@id/tvYear)` untuk menampilkan tahun, posisinya di sebelah kanan nama.
- Pada line 59-67, `TextView (@id/tvDescription)` untuk deskripsi singkat. Margin atas 4dp, teks 14sp, maksimal 3 baris, dan `ellipsize="end"` artinya kalau terlalu panjang, akan dipotong dan dikasih tanda titik tiga (...).
- Pada line 69-89, `LinearLayout` horizontal untuk dua tombol: Website & Detail. Margin atas 8dp biar terpisah dari deskripsi.
- Pada line 75-80, `Button (@id/btnWeb)` untuk buka URL website rasi. Pakai `layout_weight="1"` untuk membagi ruang setengah layar.
- Pada line 82-88, `Button (@id/btnDetail)` untuk masuk ke halaman detail rasi. Dikasih `layout_marginStart="8dp"` agar tidak mepet ke tombol sebelumnya.

13. nav_graph.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, terdapat elemen root `<navigation>` yang menjafi tempat mengatur alur navigasi antar fragment. Terdapat namespace android untuk atribut standar Android.
- Pada line 3, namespace app untuk properti khusus dari Android Jetpack Navigation.
- Pada line 4, `android:id="@+id/nav_graph"` adalah ID untuk grafik navigasi ini.
- Pada line 5, `app:startDestination="@id/homeFragment"` menunjukkan bahwa homeFragment adalah layar awal saat aplikasi dibuka.

- Pada line 7-14, deklarasi fragment untuk homeFragment (@+id/homeFragment).
- Pada line 9, menunjukkan bahwa kelas fragment-nya adalah `com.example.starconstellations.HomeFragment`
- Pada line 10, menunjukkan bahwa label yang muncul di toolbar adalah "Home"
- Pada line 11-13, terdapat `<action>` yang mengatur aksi `action_homeFragment_to_detailFragment` untuk navigasi dari Home ke Detail. `app:destination="@id/detailFragment"` menunjukkan tujuan navigasinya
- Pada line 14–18, deklarasi fragment untuk detailFragment (@+id/detailFragment).
- Pada line 18, menunjukkan bahwa kelas fragment-nya: `com.example.starconstellations.DetailFragment`
- Pada line 19, menunjukkan bahwa nama label adalah "Detail"
- Pada line 20-22, terdapat argument bernama "constellation". Hal ini digunakan untuk mengirim data objek bertipe `com.example.starconstellations.Constellation` dari Home ke Detail.

14. styles.xml:

- Pada line 1, deklarasi encoding XML UTF-8.
- Pada line 2, terdapat `<resources>` yang merupakan root element untuk menyimpan semua resource seperti style, color, dll.
- Pada line 3, deklarasi style baru bernama `RoundedImageView`, tidak mewarisi style lain (`parent=""`).
- Pada line 4, terdapat `cornerFamily` yang di-set ke `rounded` artinya semua sudut akan dibulatkan.

- Pada line 5, `cornerSize` di-set ke 16dp artinya radius lengkung sudut sebesar 16dp.
- Pada line 7, mengoverride fungsi `create` dari interface `ViewModelProvider.Factory`, yang akan dipanggil saat sistem membutuhkan instance `ViewModel`.
- Pada line 8, memeriksa apakah `modelClass` yang diminta adalah `MainViewModel` atau subclass-nya menggunakan `isAssignableFrom`.
- Pada line 9, jika benar, maka membuat instance `MainViewModel` dan mengembalikannya sebagai objek bertipe `T`.
- Pada line 11, jika `modelClass` bukan `MainViewModel`, maka melempar exception `IllegalArgumentException` dengan pesan “Unknown ViewModel class”.

15. Penjelasan Fungsi Debugger

- **Debugger** digunakan untuk memeriksa alur program dan nilai variabel secara detail.
- Penggunaan debugger memungkinkan saya untuk melihat alur eksekusi kode, memeriksa isi variabel, dan mengetahui letak kesalahan secara spesifik.
- Breakpoint adalah titik henti sementara dalam kode program. Saat eksekusi mencapai titik ini, program akan di-pause, memungkinkan kita untuk memeriksa isi dan alur.
- Fitur debugger sangat penting saat pengembangan aplikasi agar bisa memahami dan mengevaluasi bagaimana logika aplikasi bekerja secara real-time.
- ☐ **Step Into** mengeksplorasi isi fungsi, **Step Over** menjalankan tanpa masuk fungsi, dan **Step Out** keluar dari fungsi ke level di atasnya

16. Cara Menggunakan Debugger

- Saya menambahkan breakpoint pada baris `viewModel.onDetailClicked(constellation)` di file `HomeFragment.kt`, karena baris ini menangani event saat user klik tombol "Detail".
- Saya menjalankan aplikasi dengan meng-klik icon debug. Lalu, saya meng-klik tombol "Detail" di list item dan aplikasi berhenti di breakpoint yang sudah saya pasang.
- Setelah aplikasi terbuka dan saya klik tombol "Detail", debugger berhenti di breakpoint, menunjukkan bahwa event telah berhasil dipicu.
- Saat program berhenti di breakpoint, saya:
 - Menggunakan Step Into (F7) untuk masuk ke dalam fungsi `onDetailClicked()` dan melihat bagaimana ViewModel memproses data.
 - Menggunakan Step Over (F8) untuk menjalankan baris kode saat ini tanpa masuk ke dalam fungsi.
 - Menggunakan Step Out (Shift + F8) untuk keluar dari fungsi dan kembali ke pemanggil sebelumnya (Fragment).
- Saya juga menggunakan panel Variables di Debugger untuk melihat isi objek `constellation` seperti `name`, `description`, dan `webUrl`.

17. Penjelasan Fitur Step Into, Step Over, dan Step Out

- **Step Into (F7):** Digunakan untuk masuk ke dalam metode atau fungsi yang sedang dipanggil. Dalam kasus ini, digunakan untuk masuk ke fungsi `onDetailClicked()` di `MainViewModel`.
- **Step Over (F8):** Menjalankan baris kode saat ini dan langsung melanjutkan ke baris berikutnya tanpa masuk ke dalam fungsi yang dipanggil.

- **Step Out (Shift + F8):** Keluar dari fungsi saat ini dan kembali ke pemanggilnya. Berguna saat kita sudah selesai memeriksa bagian dalam dari fungsi dan ingin melanjutkan ke level di atasnya.
- Saya juga melihat nilai dari variabel constellation melalui panel debugger, yang menunjukkan data item yang sedang diproses.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Application adalah kelas dasar dari Android yang mewakili seluruh aplikasi, berfungsi sebagai titik masuk utama dan tempat untuk menginisialisasi dan mengelola status aplikasi secara global. Kelas ini dijalankan satu kali saat aplikasi pertama kali diluncurkan dan tetap aktif selama proses aplikasi berjalan. Secara teknis, Application adalah subclass dari `android.app.Application`. Kelas ini dibuat sebelum komponen lain seperti Activity dan Service, sehingga menyediakan tempat yang tepat untuk menginisialisasi hal-hal yang dibutuhkan oleh seluruh aplikasi, seperti menginisialisasi database atau membuat objek global. Beberapa fungsi utama dari application class adalah sebagai tempat untuk menginisialisasi hal-hal global seperti ViewModelFactory, Library, atau Dependency Injection (Hilt, Dagger), menyimpan state atau variabel global yang ingin diakses dari berbagai komponen aplikasi, membuat sub class dari Application Class dan menentukan nama sub class ini dalam `AndroidManifest.xml` sebagai `android:name` atribut dalam tag `<application>`, serta dapat menyediakan `onCreate()` yang dipanggil pertama kali saat aplikasi dibuka, bahkan sebelum MainActivity. Application class ini cocok untuk setup logger global, crash reporting (misalnya Firebase Crashlytics), atau exception handler.

MODUL 5 : Connect To the Internet

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- f. Gunakan caching strategy pada Room
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. data/local/ConstellationDao.kt

Tabel 33. Source Code Jawaban Soal 1 Modul 5 (ConstellationDao.kt)

1	<code>package com.example.modul5.data.local</code>
2	
3	<code>import androidx.room.*</code>
4	<code>import kotlinx.coroutines.flow.Flow</code>
5	
6	<code>@Dao</code>
7	<code>interface ConstellationDao {</code>

8	@Query("SELECT * FROM constellations")
9	fun getAll(): Flow<List<ConstellationEntity>>
10	
11	@Insert(onConflict = OnConflictStrategy.REPLACE)
12	suspend fun insertAll(data: List<ConstellationEntity>)
13	}

2. data/local/Constellationdatabase.kt

Tabel 34. Source Code Jawaban Soal 1 Modul 5 (ConstellationDatabase.kt)

1	package com.example.modul5.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	
8	@Database(entities = [ConstellationEntity::class], version
9	= 1, exportSchema = false)
10	abstract class ConstellationDatabase : RoomDatabase() {
11	abstract fun constellationDao(): ConstellationDao
12	
13	companion object {
14	@Volatile
15	private var INSTANCE: ConstellationDatabase? =
16	null
17	
18	fun getInstance(context: Context):
19	ConstellationDatabase {
20	return INSTANCE ?: synchronized(this) {
21	Room.databaseBuilder(
22	context.applicationContext,
23	ConstellationDatabase::class.java,
24	"constellation_db"

25	<code>)</code> .build()).also { INSTANCE = it }
26	<code>}</code>
	<code>}</code>
	<code>}</code>
	<code>}</code>

3. data/local/ConstellationEntity.kt

Tabel 35. Source Code Jawaban Soal 1 Modul 5 (ConstellationEntity.kt)

1	<code>package com.example.modul5.data.local</code>
2	
3	<code>import android.os.Parcelable</code>
4	<code>import androidx.room.Entity</code>
5	<code>import androidx.room.PrimaryKey</code>
6	<code>import kotlinx.parcelize.Parcelize</code>
7	
8	<code>@Parcelize</code>
9	<code>@Entity(tableName = "constellations")</code>
10	<code>data class ConstellationEntity(</code>
11	<code> @PrimaryKey val name: String,</code>
12	<code> val description: String,</code>
13	<code> val year: String,</code>
14	<code> val imageUrl: String,</code>
15	<code> val webUrl: String</code>
16	<code>) : Parcelable</code>

4. data./model/ApiResponse.kt

Tabel 36. Source Code Jawaban Soal 1 Modul 5 (ApiResponse.kt)

1	<code>package com.example.modul5.data.model</code>
2	
3	<code>import kotlinx.serialization.Serializable</code>
4	
5	<code>@Serializable</code>
6	<code>data class ApiResponse<T>(</code>

7	val status: String,
8	val message: String,
9	val data: T
10	)
11	

5. data/model/Constellation.kt

Tabel 37. Source Code Jawaban Soal 1 Modul 5 (Constellation.kt)

1	package com.example.modul5.data.model
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class Constellation(
8	val name: String,
9	val description: String,
10	val year: String,
11	@SerialName("image_url") val imageUrl: String,
12	@SerialName("web_url") val webUrl: String
13	)

6. data/remote/ApiService.kt

Tabel 38. Source Code Jawaban Soal 1 Modul 5 (ApiService.kt)

1	package com.example.modul5.data.local
2	package com.example.modul5.data.remote
3	
4	import com.example.modul5.data.model.ApiResponse
5	import com.example.modul5.data.model.Constellation
6	import retrofit2.http.GET
7	
8	interface ApiService {
9	@GET("constellation")

10	<pre>suspend fun getConstellations(): ApiResponse<List<Constellation>> { }</pre>
----	--

7. data/remote/RetrofitInstance.kt

Tabel 39. Source Code Jawaban Soal 1 Modul 5 (RetrofitInstance.kt)

1	package com.example.modul5.data.remote
2	
3	import
4	com.jakewharton.retrofit2.converter.kotlinx.serialization
5	on.asConverterFactory
6	import kotlinx.serialization.json.Json
7	import okhttp3.MediaType.Companion.toMediaType
8	import okhttp3.OkHttpClient
9	import okhttp3.logging.HttpLoggingInterceptor
10	import retrofit2.Retrofit
11	
12	object RetrofitInstance {
13	
14	private const val BASE_URL =
15	"https://constellation-api.free.beeceptor.com/"
16	
17	private val json = Json {
18	ignoreUnknownKeys = true
19	}
20	
21	private val client = OkHttpClient.Builder()
22	.addInterceptor(HttpLoggingInterceptor().apply
23	{
24	level = HttpLoggingInterceptor.Level.BODY
25	})
26	.build()
27	

28	<code>val api: ApiService by lazy {</code>
29	<code> Retrofit.Builder()</code>
30	<code> .baseUrl(BASE_URL)</code>
31	<code></code>
32	<code>.addConverterFactory(json.asConverterFactory("applicati</code>
	<code>on/json".toMediaType()))</code>
	<code> .client(client)</code>
	<code> .build()</code>
	<code> .create(ApiService::class.java)</code>
	<code> }</code>
	<code>}</code>

8. data/repository/ConstellationRepository.kt

Tabel 40. Source Code Jawaban Soal 1 Modul 5 (ConstellationRepository.kt)

1	<code>package com.example.modul5.data.repository</code>
2	<code></code>
3	<code>import com.example.modul5.data.local.ConstellationDao</code>
4	<code>import</code>
5	<code>com.example.modul5.data.local.ConstellationEntity</code>
6	<code>import com.example.modul5.data.remote.RetrofitInstance</code>
7	<code>import kotlinx.coroutines.flow.Flow</code>
8	<code>import kotlinx.coroutines.flow.flow</code>
9	<code>import kotlinx.coroutines.flow.emitAll</code>
10	<code></code>
11	<code>class ConstellationRepository(</code>
12	<code> private val dao: ConstellationDao</code>
13	<code>) {</code>
14	<code></code>
15	<code> fun getConstellations():</code>
16	<code>Flow<List<ConstellationEntity>> = flow {</code>
17	<code> try {</code>
18	<code> val response =</code>
19	<code>RetrofitInstance.api.getConstellations()</code>

20	if (response.status == "success") {
21	val mapped = response.data.map {
22	ConstellationEntity(
23	name = it .name,
24	description = it .description,
25	year = it .year,
26	imageUrl = it .imageUrl,
27	webUrl = it .webUrl
28	)
29	}
30	dao.insertAll(mapped)
31	}
32	} catch (e: Exception) {
33	e.printStackTrace()
34	}
35	
	emitAll(dao.getAll())
	}
	}

9. ui/adapt er/ListConstellationAdapter.kt

Tabel 41. Source Code Jawaban Soal 1 Modul 5 (ListConstellationAdapter.kt)

1	package com.example.modul5.ui.adapter
2	
3	import android.util.TypedValue
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.recyclerview.widget.DiffUtil
7	import androidx.recyclerview.widget.ListAdapter
8	import androidx.recyclerview.widget.RecyclerView
9	import com.bumptech.glide.Glide
10	import
11	com.bumptech.glide.load.resource.bitmap.RoundedCorners

```

12 import
13 com.example.modul5.data.local.ConstellationEntity
14 import com.example.modul5.databinding.StarItemBinding
15
16 class ListConstellationsAdapter(
17     private val onClick: (ConstellationEntity) ->
18     Unit,
19     private val onDetailClick: (ConstellationEntity) ->
20     Unit
21 ) : ListAdapter<ConstellationEntity,
22 ListConstellationsAdapter.ViewHolder>(DIFF_CALLBACK) {
23
24     inner class ViewHolder(val binding:
25     StarItemBinding) :
26     RecyclerView.ViewHolder(binding.root)
27
28     override fun onCreateViewHolder(parent: ViewGroup,
29     viewType: Int): ViewHolder {
30         val binding =
31         StarItemBinding.inflate(LayoutInflater.from(parent.cont
32         ext), parent, false)
33         return ViewHolder(binding)
34     }
35
36     override fun onBindViewHolder(holder: ViewHolder,
37     position: Int) {
38         val item = getItem(position)
39         with(holder.binding) {
40             tvName.text = item.name
41             tvDescription.text = item.description
42             tvYear.text = item.year
43             val radius = TypedValue.applyDimension(
44                 TypedValue.COMPLEX_UNIT_DIP, 16f,

```

45	<code>holder.itemView.context.resources.displayMetrics</code>
46	<code>).toInt()</code>
47	
48	<code>Glide.with(holder.itemView.context)</code>
49	<code>.load(item.imageUrl)</code>
50	<code>.transform(RoundedCorners(radius))</code>
51	<code>.into(holder.binding.img)</code>
52	
53	
54	<code>btnWeb.setOnClickListener {</code>
55	<code>onWebClick(item) }</code>
56	<code>btnDetail.setOnClickListener {</code>
	<code>onDetailClick(item) }</code>
	<code>}</code>
	<code>}</code>
	<code>companion object {</code>
	<code>val DIFF_CALLBACK = object :</code>
	<code>DiffUtil.ItemCallback<ConstellationEntity>() {</code>
	<code>override fun areItemsTheSame(oldItem:</code>
	<code>ConstellationEntity, newItem: ConstellationEntity) =</code>
	<code>oldItem.name == newItem.name</code>
	<code>override fun areContentsTheSame(oldItem:</code>
	<code>ConstellationEntity, newItem: ConstellationEntity) =</code>
	<code>oldItem == newItem</code>
	<code>}</code>
	<code>}</code>
	<code>}</code>

10. ui/detail/DetailFragment.kt

Tabel 42. Source Code Jawaban Soal 1 Modul 5 (DetailFragment.kt)

```

1 package com.example.modul5.ui.detail
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.navigation.fragment.navArgs
9 import com.bumptech.glide.Glide
10 import
11 com.example.modul5.databinding.FragmentDetailBinding
12
13 class DetailFragment : Fragment() {
14
15     private var _binding: FragmentDetailBinding? = null
16     private val binding get() = _binding!!
17     private val args: DetailFragmentArgs by navArgs()
18
19     override fun onCreateView(inflater: LayoutInflater,
20 container: ViewGroup?, savedInstanceState: Bundle?):
21 View {
22         _binding =
23 FragmentDetailBinding.inflate(inflater, container,
24 false)
25         return binding.root
26     }
27
28     override fun onViewCreated(view: View,
29 savedInstanceState: Bundle?) {
30         val constellation = args.constellation
31         binding.detailName.text = constellation.name
32         binding.detailYear.text = constellation.year
33         binding.detailDescription.text =

```

34	constellation.description
35	Glide.with(this).load(constellation.imageUrl).into(binding.detailImage)
	}
	 override fun onDestroyView() {
	super.onDestroyView()
	_binding = null
	}
	}

11. ui/home/HomeFragment.kt

Tabel 43. Source Code Jawaban Soal 1 Modul 5 (HomeFragment.kt)

1	package com.example.modul5.ui.home
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.lifecycleScope
12	import androidx.navigation.fragment.findNavController
13	import androidx.recyclerview.widget.LinearLayoutManager
14	import
15	com.example.modul5.databinding.FragmentHomeBinding
16	import
17	com.example.modul5.ui.adapter.ListConstellationsAdapter
18	import com.example.modul5.viewmodel.MainViewModel
19	import


```

20 com.example.modul5.viewmodel.MainViewModelFactory
21 import kotlinx.coroutines.flow.collectLatest
22 import kotlinx.coroutines.launch
23
24 class HomeFragment : Fragment() {
25
26     private var _binding: FragmentHomeBinding? = null
27     private val binding get() = _binding!!
28     private val viewModel: MainViewModel by viewModels
29 {
30         MainViewModelFactory(requireContext())
31     }
32
33     override fun onCreateView(inflater: LayoutInflater,
34 container: ViewGroup?, savedInstanceState: Bundle?):
35 View {
36         _binding =
37 FragmentHomeBinding.inflate(inflater, container, false)
38         return binding.root
39     }
40
41     override fun onViewCreated(view: View,
42 savedInstanceState: Bundle?) {
43         val adapter = ListConstellationsAdapter(
44             onClick = {
45                 val intent = Intent(Intent.ACTION_VIEW,
46 Uri.parse(it.webUrl))
47                 startActivity(intent)
48             },
49             onDetailClick = {
50                 val action =
51 HomeFragmentDirections.actionHomeFragmentToDetailFragme
52 nt(it)

```

53	<code>findNavController().navigate(action)</code>
54	<code>}</code>
55	<code>)</code>
56	
57	<code>binding.rvStars.layoutManager =</code>
58	<code>LinearLayoutManager(requireContext())</code>
59	<code>binding.rvStars.adapter = adapter</code>
60	
	<code>lifecycleScope.launch {</code>
	<code>viewModel.state.collectLatest { data -></code>
	<code>adapter.submitList(data)</code>
	<code>}</code>
	<code>}</code>
	<code>override fun onDestroyView() {</code>
	<code>super.onDestroyView()</code>
	<code>_binding = null</code>
	<code>}</code>
	<code>}</code>

12. ui/MainActivity.kt

Tabel 44. Source Code Jawaban Soal 1 Modul 5 (MainActivity.kt)

1	<code>package com.example.modul5.ui</code>
2	
3	<code>import android.content.res.Configuration</code>
4	<code>import android.os.Bundle</code>
5	<code>import android.text.SpannableString</code>
6	<code>import android.text.style.ForegroundColorSpan</code>
7	<code>import android.view.Menu</code>
8	<code>import android.view.MenuItem</code>
9	<code>import androidx.appcompat.app.AppCompatActivity</code>
10	<code>import androidx.appcompat.app.AppCompatActivity</code>

```

11 import androidx.core.content.ContextCompat
12 import androidx.navigation.fragment.NavHostFragment
13 import
14 androidx.navigation.ui.setupActionBarWithNavController
15 import com.example.modul5.R
16 import
17 com.example.modul5.databinding.ActivityMainBinding
18
19 class MainActivity : AppCompatActivity() {
20
21     private lateinit var binding: ActivityMainBinding
22
23     override fun onCreate(savedInstanceState: Bundle?)
24 {
25         super.onCreate(savedInstanceState)
26
27         binding =
28 ActivityMainBinding.inflate(layoutInflater)
29         setContentView(binding.root)
30         setSupportActionBar(binding.toolbar)
31
32         val navHostFragment = supportFragmentManager
33             .findFragmentById(R.id.nav_host_fragment)
34 as NavHostFragment
35
36 setupActionBarWithNavController(navHostFragment.navCont
37 roller)
38     }
39
40     override fun onCreateOptionsMenu(menu: Menu):
41 Boolean {
42         menuInflater.inflate(R.menu.menu_theme, menu)
43

```

```

44         val menuItem =
45 menu.findItem(R.id.action_toggle_theme)
46         val span = SpannableString(menuItem.title)
47
48 span.setSpan(ForegroundColorSpan(ContextCompat.getColor
49 (this, R.color.purple_500)), 0, span.length, 0)
50         menuItem.title = span
51
52         return true
53     }
54
55
56     override fun onOptionsItemSelected(item: MenuItem):
57 Boolean {
58         return when (item.itemId) {
59             R.id.action_toggle_theme -> {
60                 val nightMode =
61 resources.configuration.uiMode and
62 Configuration.UI_MODE_NIGHT_MASK
63                 val newMode = if (nightMode ==
64 Configuration.UI_MODE_NIGHT_YES)
65                     AppCompatActivity.MODE_NIGHT_NO
66                 else
67                     AppCompatActivity.MODE_NIGHT_YES
68
69 AppCompatActivity.setDefaultNightMode(newMode)
70                 true
71             }
72             else -> super.onOptionsItemSelected(item)
73         }
74     }
75
76     override fun onSupportNavigateUp(): Boolean {

```

	<pre> val navHostFragment = supportFragmentManager .findFragmentById(R.id.nav_host_fragment) as NavHostFragment return navHostFragment.navController.navigateUp() super.onSupportNavigateUp() } }</pre>
--	--

13. utils/Resource.kt

Tabel 45. Source Code Jawaban Soal 1 Modul 5 (Resource.kt)

1	package com.example.modul5.utils
2	
3	sealed class Resource<out T> {
4	data class Success<out T>(val data: T):
5	Resource<T>()
6	data class Error(val message: String):
7	Resource<Nothing>()
	object Loading : Resource<Nothing>()
	}

14. viewmodel/MainViewModel.kt

Tabel 46. Source Code Jawaban Soal 1 Modul 5 (MainViewModel.kt)

1	package com.example.modul5.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import
6	com.example.modul5.data.local.ConstellationEntity
7	import
8	com.example.modul5.data.repository.ConstellationReposit
9	ory
10	import kotlinx.coroutines.flow.*

11	import kotlinx.coroutines.launch
12	
13	class MainViewModel(
14	private val repository: ConstellationRepository
15	) : ViewModel() {
16	
17	private val _state =
18	MutableStateFlow<List<ConstellationEntity>>(emptyList())
19	)
20	val state: StateFlow<List<ConstellationEntity>> =
21	_state.asStateFlow()
22	
23	init {
24	fetchConstellations()
25	}
26	
27	private fun fetchConstellations() {
28	viewModelScope.launch {
	repository.getConstellations()
	.catch { it.printStackTrace() }
	.collect { _state.value = it }
	}
	}
	}

15. data/viewmodel/MainViewModelFactory.kt

Tabel 47. Source Code Jawaban Soal 1 Modul 5 (MainViewModelFactory.kt)

1	package com.example.modul5.viewmodel
2	
3	import android.content.Context
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelProvider
6	import

7	<code>com.example.modul5.data.local.ConstellationDatabase</code>
8	<code>import</code>
9	<code>com.example.modul5.data.repository.ConstellationRepository</code>
10	<code>ory</code>
11	
12	<code>class MainViewModelFactory(private val context:</code>
13	<code>Context) : ViewModelProvider.Factory {</code>
14	<code> override fun <T : ViewModel> create(modelClass:</code>
15	<code>Class<T>): T {</code>
16	<code> if</code>
17	<code>(modelClass.isAssignableFrom(MainViewModel::class.java)</code>
18	<code>) {</code>
	<code> val dao =</code>
	<code>ConstellationDatabase.getInstance(context).constellationDao()</code>
	<code> val repo = ConstellationRepository(dao)</code>
	<code> return MainViewModel(repo) as T</code>
	<code> }</code>
	<code> throw IllegalArgumentException("Unknown</code>
	<code>ViewModel class")</code>
	<code> }</code>
	<code>}</code>

16. ui/AndroidAPIApp.kt

Tabel 48. Source Code Jawaban Soal 1 Modul 5 (AndroidAPIApp.kt)

1	<code>package com.example.modul5</code>
2	
3	<code>import android.app.Application</code>
4	
5	<code>class AndroidAPIApp : Application()</code>

17. res/layout/activity_main.xml

Tabel 49. Source Code Jawaban Soal 1 Modul 5 (activity_main.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro
5	id"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent">
9	
10	<com.google.android.material.appbar.AppBarLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	
14	android:theme="@style/ThemeOverlay.MaterialComponents.D
15	ark.ActionBar">
16	
17	<androidx.appcompat.widget.Toolbar
18	android:id="@+id/toolbar"
19	android:layout_width="match_parent"
20	android:layout_height="?attr/actionBarSize"
21	android:background="?attr/colorPrimary"
22	android:elevation="4dp"
23	app:titleTextColor="@android:color/white"
24	/>
25	</com.google.android.material.appbar.AppBarLayout>
26	
27	<androidx.fragment.app.FragmentContainerView
28	android:id="@+id/nav_host_fragment"
29	
30	android:name="androidx.navigation.fragment.NavHostFragm ent"

	<pre> android:layout_width="match_parent" android:layout_height="match_parent" app:navGraph="@navigation/nav_graph" app:defaultNavHost="true" android:layout_marginTop="?attr/actionBarSize"/> </androidx.coordinatorlayout.widget.CoordinatorLayout> </pre>
--	--

18. res/layout/fragment_detail.xml

Tabel 50. Source Code Jawaban Soal 1 Modul 5 (fragment_detail.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro
5	id"
6	android:id="@+id/detailFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	android:padding="16dp">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="vertical"
15	android:gravity="center_horizontal">
16	
17	<ImageView
18	android:id="@+id/detailImage"
19	android:layout_width="300dp"
20	android:layout_height="300dp"
21	android:scaleType="centerCrop"
22	android:layout_marginBottom="16dp" />
23	

24	<TextView
25	android:id="@+id/detailName"
26	android:layout_width="wrap_content"
27	android:layout_height="wrap_content"
28	android:text="Name"
29	android:textSize="22sp"
30	android:textStyle="bold"
31	android:layout_marginBottom="8dp" />
32	
33	<TextView
34	android:id="@+id/detailYear"
35	android:layout_width="wrap_content"
36	android:layout_height="wrap_content"
37	android:text="Year"
38	android:textSize="16sp"
39	android:layout_marginBottom="16dp" />
40	
41	<TextView
42	android:id="@+id/detailDescription"
43	android:layout_width="match_parent"
44	android:layout_height="wrap_content"
45	android:text="Description"
46	android:textSize="16sp" />
	</LinearLayout>
	</ScrollView>

19. res/layout/fragment_home.xml

Tabel 51. Source Code Jawaban Soal 1 Modul 5 (fragment_home.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro
5	id"

6	<code>android:id="@+id/homeFragment"</code>
7	<code>android:layout_width="match_parent"</code>
8	<code>android:layout_height="match_parent"</code>
9	<code>android:padding="8dp"></code>
10	
11	<code><androidx.recyclerview.widget.RecyclerView</code>
12	<code>android:id="@+id/rvStars"</code>
13	<code>android:layout_width="match_parent"</code>
14	<code>android:layout_height="match_parent"</code>
15	<code>android:clipToPadding="false" /></code>
	<code></FrameLayout></code>

20. res/layout/star_item.xml

Tabel 52. Source Code Jawaban Soal 1 Modul 5 (star_item.xml)

1	<code>package com.example.modul5.data.local</code>
2	<code><?xml version="1.0" encoding="utf-8"?></code>
3	<code><LinearLayout</code>
4	
5	<code>xmlns:android="http://schemas.android.com/apk/res/andr</code>
6	<code>oid"</code>
7	<code>xmlns:app="http://schemas.android.com/apk/res-</code>
8	<code>auto"</code>
9	<code>android:layout_width="match_parent"</code>
10	<code>android:layout_height="wrap_content"</code>
11	<code>android:orientation="vertical"</code>
12	<code>android:padding="8dp"></code>
13	
14	<code><androidx.cardview.widget.CardView</code>
15	<code>android:layout_width="match_parent"</code>
16	<code>android:layout_height="wrap_content"</code>
17	<code>android:layout_margin="8dp"</code>
18	<code>app:cardCornerRadius="12dp"</code>

19	app:cardElevation="4dp">
20	
21	<LinearLayout
22	android:layout_width="match_parent"
23	android:layout_height="wrap_content"
24	android:orientation="horizontal"
25	android:padding="8dp">
26	
27	<ImageView
28	android:id="@+id/img"
29	android:layout_width="100dp"
30	android:layout_height="150dp"
31	android:scaleType="centerCrop"
32	android:layout_marginEnd="8dp" />
33	
34	<LinearLayout
35	android:layout_width="0dp"
36	android:layout_height="match_parent"
37	android:layout_weight="1"
38	android:orientation="vertical">
39	
40	<LinearLayout
41	
42	android:layout_width="match_parent"
43	
44	android:layout_height="wrap_content"
45	android:orientation="horizontal"
46	android:gravity="center_vertical">
47	
48	<TextView
49	android:id="@+id/tvName"
50	android:layout_width="0dp"
51	

52	android:layout_height="wrap_content"
53	android:layout_weight="1"
54	android:text="Constellations
55	Name"
56	android:textStyle="bold"
57	android:textSize="16sp" />
58	
59	<TextView
60	android:id="@+id/tvYear"
61	
62	android:layout_width="wrap_content"
63	
64	android:layout_height="wrap_content"
65	android:text="Year"
66	android:textSize="14sp" />
67	</LinearLayout>
68	
69	<TextView
70	android:id="@+id/tvDescription"
71	
72	android:layout_width="match_parent"
73	
74	android:layout_height="wrap_content"
75	android:text="Description"
76	android:maxLines="3"
77	android:ellipsize="end"
78	android:textSize="14sp"
79	android:layout_marginTop="4dp" />
80	
81	<LinearLayout
82	
83	android:layout_width="match_parent"
84	

85	android:layout_height="wrap_content"
86	android:orientation="horizontal"
87	android:layout_marginTop="8dp">
88	
89	
90	<com.google.android.material.button.MaterialButton
91	android:id="@+id/btnWeb"
92	
93	style="@style/Widget.MaterialComponents.Button"
94	android:layout_width="0dp"
95	
96	android:layout_height="wrap_content"
97	android:layout_weight="1"
98	android:text="Website"
99	android:textAllCaps="false"
101	app:cornerRadius="50dp"
102	app:iconPadding="0dp" />
	<com.google.android.material.button.MaterialButton
	android:id="@+id/btnDetail"
	style="@style/Widget.MaterialComponents.Button"
	android:layout_width="0dp"
	android:layout_height="wrap_content"
	android:layout_weight="1"
	android:text="Detail"
	android:textAllCaps="false"
	app:cornerRadius="50dp"
	android:layout_marginStart="8dp"

	<pre> app:iconPadding="0dp" /> </LinearLayout> </LinearLayout> </LinearLayout> </androidx.cardview.widget.CardView> </LinearLayout> </pre>
--	---

21. res/menu/menu_theme.xml

Tabel 53. Source Code Jawaban Soal 1 Modul 5 (menu_theme.xml)

1	<pre><?xml version="1.0" encoding="utf-8"?></pre>
2	<pre><menu</pre>
3	<pre>xmlns:android="http://schemas.android.com/apk/res/andro</pre>
4	<pre>id"</pre>
5	<pre> xmlns:app="http://schemas.android.com/apk/res-</pre>
6	<pre>auto"></pre>
7	<pre> <item</pre>
8	<pre> android:id="@+id/action_toggle_theme"</pre>
	<pre> android:title="Light/Dark Mode"</pre>
	<pre> app:showAsAction="never" /></pre>
	<pre></menu></pre>

22. res/navigation/navigation_graph.xml

Tabel 54. Source Code Jawaban Soal 1 Modul 5 (navigation_graph.xml)

1	<pre><?xml version="1.0" encoding="utf-8"?></pre>
2	<pre><navigation</pre>
3	
4	<pre>xmlns:android="http://schemas.android.com/apk/res/andro</pre>
5	<pre>id"</pre>
6	<pre> xmlns:app="http://schemas.android.com/apk/res-auto"</pre>
7	<pre> android:id="@+id/nav_graph"</pre>
8	<pre> app:startDestination="@id/homeFragment"></pre>
9	
10	<pre><fragment</pre>

11	android:id="@+id/homeFragment"
12	
13	android:name="com.example.modul5.ui.home.HomeFragment"
14	android:label="Home" >
15	<action
16	
17	android:id="@+id/action_homeFragment_to_detailFragment"
18	app:destination="@id/detailFragment" />
19	</fragment>
20	
21	<fragment
22	android:id="@+id/detailFragment"
23	
24	android:name="com.example.modul5.ui.detail.DetailFragme
25	nt"
	android:label="Detail">
	<argument
	android:name="constellation"
	app:argType="com.example.modul5.data.local.Constellatio
	nEntity" />
	</fragment>
	</navigation>

23. res/values/themes/themes.xml

Tabel 55. Source Code Jawaban Soal 1 Modul 5 (themes.xml)

1	<resources
2	xmlns:tools="http://schemas.android.com/tools">
3	<style name="Theme.MODUL5"
4	parent="Theme.MaterialComponents.DayNight.NoActionBar">
5	<item
6	name="colorPrimary">@color/purple_500</item>
7	<item

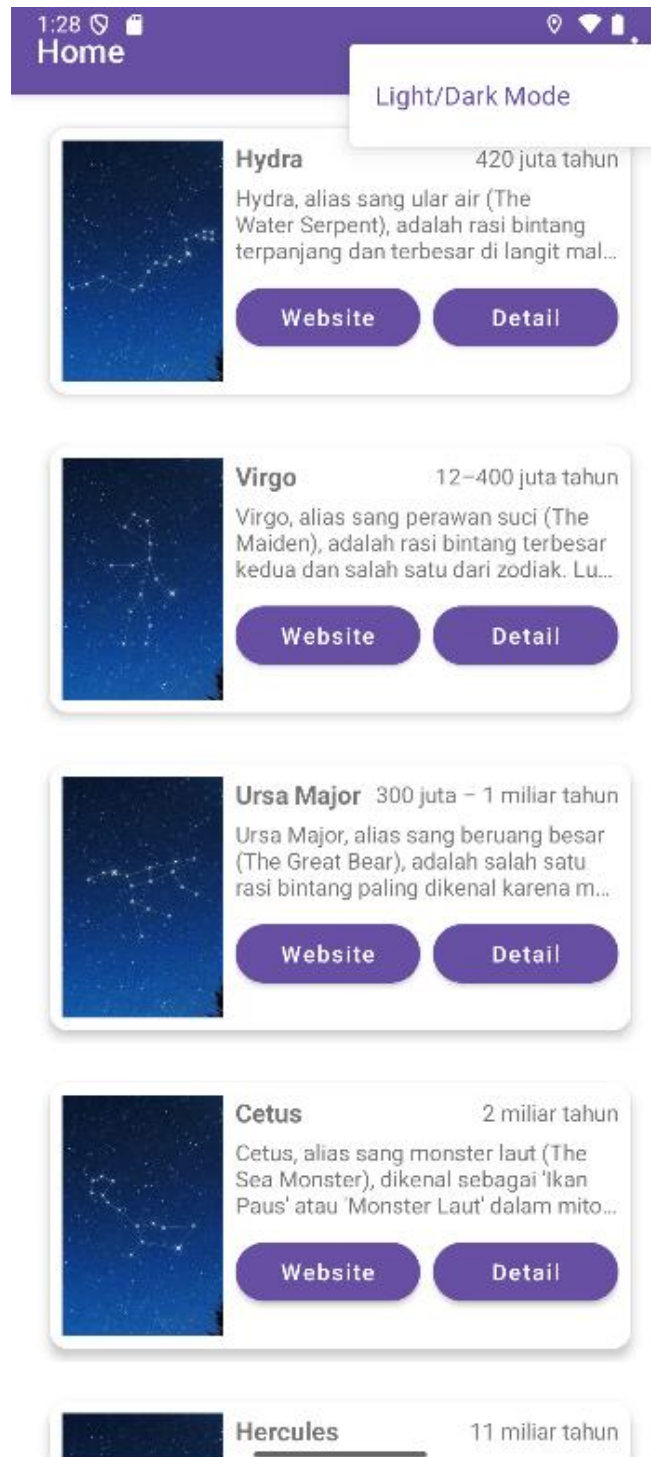
8	name="colorPrimaryVariant">@color/purple_700</item>
9	<item
10	name="colorOnPrimary">@android:color/white</item>
	<item
	name="colorSecondary">@color/teal_200</item>
	<item
	name="colorSecondaryVariant">@color/teal_700</item>
	<item
	name="colorOnSecondary">@android:color/white</item>
	</style>
	</resources>

24. res/values/themes/themes.xml (night)

Tabel 56. Source Code Jawaban Soal 1 Modul 5 (themes.xml (night))

1	<resources
2	xmlns:tools="http://schemas.android.com/tools">
3	<style name="Theme.MODUL5"
4	parent="Theme.MaterialComponents.DayNight.NoActionBar">
5	<item
6	name="colorPrimary">@color/purple_200</item>
7	<item
8	name="colorPrimaryVariant">@color/purple_700</item>
9	<item
10	name="colorOnPrimary">@android:color/white</item>
	<item
	name="colorSecondary">@color/teal_200</item>
	<item
	name="colorSecondaryVariant">@color/teal_700</item>
	<item
	name="colorOnSecondary">@android:color/white</item>
	</style>
	</resources>

B. Output Program

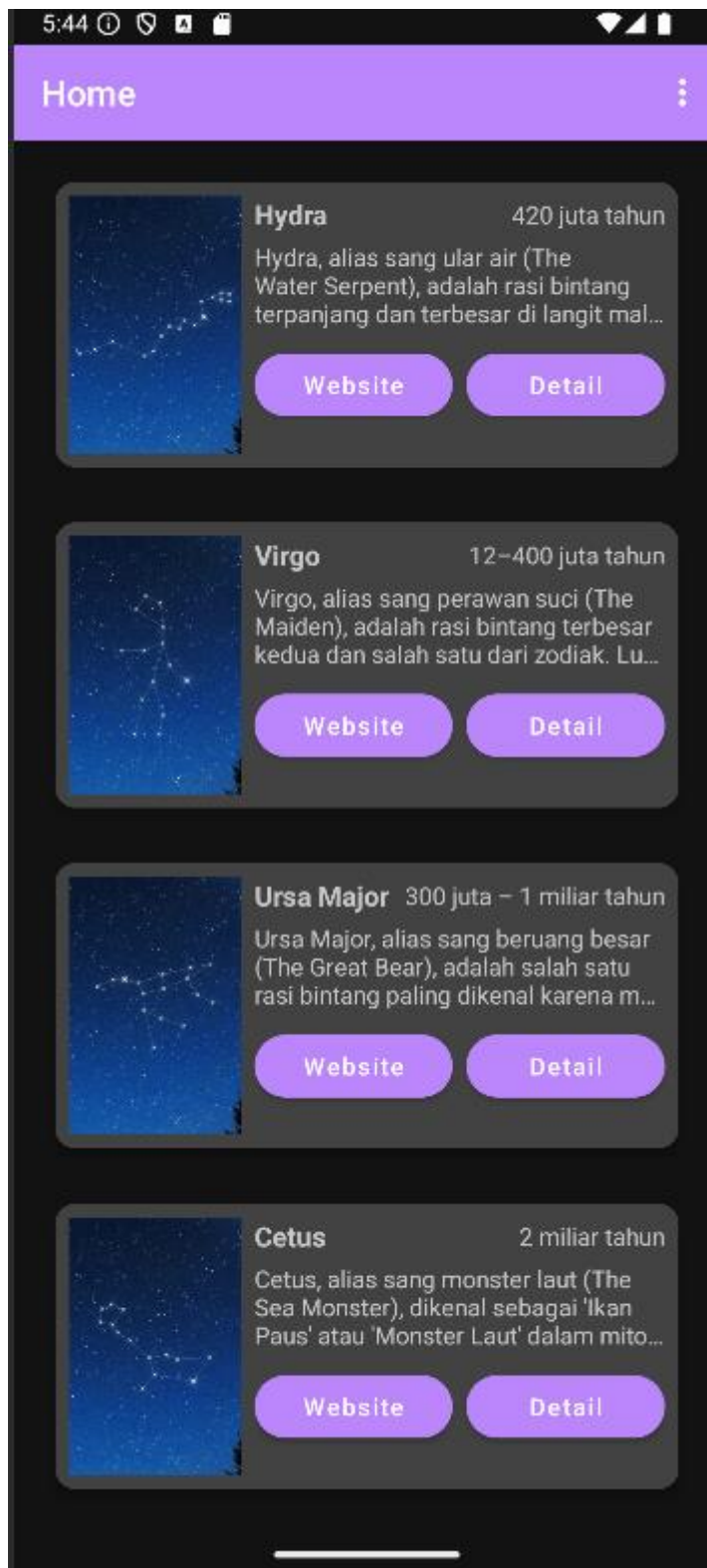


Gambar 28. Screenshot Hasil Jawaban Soal 1 Modul 5 (Home Light Mode)



Hydra, alias sang ular air (The Water Serpent), adalah rasi bintang terpanjang dan terbesar di langit malam. Luasnya sebesar 1.303 derajat persegi (~3,16% dari seluruh langit malam). Bentuknya menyerupai ular air dan membentang dari selatan Cancer hingga Libra. Meski ukurannya besar, hanya sedikit bintang terang yang terlihat. Hydra dikenal dalam mitologi Yunani sebagai ular berkepala banyak yang dikalahkan oleh Herkules.

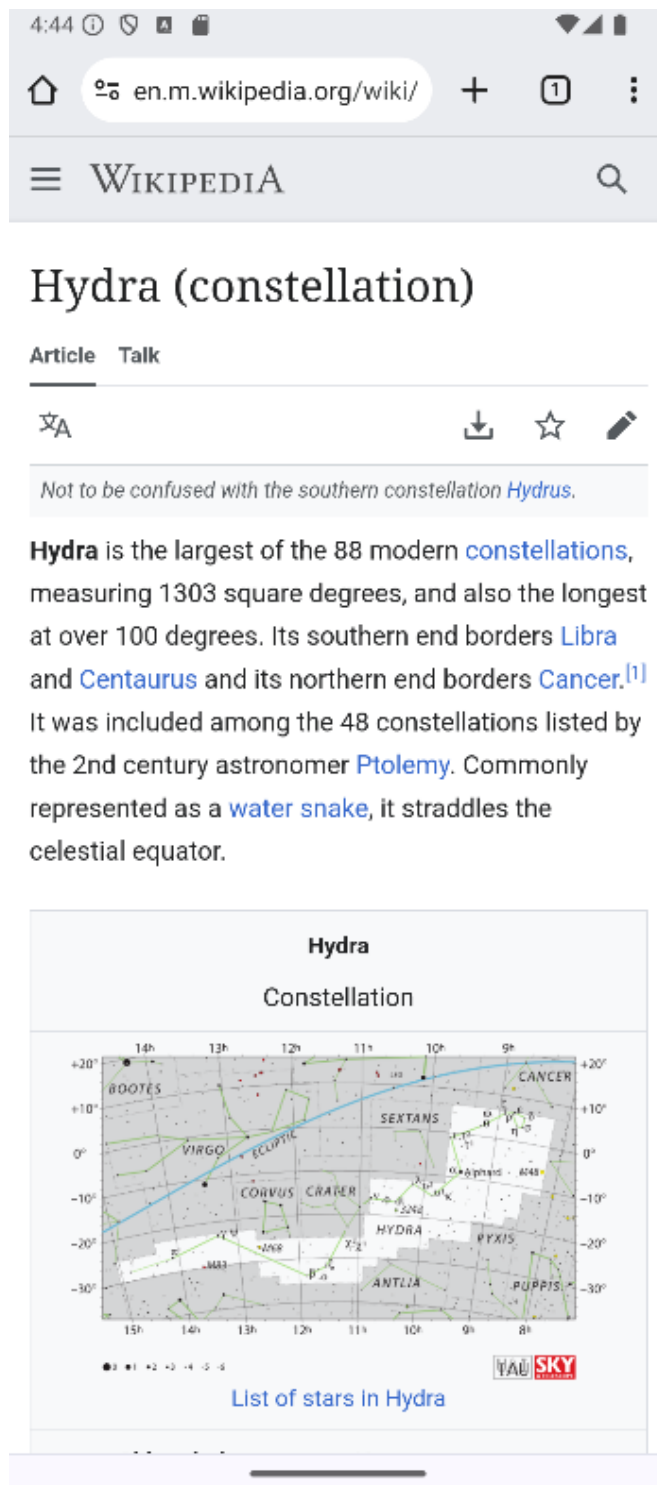
Gambar 29. Screenshot Hasil Jawaban Soal 1 Modul 5 (Detail Light Mode)



Gambar 30. Screenshot Hasil Jawaban Soal 1 Modul 5 (Home Dark Mode)



Gambar 31. Screenshot Hasil Jawaban Soal 1 Modul 5 (Detail Dark Mode)



Gambar 32. Screenshot Hasil Jawaban Soal 1 Modul 5 (Website)

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. data/local/ConstellationDao.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.local`
- Pada line 3, mengimpor anotasi dan class dari Room seperti `Dao`, `Insert`, `OnConflictStrategy`, dan `Query`.
- Pada line 4, mengimpor `Flow` dari `kotlinx.coroutines.flow` untuk mendukung data yang mengalir secara asinkron.
- Pada line 6, mendeklarasikan interface `ConstellationDao` dan menandainya sebagai DAO dengan anotasi `@Dao`.
- Pada line 9, mendeklarasikan fungsi `getAll()` dengan anotasi `@Query`, untuk mengambil seluruh data dari tabel `constellations`, dan mengembalikannya dalam bentuk `Flow<List<ConstellationEntity>>`.
- Pada line 11, mendeklarasikan fungsi `insertAll()` dengan anotasi `@Insert`, untuk menyimpan data list `ConstellationEntity`, dan menggunakan strategi `REPLACE` jika ada konflik primary key.

2. data/local/ConstellationDatabase.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.local`.
- Pada line 3, mengimpor `Context` dari Android untuk digunakan saat inisialisasi database.
- Pada line 4, mengimpor anotasi `Database` dari Room untuk mendefinisikan konfigurasi database.
- Pada line 5, mengimpor class Room untuk membangun instance database.
- Pada line 6, mengimpor class `RoomDatabase` sebagai superclass untuk database Room.

- Pada line 8, mendeklarasikan anotasi `@Database` dengan entitas `ConstellationEntity`, versi 1, dan tanpa mengeksport skema.
- Pada line 9, mendeklarasikan class abstract `ConstellationDatabase` yang mewarisi `RoomDatabase`.
- Pada line 10, mendeklarasikan fungsi abstrak `constellationDao()` untuk mengakses DAO dari database.
- Pada line 12, mendeklarasikan blok companion object untuk menyimpan instance singleton database.
- Pada line 13, mendeklarasikan variabel `INSTANCE` bertipe nullable `ConstellationDatabase`, dan ditandai dengan `@Volatile` agar aman untuk akses dari banyak thread.
- Pada line 15, mendeklarasikan fungsi `getInstance()` untuk mengembalikan instance database, menerima parameter `context`.
- Pada line 16, menggunakan operator Elvis (`?:`) untuk mengecek apakah `INSTANCE` sudah tersedia, jika belum maka lanjut membuat instance baru.
- Pada line 17-21, membuat instance database menggunakan `Room.databaseBuilder`, dengan konteks aplikasi, referensi class database, dan nama database `"constellation_db"`.
- Pada line 22, menyimpan instance yang telah dibuat ke dalam variabel `INSTANCE` dan mengembalikannya.

3. `data/local/ConstellationEntity.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.local`.
- Pada line 3, mengimpor interface `Parcelable` dari Android untuk mendukung passing objek antar komponen.
- Pada line 4, mengimpor anotasi `Entity` dari Room untuk mendefinisikan class sebagai entitas tabel database.

- Pada line 5, mengimpor anotasi `PrimaryKey` dari `Room` untuk menandai atribut primary key.
- Pada line 6, mengimpor anotasi `Parcelize` dari Kotlin untuk mengotomatisasi implementasi `Parcelable`.
- Pada line 8, menandai class `ConstellationEntity` dengan anotasi `@Parcelize` agar dapat di-parcel secara otomatis.
- Pada line 9, menandai class `ConstellationEntity` sebagai entitas `Room` dengan nama tabel `constellations`.
- Pada line 11, mendeklarasikan properti `name` sebagai primary key dan bertipe `String`.
- Pada line 12-15, mendeklarasikan properti tambahan: `description`, `year`, `imageUrl`, dan `webUrl`, semuanya bertipe `String`.
- Pada line 16, menandai bahwa class `ConstellationEntity` mengimplementasikan `Parcelable`.

4. `data/model/ApiResponse.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.model`.
- Pada line 3, mengimpor anotasi `Serializable` dari Kotlinx `Serialization` untuk mendukung serialisasi data.
- Pada line 5, menandai data class `ApiResponse` sebagai serializable menggunakan anotasi `@Serializable`.
- Pada line 6-10, mendeklarasikan properti `status`, `message`, dan `data` bertipe generic `T`, yang merepresentasikan format response dari API.

5. `data/model/Constellation.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.model`.

- Pada line 3, mengimpor anotasi `SerializedName` dari Kotlinx Serialization untuk mapping nama field JSON.
- Pada line 4, mengimpor anotasi `Serializable` dari Kotlinx Serialization.
- Pada line 6, menandai data class `Constellation` sebagai serializable menggunakan anotasi `@Serializable`.
- Pada line 7-9, mendeklarasikan properti `name`, `description`, dan `year` bertipe `String`.
- Pada line 11, menggunakan anotasi `@SerializedName("image_url")` untuk memetakan JSON field `image_url` ke properti `imageUrl`.
- Pada line 12, menggunakan anotasi `@SerializedName("web_url")` untuk memetakan JSON field `web_url` ke properti `webUrl`.

6. data/remote/ApiService.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.remote`.
- Pada line 3, mengimpor model `ApiResponse` dari package `data.model`.
- Pada line 4, mengimpor model `Constellation` dari package `data.model`.
- Pada line 5, mengimpor anotasi `GET` dari Retrofit untuk request HTTP GET.
- Pada line 7, mendeklarasikan interface `ApiService` untuk definisi endpoint API.
- Pada line 8, menggunakan anotasi `@GET("constellation")` untuk mendefinisikan endpoint GET ke path `constellation`.

- Pada line 9, mendeklarasikan fungsi `getConstellations()` sebagai fungsi `suspending` yang mengembalikan `ApiResponse<List<Constellation>>`.

7. `data/remote/RetrofitInstance.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.remote`.
- Pada line 3, mengimpor converter factory dari `kotlinx.serialization` untuk digunakan oleh Retrofit.
- Pada line 4, mengimpor `Json` dari `Kotlinx Serialization` sebagai konfigurasinya.
- Pada line 5, mengimpor `toMediaType` dari `okhttp3` untuk menentukan jenis media request.
- Pada line 6, mengimpor `OkHttpClient` untuk membuat client HTTP.
- Pada line 7, mengimpor `HttpLoggingInterceptor` untuk log detail request dan response HTTP.
- Pada line 8, mengimpor `Retrofit` sebagai builder HTTP client utama.
- Pada line 10, mendeklarasikan objek singleton `RetrofitInstance`.
- Pada line 12, mendeklarasikan constant `BASE_URL` sebagai base URL API.
- Pada line 14-16, menginisialisasi objek `Json` dengan pengaturan `ignoreUnknownKeys = true` untuk mengabaikan field JSON yang tidak dikenali.
- Pada line 18-22, menginisialisasi objek `OkHttpClient` dan menambahkan `HttpLoggingInterceptor` dengan level log BODY.
- Pada line 24, mendeklarasikan properti api bertipe `ApiService` yang dibuat secara lazy.

- Pada line 24-30, membangun objek Retrofit dengan baseUrl, converter untuk JSON, client, dan kemudian membuat instance ApiService.

8. data/repository/ConstellationRepository.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.repository`.
- Pada line 3, mengimpor `ConstellationDao` dari local data source.
- Pada line 4, mengimpor model `ConstellationEntity` sebagai data yang akan disimpan ke database lokal.
- Pada line 5, mengimpor `RetrofitInstance` untuk melakukan request ke API.
- Pada line 6, mengimpor `Flow` dari Kotlin untuk mengalirkan data secara reaktif.
- Pada line 7, mengimpor fungsi `flow` dari Kotlin untuk membuat objek `Flow` baru.
- Pada line 8, mengimpor `emitAll` untuk meneruskan aliran data dari DAO ke UI.
- Pada line 10, mendeklarasikan class `ConstellationRepository` dengan parameter `dao` sebagai instance dari `ConstellationDao`.
- Pada line 12, mendefinisikan fungsi `getConstellations()` yang mengembalikan `Flow<List<ConstellationEntity>>`.
- Pada line 14, menggunakan `flow {}` untuk membuat aliran data yang bisa dikoleksi.
- Pada line 15, membuat blok try-catch untuk menghindari error saat fetch data.
- Pada line 16, memanggil fungsi `getConstellations()` dari API dan menyimpannya ke variabel `response`.

- Pada line 17, melakukan pengecekan apakah status dari response bernilai "success".
- Pada line 18-25, melakukan mapping data dari response API ke `ConstellationEntity`, lalu menyimpannya ke database lokal dengan `dao.insertAll(mapped)`.
- Pada line 29-31, menangani exception dengan mencetak stack trace.
- Pada line 33, meneruskan seluruh data dari DAO dengan `emitAll(dao.getAll())`.

9. `ui/adapter/ListConstellationAdapter kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui.adapter`.
- Pada line 3-5, mengimpor class yang dibutuhkan untuk manipulasi tampilan dan pengaturan ukuran.
- Pada line 6-8, mengimpor class dari `RecyclerView` untuk pembuatan adapter dan perbandingan data.
- Pada line 9-10, mengimpor `Glide` dan `RoundedCorners` untuk pemuatan dan pengubahan tampilan gambar.
- Pada line 11, mengimpor `ConstellationEntity` sebagai model data untuk list item.
- Pada line 12, mengimpor binding layout `StarItemBinding`.
- Pada line 14-17, mendeklarasikan class `ListConstellationsAdapter` yang mewarisi `ListAdapter`, serta menerima dua lambda `onWebClick` dan `onDetailClick`.
- Pada line 19, membuat inner class `ViewHolder` untuk menyimpan objek binding per item.
- Pada line 21-24, mengimplementasikan `onCreateViewHolder` untuk meng-inflate layout dan membuat instance `ViewHolder`.

- Pada line 26-45, mengimplementasikan `onBindViewHolder` untuk menampilkan data dan menangani klik tombol.
- Pada line 29-31, mengisi `TextView` dengan nama, deskripsi, dan tahun dari item.
- Pada line 32-34, menghitung ukuran radius sudut gambar dalam satuan `dp` ke `px`.
- Pada line 36-39, memuat gambar menggunakan `Glide` dan menerapkan efek sudut membulat.
- Pada line 42-43, menangani event klik pada tombol `Web` dan tombol `Detail`.
- Pada line 47-53, membuat objek `DIFF_CALLBACK` untuk membandingkan item list berdasarkan nama dan isi secara menyeluruh.

10. ui/detail/DetailFragment.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui.detail`.
- Pada line 3-5, mengimpor class `Android` untuk pengelolaan fragment dan view.
- Pada line 6, mengimpor `navArgs` untuk mengambil argumen navigasi antar fragment.
- Pada line 7, mengimpor `Glide` untuk pemrosesan gambar.
- Pada line 8, mengimpor binding `FragmentDetailBinding`.
- Pada line 10, mendeklarasikan class `DetailFragment` yang mewarisi `Fragment`.
- Pada line 12, mendeklarasikan variabel `_binding` yang bersifat nullable.
- Pada line 14, mendeklarasikan properti binding yang menggunakan `!!` untuk memastikan `_binding` tidak null.
- Pada line 16, mendeklarasikan properti `args` untuk menerima data dari navigasi.

- Pada line 18-21, meng-inflate layout fragment menggunakan `FragmentDetailBinding`.
- Pada line 23-29, mengisi komponen UI dengan data dari argumen `constellation` dan memuat gambar menggunakan `Glide`.
- Pada line 31-33, mengatur `_binding` menjadi null saat view dihancurkan untuk mencegah memory leak.

11. ui/home/HomeFragment.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui.home`.
- Pada line 3-5, mengimpor class `Android` untuk keperluan navigasi dan `intent`.
- Pada line 6-9, mengimpor class `fragment`, `view model`, dan navigasi dari `AndroidX`.
- Pada line 13, mengimpor `LinearLayoutManager` untuk mengatur layout `RecyclerView`.
- Pada line 14, mengimpor binding layout `FragmentHomeBinding`.
- Pada line 15, mengimpor `ListConstellationsAdapter` untuk menampilkan list.
- Pada line 13-14, mengimpor `MainViewModel` dan `MainViewModelFactory` sebagai penyedia data.
- Pada line 16-17, mengimpor `Flow` dan `coroutine` untuk mengoleksi data dari `ViewModel`.
- Pada line 18, mendeklarasikan class `HomeFragment` yang mewarisi `Fragment`.
- Pada line 23, mendeklarasikan variabel `_binding` untuk binding layout fragment.
- Pada line 24, membuat properti binding dari `_binding` dengan asumsi tidak null.

- Pada line 25-26, mendeklarasikan dan menginisialisasi `viewModel` menggunakan `viewModels` dengan `MainViewModelFactory`.
- Pada line 29-31, meng-inflate layout `FragmentHomeBinding` saat fragment dibuat.
- Pada line 34-54, mengatur tampilan list dengan adapter dan menghubungkannya dengan data dari `ViewModel`.
- Pada line 35-44, menginisialisasi adapter dan mengatur navigasi tombol `Web` dan `Detail`.
- Pada line 46-47, mengatur layout manager dan adapter untuk `RecyclerView`.
- Pada line 49-53, meluncurkan coroutine untuk mengoleksi data terbaru dari `ViewModel` dan mengirimkannya ke adapter.
- Pada line 56-59, membersihkan binding saat view dihancurkan.

12. `ui/MainActivity.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui`.
- Pada line 3-8, mengimpor class `Android` untuk konfigurasi tema dan tampilan teks menu.
- Pada line 9-10, mengimpor class `AppCompatActivity`, tema, dan akses warna.
- Pada line 12, mengimpor `NavHostFragment` dari library navigasi.
- Pada line 13, mengimpor `setupActionBarWithNavController` untuk integrasi action bar.
- Pada line 14, mengimpor `R` untuk mengakses resource.
- Pada line 15, mengimpor binding layout `ActivityMainBinding`.
- Pada line 17, mendeklarasikan class `MainActivity` yang mewarisi `AppCompatActivity`.

- Pada line 19, mendeklarasikan properti binding untuk akses komponen UI.
- Pada line 24-26, menginisialisasi binding, menyetel layout sebagai konten, dan mengatur toolbar sebagai action bar.
- Pada line 28-30, mendapatkan `NavHostFragment` dan menyetel controller untuk navigasi.
- Pada line 33-42, meng-override `onCreateOptionsMenu` untuk menampilkan item menu tema dan memberi warna khusus pada teksnya.
- Pada line 45-58, meng-override `onOptionsItemSelected` untuk menangani perubahan tema antara light dan dark mode saat menu ditekan.
- Pada line 60-64, meng-override `onSupportNavigateUp` untuk menangani tombol navigasi kembali di action bar.

13. `utils/Resource.kt`

- Pada line 1, mendeklarasikan package `com.example.modul5.utils`.
- Pada line 3, mendeklarasikan class `Resource` sebagai sealed class yang mewakili tiga status: `Success`, `Error`, dan `Loading`.
- Pada line 4, data class `Success` membawa data bertipe `T`, menunjukkan permintaan berhasil.
- Pada line 5, data class `Error` membawa pesan error, tanpa data (`Nothing`).
- Pada line 6, object `Loading` digunakan saat proses sedang berjalan (juga bertipe `Resource<Nothing>`).

14. `viewmodel/MainViewModel.kt`

- Pada line 1, mendeklarasikan package `com.example.modul5.viewmodel`.
- Pada line 3-4, mengimpor `ViewModel` dan `viewModelScope` dari `AndroidX Lifecycle`.

- Pada line 5-6, mengimpor `ConstellationEntity` dan `ConstellationRepository`.
- Pada line 7-8, mengimpor `Flow` dan `coroutine` dari `kotlinx.coroutines`.
- Pada line 10, mendeklarasikan class `MainViewModel` yang menerima repository dan mewarisi `ViewModel`.
- Pada line 14, mendeklarasikan `_state` sebagai `MutableStateFlow` dengan nilai awal list kosong.
- Pada line 15, mengekspos state sebagai `StateFlow` agar hanya bisa dibaca oleh luar kelas.
- Pada line 17-19, memanggil `fetchConstellations()` saat `ViewModel` pertama kali dibuat.
- Pada line 21-26, mendefinisikan fungsi `fetchConstellations()` untuk mengambil data dari repository:
- Pada line 22, menjalankan coroutine dalam scope `ViewModel`.
- Pada line 23, memanggil `getConstellations()` dari repository.
- Pada line 24, mencetak error ke log jika terjadi kesalahan.
- Pada line 25, mengisi state dengan hasil yang dikoleksi dari flow.

15. viewmodel/MainViewModelFactory.kt

- Pada line 1, mendeklarasikan package `com.example.modul5.viewmodel`.
- Pada line 3-4, mengimpor konteks `Android` dan class untuk membuat `ViewModel`.
- Pada line 6-7, mengimpor database dan repository.
- Pada line 9, mendeklarasikan class `MainViewModelFactory` yang menerima `Context`.

- Pada line 10, meng-override fungsi `create()` dari `ViewModelProvider.Factory`.
- Pada line 11, memeriksa apakah `modelClass` cocok dengan `MainViewModel`.
- Pada line 12, mengambil DAO dari singleton `ConstellationDatabase`.
- Pada line 13, membuat instance `ConstellationRepository` menggunakan DAO.
- Pada line 14, mengembalikan `MainViewModel` dengan repository sebagai argumen.
- Pada line 16, melempar error jika class `ViewModel` tidak dikenal.

16. com.example.modul5/AndroidAPIApp.kt

- Pada line 1, mendeklarasikan package utama aplikasi: `com.example.modul5`.
- Pada line 3, mengimpor `Application` dari `Android`.
- Pada line 5, mendeklarasikan class `AndroidAPIApp` yang mewarisi `Application`, digunakan untuk inisialisasi global (misalnya Room atau library lain di masa depan).

17. res/layout/activity_main.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–25, terdapat elemen root `CoordinatorLayout` dipakai untuk mendukung layout berbasis koordinasi antar elemen, seperti toolbar dan fragment. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 5–6, terdapat `layout_width` dan `layout_height` yang di-set ke `match_parent`, artinya mengisi seluruh layar.

- Pada line 8–14, terdapat elemen `AppBarLayout` yang digunakan untuk membungkus toolbar dan mendukung scroll behavior:
- `layout_width` dan `layout_height` mengikuti layar dan konten.
- `android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"` memberi tampilan tema gelap pada AppBar.
- Pada line 10–13, terdapat `Toolbar` yang digunakan sebagai action bar:
- `layout_height="?attr/actionBarSize"` mengikuti tinggi standar action bar.
- `background="?attr/colorPrimary"` mengikuti warna utama dari tema.
- `elevation="4dp"` memberi efek bayangan di bawah toolbar.
- `app:titleTextColor="@android:color/white"` membuat warna teks judul toolbar menjadi putih.
- Pada line 16–23, terdapat `FragmentContainerView` yang digunakan sebagai tempat menampilkan fragment:
 - `android:name="androidx.navigation.fragment.NavHostFragment"` menjadikan komponen ini sebagai host fragment dari `Navigation Component`.
 - `app:navGraph="@navigation/nav_graph"` menunjuk ke file navigasi `nav_graph.xml`.
 - `app:defaultNavHost="true"` menjadikan fragment ini sebagai host utama aplikasi.
 - `android:layout_marginTop="?attr/actionBarSize"` memberi margin atas agar tidak tertutup oleh toolbar.
- Pada line 25, penutup `CoordinatorLayout`.

18. `res/layout/fragment_detail.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–25, terdapat elemen root `CoordinatorLayout` dipakai untuk mendukung layout berbasis koordinasi antar elemen, seperti toolbar dan

fragment. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.

- Pada line 5–6, terdapat `layout_width` dan `layout_height` yang di-set ke `match_parent`, artinya mengisi seluruh layar.
- Pada line 8–14, terdapat elemen `AppBarLayout` yang digunakan untuk membungkus toolbar dan mendukung scroll behavior:
 - `layout_width` dan `layout_height` mengikuti layar dan konten.
 - `android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"` memberi tampilan tema gelap pada AppBar.
- Pada line 10–13, terdapat `Toolbar` yang digunakan sebagai action bar:
 - `layout_height="?attr/actionBarSize"` mengikuti tinggi standar action bar.
 - `background="?attr/colorPrimary"` mengikuti warna utama dari tema.
 - `elevation="4dp"` memberi efek bayangan di bawah toolbar.
 - `app:titleTextColor="@android:color/white"` membuat warna teks judul toolbar menjadi putih.
- Pada line 16–23, terdapat `FragmentContainerView` yang digunakan sebagai tempat menampilkan fragment:
 - `android:name="androidx.navigation.fragment.NavHostFragment"` menjadikan komponen ini sebagai host fragment dari Navigation Component.
 - `app:navGraph="@navigation/nav_graph"` menunjuk ke file navigasi `nav_graph.xml`.
 - `app:defaultNavHost="true"` menjadikan fragment ini sebagai host utama aplikasi.

- `android:layout_marginTop="?attr/actionBarSize"` memberi margin atas agar tidak tertutup oleh toolbar.
- Pada line 25, penutup `CoordinatorLayout`.

19. `res/layout/fragment_home.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–10, terdapat elemen root `FrameLayout` sebagai kontainer utama untuk layout fragment home.
- Pada line 3, `android:id="@+id/homeFragment"` digunakan untuk mengidentifikasi fragment di kode.
- Pada line 4–5, `layout_width` dan `layout_height` di-set ke `match_parent` agar mengisi seluruh layar.
- Pada line 6, `android:padding="8dp"` memberikan jarak di seluruh sisi dalam layout.
- Pada line 8, terdapat elemen `RecyclerView` (id: `rvStars`) sebagai komponen utama untuk menampilkan daftar konstelasi secara scrollable:
 - `layout_width` dan `layout_height` di-set ke `match_parent` untuk mengisi layout.
 - `android:clipToPadding="false"` memungkinkan isi `recycler view` tetap terlihat meski melebihi batas padding.
- Pada line 10, penutup `FrameLayout`.

20. `res/layout/star_item.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–64, terdapat elemen root `LinearLayout` vertikal yang menjadi pembungkus setiap item konstelasi dalam daftar.
- Pada line 3–4, terdapat namespace `xmlns:android` dan `xmlns:app` agar atribut XML dikenali.

- Pada line 5–6, `layout_width="match_parent"` dan `layout_height="wrap_content"` membuat item menyesuaikan tinggi konten dan selebar layar.
- Pada line 7, `android:padding="8dp"` memberikan jarak dalam layout item.
- Pada line 9–59, terdapat elemen `CardView` sebagai kartu tampilan item konstelasi:
 - `cardCornerRadius="12dp"` memberikan sudut membulat.
 - `cardElevation="4dp"` memberikan efek bayangan agar tampil menonjol.
- Pada line 11–57, terdapat `LinearLayout` horizontal yang membungkus gambar dan teks.
- Pada line 13–16, terdapat `ImageView` (id: `img`) untuk menampilkan gambar konstelasi:
 - `layout_width="100dp"` dan `layout_height="150dp"` membuat gambar berukuran proporsional.
 - `scaleType="centerCrop"` untuk menjaga proporsi gambar.
 - `layout_marginEnd="8dp"` memberi jarak dari teks.
- Pada line 18–54, terdapat `LinearLayout` vertikal yang berisi informasi teks dan tombol:
 - `layout_weight="1"` agar mengambil sisa ruang horizontal.
- Pada line 20–25, terdapat `LinearLayout` horizontal untuk nama dan tahun konstelasi:
 - `gravity="center_vertical"` meratakan isi secara vertikal di tengah.
- Pada line 22, `TextView` (id: `tvName`) untuk menampilkan nama konstelasi:
 - `layout_weight="1"` membuatnya mengisi sisa ruang.
 - `textStyle="bold"` dan `textSize="16sp"` membuat teks tebal dan jelas.
- Pada line 24, `TextView` (id: `tvYear`) untuk menampilkan tahun konstelasi dengan `textSize="14sp"`.

- Pada line 27–30, `TextView (id: tvDescription)` untuk menampilkan deskripsi pendek:
 - `maxLines="3"` dan `ellipsize="end"` digunakan agar teks tidak terlalu panjang dan terpotong dengan titik-titik.
- Pada line 32–52, terdapat `LinearLayout` horizontal untuk dua tombol aksi:
- Pada line 34–38, `MaterialButton (id: btnWeb)` digunakan untuk membuka tautan ke website konstelasi:
 - `layout_weight="1"` agar lebar tombol seimbang dengan tombol lain.
 - `text="Website"` dengan `cornerRadius="50dp"` membuat tampilan membulat.
- Pada line 40–46, `MaterialButton (id: btnDetail)` untuk navigasi ke detail konstelasi:
 - `layout_marginStart="8dp"` memberi jarak dari tombol sebelumnya.
 - `text="Detail"` dan `cornerRadius="50dp"` membuat tombol tampil konsisten.
- Pada line 64, penutup `LinearLayout`.

21. `res/menu/menu_theme.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–7, terdapat elemen root `<menu>` yang digunakan untuk mendefinisikan item menu di action bar atau toolbar. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 3–6, terdapat elemen `<item>` untuk menampilkan menu switch tema:
 - `android:id="@+id/action_toggle_theme"` memberikan ID unik untuk aksi toggle.
 - `android:title="Light/Dark Mode"` menampilkan teks judul pada menu.

- `app:showAsAction="never"` artinya item tidak ditampilkan sebagai ikon di toolbar, hanya muncul di overflow menu.
- Pada line 7, penutup elemen `<menu>`.

22. `res/navigation/navigation_graph.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–19, terdapat elemen root `<navigation>` yang digunakan untuk mendefinisikan arah navigasi antar fragment menggunakan Jetpack Navigation Component. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 3, `android:id="@+id/nav_graph"` memberikan ID untuk graph ini.
- Pada line 4, `app:startDestination="@id/homeFragment"` menentukan fragment awal saat aplikasi dibuka.
- Pada line 6–10, terdapat deklarasi fragment pertama (HomeFragment):
 - `android:id="@+id/homeFragment"` memberikan ID ke fragment.
 - `android:name="com.example.modul5.ui.home.HomeFragment"` menunjuk ke class fragment.
 - `android:label="Home"` menampilkan label pada toolbar.
- Pada line 8–9, terdapat elemen `<action>` sebagai navigasi ke detail fragment:
 - `android:id="@+id/action_homeFragment_to_detailFragment"` memberi ID pada aksi.
 - `app:destination="@id/detailFragment"` menunjukkan arah navigasi ke fragment tujuan.
- Pada line 12–17, terdapat fragment `detailFragment` untuk menampilkan detail konstelasi:
 - `android:id="@+id/detailFragment"` memberikan ID ke fragment.

- `android:name="com.example.modul5.ui.detail.DetailFragment"` menunjuk ke class fragment.
- `android:label="Detail"` menampilkan label "Detail".
- Pada line 15–16, terdapat elemen `<argument>` untuk menerima data objek konstelasi dari home:
 - `android:name="constellation"` nama parameter yang dikirim.
 - `app:argType="com.example.modul5.data.local.ConstellationEntity"` menunjukkan tipe data dari argumen yang diterima.
- Pada line 19, penutup elemen `<navigation>`.

23. `res/values/themes/themes.xml`

- Pada line 1, terdapat deklarasi XML dan namespace `tools` yang digunakan untuk tooling di Android Studio.
- Pada line 2–10, terdapat deklarasi style utama `Theme.MODUL5` yang mewarisi `Theme.MaterialComponents.DayNight.NoActionBar`, artinya tema ini mendukung mode gelap dan tidak menggunakan action bar default.
- Pada line 3, `colorPrimary` di-set ke `@color/purple_500` sebagai warna utama aplikasi.
- Pada line 4, `colorPrimaryVariant` di-set ke `@color/purple_700` untuk varian warna utama.
- Pada line 5, `colorOnPrimary` di-set ke `@android:color/white` agar teks/icon di atas warna utama tetap terbaca.
- Pada line 6, `colorSecondary` di-set ke `@color/teal_200` sebagai warna sekunder aplikasi.

- Pada line 7, `colorSecondaryVariant` di-set ke `@color/teal_700` sebagai varian warna sekunder.
- Pada line 8, `colorOnSecondary` di-set ke `@android:color/white` agar teks/icon di atas warna sekunder tetap terbaca.
- Pada line 10, penutup elemen `<style>` dan `<resources>`.

24. `res/values/themes/themes.xml (night)`

- Pada line 1, terdapat deklarasi XML dan namespace `tools` yang digunakan untuk tooling di Android Studio.
- Pada line 2–10, terdapat deklarasi style utama `Theme.MODUL5` yang mewarisi `Theme.MaterialComponents.DayNight.NoActionBar`, artinya tema ini mendukung mode gelap dan tidak menggunakan action bar default.
- Pada line 3, `colorPrimary` di-set ke `@color/purple_200` sebagai warna utama aplikasi.
- Pada line 4, `colorPrimaryVariant` di-set ke `@color/purple_700` untuk varian warna utama.
- Pada line 5, `colorOnPrimary` di-set ke `@android:color/white` agar teks/icon di atas warna utama tetap terbaca.
- Pada line 6, `colorSecondary` di-set ke `@color/teal_200` sebagai warna sekunder aplikasi.
- Pada line 7, `colorSecondaryVariant` di-set ke `@color/teal_700` sebagai varian warna sekunder.
- Pada line 8, `colorOnSecondary` di-set ke `@android:color/white` agar teks/icon di atas warna sekunder tetap terbaca.
- Pada line 10, penutup elemen `<style>` dan `<resources>`.

Tautan Git

Berikut adalah tautan untuk semua source code yang telah dibuat.

[desyyn/PEMROGRAMAN-MOBILE](https://github.com/desyyn/PEMROGRAMAN-MOBILE)